BUKU MATERI

Belajar Pemrograman Python

Materi dasar, Data Analisis, Visualisasi data, dan Capstone Project

Edisi 1 · Python 3.11

```
x = 42
if x > 0:
    print("halo")
```

Python Intensive Bootcamp · 2026

Tentang buku ini

Belajar Pemrograman Python — Buku Materi adalah materi lengkap pemrograman Python dari dasar hingga analisis data, visualisasi, dan capstone project (disusun untuk bootcamp enam hari), ditambah bab persiapan lingkungan. Setiap pembahasan disertai kasus beragam, kode yang **sudah dijalankan**, dan **contoh output** asli sehingga Anda dapat membandingkan hasil di komputer sendiri.

Isi buku

- **Modul 0** — Persiapan lingkungan: Python, terminal, venv, pip, Jupyter.
- **Modul 1–2** — Dasar Python, function, module, error handling, file.
- **Modul 3** — Object-Oriented Programming.
- **Modul 4–5** — NumPy, Pandas, visualisasi, API, dan scraping.
- **Modul 6** — Capstone: empat jalur proyek analisis data.

Cara memakai

- **Ketik ulang** contoh kode, lalu ubah angka atau konteksnya — jangan hanya membaca.
- Bandingkan hasil Anda dengan blok **Output** (berlatar hijau). Blok **Pseudocode** (kuning) adalah kerangka berpikir sebelum menulis kode.
- Kerjakan latihan di akhir tiap bab sebelum melihat **Buku Solusi** (produk pendamping).
- Bagian **Troubleshooting** memuat kasus error nyata beserta perbaikannya.

Lingkungan pengujian

Seluruh kode dijalankan dengan Python 3.11 (pandas 3.0.6, numpy 2.4.6, matplotlib 3.11.2, seaborn 0.13.2). Blok yang memakai `input()` dijalankan dengan input simulasi; blok yang memanggil API yang tidak dapat diakses saat pengujian memakai respons tiruan dan diberi catatan.

Produk pendamping

- **Buku Solusi** — solusi lengkap setiap latihan dan mini project.
- **Buku kerja Jupyter** (folder `3-Buku-Kerja` pada paket) dengan pemeriksa jawaban otomatis untuk setiap latihan.

© 2026 Python Intensive Bootcamp. Seluruh hak dilindungi. Dilarang memperbanyak atau mendistribusikan ulang tanpa izin tertulis.

Daftar Isi

Modul 0 — Persiapan Lingkungan 6

1. Apa yang Akan Anda Siapkan 7
2. Python 7
3. Terminal Dasar 8
4. Virtual Environment (venv) 9
5. pip dan Paket 10
6. Jupyter Notebook 11
7. Menyiapkan Buku Kerja 12
8. Pemecahan Masalah Cepat 13
9. Alternatif Jika Instalasi Lokal Bermasalah 13
10. Checklist Akhir 13

Modul 1 — Python Fundamentals 14

1. Apa yang Akan Anda Pelajari 15
2. Persiapan Lingkungan 15
3. Variabel, Tipe Data, Operator & I/O 16
4. Control Flow 19
5. Data Structures 23
6. Best Practice — PEP 8 27
7. Mini Project 1 — Sistem Data Mahasiswa (Dasar) 28
8. Mini Project 2 — Sistem Nilai & Statistik 33
9. Troubleshooting 36
10. Solusi Latihan (Cuplikan Utama) 39
11. Checklist Sebelum Modul 2 41

Modul 2 — Functions, Modules & File Handling 42

1. Apa yang Akan Anda Pelajari 43
2. Functions — Pembahasan Dalam 43
3. Lambda, map, filter, Comprehension 47
4. Modules & Packages 49
5. Error Handling 50
6. File Handling 52
7. Mini Project 1 — Pencatat Kegiatan Akademik 54
8. Mini Project 2 — Sistem Perpustakaan Mini 56
9. Troubleshooting 60
10. Solusi Latihan (Cuplikan) 63
11. Checklist Sebelum Modul 3 64

Modul 3 — Object-Oriented Programming 65

1. Apa yang Akan Anda Pelajari 66
2. Class, Object, Attribute & Method 66
3. Inheritance (Pewarisan) 70
4. Encapsulation & Special Methods 72
5. Mini Project 1 — Manajemen Tugas Kuliah 75
6. Mini Project 2 — Perpustakaan dengan Inheritance 77
7. Troubleshooting 79
8. Solusi Latihan (Cuplikan) 83
9. Checklist Sebelum Modul 4 84

Modul 4 — NumPy & Pandas 85

1. Apa yang Akan Anda Pelajari 86
2. NumPy — Array Numerik 86
3. Pandas — Series & DataFrame 89
4. Cleaning & Operasi Lanjutan 91
5. Mini Project (5 Berjenjang) 94
6. Troubleshooting 97
7. Solusi Latihan (Cuplikan) 101
8. Checklist Sebelum Modul 5 102

Modul 5 — Visualisasi, API & Scraping 103

1. Apa yang Akan Anda Pelajari 104
2. Matplotlib — Visualisasi Dasar 104
3. Seaborn — Visualisasi Statistik 109
4. Bekerja dengan API (requests) 113
5. Scraping & arXiv API 115
6. Mini Project 117
7. Troubleshooting 131
8. Solusi Latihan (Cuplikan) 135
9. Checklist Sebelum Modul 6 136

Modul 6 — Capstone Project 137

1. Apa Itu Capstone di Modul Ini 138
2. Tujuan Pembelajaran 138
3. Memilih Topik 138
4. Deliverable Wajib 139
5. Alur Kerja yang Disarankan 139
5b. Agenda Sehari & Alokasi Waktu 140
6. Struktur Folder Proyek 141
7. Template Dokumentasi Capstone 141
8. Skeleton Kode Capstone (Pola Umum) 142

9. Empat Jalur Capstone (Pilih Satu) 148
10. Checklist Kualitas (Self-Review) 150
11. Troubleshooting Capstone 150
11b. Demonstrasi Error — Output Nyata 150
12. Setelah Capstone 152

0

Modul 0 — Persiapan Lingkungan

Python, terminal, virtual environment (venv), pip, dan Jupyter ·
Estimasi: ± 75 menit · Wajib sebelum Modul 1

1. Apa yang Akan Anda Siapkan
2. Python
3. Terminal Dasar
4. Virtual Environment (venv)
5. pip dan Paket
6. Jupyter Notebook
7. Menyiapkan Buku Kerja
8. Pemecahan Masalah Cepat
9. Alternatif Jika Instalasi Lokal Bermasalah
10. Checklist Akhir

> **Catatan:** perintah pada modul ini berlaku untuk Windows, macOS, dan Linux; perbedaan antarsistem ditandai jelas. Bagian **Contoh output** diambil dari Linux dan nomor versinya dapat berbeda di komputer Anda.

1. Apa yang Akan Anda Siapkan

Setelah modul ini selesai, Anda memiliki:

- Python 3.11 atau lebih baru yang terpasang dan dikenali terminal.
- Folder proyek `bootcamp/` dengan virtual environment `.venv`.
- Paket yang dibutuhkan (pandas, numpy, matplotlib, dst.) di dalam venv.
- Jupyter Notebook yang berjalan dari venv tersebut.
- Buku kerja Hari 1 yang bisa dibuka dan dijalankan.

> **TIPS**
> **Aturan emas**
>
> Setiap bagian diakhiri **cek**. Jangan lanjut sebelum cek berhasil — masalah lingkungan yang dibiarkan akan menumpuk dan membuat bagian berikutnya membingungkan.

2. Python

2.1 Cek: apakah Python sudah ada?

Buka terminal lalu ketik:

```
# Windows
py --version
python --version

# macOS / Linux
python3 --version
```

CONTOH OUTPUT:

```
Python 3.11.9
```

Yang dicari adalah **Python 3.11** atau lebih baru (3.12 dan 3.13 juga baik). Jika muncul pesan *not recognized* (Windows) atau *command not found* (macOS/Linux), Python belum terpasang atau belum masuk **PATH**.

2.2 Instal Python

Sistem	Langkah
Windows	Unduh installer dari python.org/downloads, jalankan, **centang "Add python.exe to PATH"**, pilih *Install Now*.
macOS	Unduh installer dari python.org/downloads, atau gunakan Homebrew: `brew install python`.
Linux (Debian/Ubuntu)	`sudo apt install python3 python3-venv python3-pip`
Linux (Fedora)	`sudo dnf install python3`

Setelah instal, **tutup dan buka kembali terminal**, lalu ulangi perintah cek versi.

> **PERHATIAN**
> **Kotak "Add python.exe to PATH"** pada installer Windows adalah sumber masalah nomor satu. Jika terlewat, jalankan ulang installer, pilih *Modify*, lalu centang opsi PATH.

2.3 Jebakan: Python terpasang tapi tidak ketemu

Gejala	Penyebab	Solusi
Windows: `python` tidak dikenali	belum centang *Add to PATH*	pakai `py` (Python launcher) atau jalankan ulang installer → *Modify* → centang PATH
Windows membuka Microsoft Store	alias bawaan Windows	instal dari python.org; matikan alias di *Manage app execution aliases*
macOS/Linux: `python` tidak ada	hanya tersedia `python3`	pakai `python3` (di dalam venv, `python` otomatis ada)
Versi lama (mis. 3.8)	beberapa Python terpasang	panggil versi baru secara eksplisit: `python3.11` atau `py -3.11`

3. Terminal Dasar

3.1 Perintah yang dipakai

Tujuan	Windows (PowerShell)	macOS / Linux
Di folder mana saya?	`pwd` atau `cd`	`pwd`
Lihat isi folder	`dir` atau `ls`	`ls`
Buat folder	`mkdir bootcamp`	`mkdir bootcamp`
Masuk folder	`cd bootcamp`	`cd bootcamp`
Naik satu level	`cd ..`	`cd ..`

Terminal Windows yang disarankan: **PowerShell** atau aplikasi **Terminal** (Windows 11). Di VS Code: menu *Terminal → New Terminal*.

3.2 Buat folder proyek

```
mkdir bootcamp
cd bootcamp
pwd
```

CONTOH OUTPUT:

```
/home/user/bootcamp
```

Semua langkah berikutnya dijalankan **di dalam folder ini**.

4. Virtual Environment (venv)

4.1 Mengapa perlu venv?

Satu komputer sering dipakai untuk banyak proyek, dan tiap proyek bisa membutuhkan versi paket yang berbeda (misalnya pandas 2.2 untuk proyek lama dan pandas 3.0 untuk proyek baru). Tanpa isolasi, memperbarui satu paket untuk proyek A bisa merusak proyek B.

Virtual environment adalah folder berisi salinan Python dan paket khusus untuk satu proyek. Python sistem tidak disentuh; proyek mudah dihapus dan diulang di komputer lain.

4.2 Membuat dan mengaktifkan venv

Langkah 1 — buat (sekali saja per proyek):

```
python -m venv .venv      # macOS/Linux: python3 -m venv .venv
```

Langkah 2 — aktifkan (setiap membuka terminal baru):

Terminal	Perintah
Windows PowerShell	`.venv\Scripts\Activate.ps1`
Windows CMD	`.venv\Scripts\activate.bat`
macOS / Linux	`source .venv/bin/activate`

Berhasil bila prompt berawalan `(.venv)`, misalnya:

```
(.venv) PS C:\bootcamp>
(.venv) user@pc:~/bootcamp$
```

4.3 Jebakan Windows: skrip dinonaktifkan

Di PowerShell, aktivasi bisa gagal dengan pesan berikut:

```
Activate.ps1 cannot be loaded because running scripts is disabled on this system.
```

Solusi (pilih salah satu):

```powershell
# A. izinkan skrip lokal untuk akun Anda (cukup sekali)
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned

# B. atau pakai CMD
.venv\Scripts\activate.bat
```

4.4 Cek venv sudah aktif

```
python -c "import sys; print(sys.prefix)"
python -m pip --version
deactivate
```

CONTOH OUTPUT:

```
/home/user/bootcamp/.venv
pip 24.0 from /home/user/bootcamp/.venv/lib/python3.11/site-packages/pip (python 3.11)
```

Baris pertama harus menunjuk ke folder `.venv` di dalam `bootcamp`. `deactivate` keluar dari venv; aktifkan lagi sebelum melanjutkan.

5. pip dan Paket

5.1 Perintah dasar

```
python -m pip install --upgrade pip
pip install pandas
pip list
pip freeze > requirements.txt
```

PERHATIAN
Pastikan `(.venv)` aktif sebelum `pip install`. Jika `pip` tidak dikenali, gunakan `python -m pip` — perintah ini selalu memakai Python yang sedang aktif.

- `pip list` menampilkan paket di venv ini saja.
- `pip freeze > requirements.txt` menyimpan daftar paket beserta versinya.

5.2 Satu perintah untuk semua kebutuhan bootcamp

File `requirements.txt` ada di paket belajar yang Anda terima (folder `Belajar-Pemrograman-Python`) dan berisi:

```
notebook
numpy
pandas
matplotlib
seaborn
scipy
openpyxl
requests
beautifulsoup4
feedparser
```

Pasang semuanya sekaligus setelah paket diekstrak (lihat bagian 7); perintahnya butuh internet dan beberapa menit:

```
pip install -r Belajar-Pemrograman-Python/requirements.txt
```

6. Jupyter Notebook

6.1 Menjalankan Jupyter

Dengan venv aktif (jika Jupyter belum terpasang: `pip install notebook`):

```
jupyter notebook
```

Perintah ini memulai server lokal (`http://localhost:8888`) dan membuka halaman Jupyter di browser. Biarkan terminal tetap terbuka selama bekerja; berhenti dengan `Ctrl+C` di terminal. Jika browser tidak terbuka sendiri, salin alamat `http://localhost:8888/...` lengkap dengan token dari terminal.

6.2 Pintasan notebook

Aksi	Pintasan
Jalankan sel dan pindah ke bawah	`Shift + Enter`
Mode perintah / mode edit	`Esc` / `Enter`
Sisipkan sel di atas / bawah	`A` / `B` (mode perintah)
Hapus sel	`D D`
Ubah ke Markdown / kode	`M` / `Y`

Jika kode berperilaku aneh, gunakan menu **Kernel → Restart Kernel and Run All Cells**.

6.3 Pastikan kernel memakai venv

Jupyter yang dijalankan dari venv otomatis memakai venv itu. Untuk mendaftarkan venv sebagai kernel yang bisa dipilih:

```
pip install ipykernel
python -m ipykernel install --user --name bootcamp --display-name "Python (bootcamp)"
```

Lalu pilih **Kernel → Change kernel → Python (bootcamp)**. Cek di sebuah sel:

```
import sys
print(sys.executable)
```

Hasilnya harus menunjuk ke folder `.venv`.

7. Menyiapkan Buku Kerja

Paket belajar Anda berupa satu berkas ZIP, `Belajar-Pemrograman-Python.zip`, yang berisi:

```
Belajar-Pemrograman-Python/
├── MULAI-DI-SINI.txt
├── 1-Buku-Materi.pdf
├── 2-Buku-Solusi.pdf
├── 3-Buku-Kerja/          hari1 … hari6 (.ipynb)
└── requirements.txt
```

Langkah:

1. **Ekstrak** ZIP ke dalam folder `bootcamp` yang dibuat pada bagian 3.2, sehingga terbentuk `bootcamp/Belajar-Pemrograman-Python/`. (Windows: klik kanan → *Extract All*; macOS: klik dua kali; Linux: `unzip Belajar-Pemrograman-Python.zip`.)
2. Buka terminal di folder `bootcamp` dan **aktifkan venv** (bagian 4.2) hingga prompt berawalan `(.venv)`.
3. **Pasang paket**, lalu **jalankan Jupyter** pada folder buku kerja:

```
pip install -r Belajar-Pemrograman-Python/requirements.txt
jupyter notebook Belajar-Pemrograman-Python/3-Buku-Kerja
```

4. Di Jupyter, buka `hari1-python-fundamentals.ipynb` dan jalankan **sel pertama (setup)** lebih dulu.

> **TIPS**
> **Cara memakai buku kerja**
>
> Tulis jawaban Anda di sel yang ditandai `# TULIS KODE ANDA DI SINI`, lalu jalankan sel **Periksa jawaban** di bawahnya: tanda ✅ berarti benar, ❌ berarti belum tepat. Bandingkan dengan **Buku Solusi** (`2-Buku-Solusi.pdf`) setelah mencoba sendiri.

8. Pemecahan Masalah Cepat

Pesan / gejala	Penyebab	Solusi
`pip` tidak dikenali	venv belum aktif / PATH	`python -m pip ...` setelah mengaktifkan venv
`ModuleNotFoundError: pandas`	paket di venv lain / kernel salah	aktifkan venv, `pip install` , pilih kernel venv
`jupyter` tidak dikenali	notebook belum terpasang di venv	`pip install notebook`
Port 8888 sibuk	Jupyter lain masih berjalan	`jupyter notebook --port 8889`
running scripts is disabled	kebijakan PowerShell	lihat bagian 4.3 / pakai CMD
Instalasi pip sangat lambat atau gagal	koneksi / proxy	ulangi; cek koneksi; coba jaringan lain

9. Alternatif Jika Instalasi Lokal Bermasalah

Opsi	Kelebihan	Catatan
VS Code + ekstensi Python & Jupyter	editor, terminal, dan notebook dalam satu jendela	tetap butuh Python dan venv di komputer
Google Colab	tanpa instal, berjalan di browser	butuh akun Google dan internet; unggah notebook dari folder `3-Buku-Kerja` ; berkas hilang saat sesi berakhir

10. Checklist Akhir

- [ ] `python --version` (atau `python3 --version`) menampilkan 3.11 atau lebih baru.
- [ ] Folder `bootcamp/` dan venv `.venv` ada; prompt berawalan `(.venv)` saat aktif.
- [ ] `pip install -r requirements.txt` selesai tanpa error; `pip list` menampilkan pandas.
- [ ] `jupyter notebook` berjalan dari venv; kernel benar.
- [ ] Notebook Hari 1 terbuka dan sel pertamanya berjalan tanpa error.

Selanjutnya: **Modul 1 — Python Fundamentals**.

1

Modul 1 — Python Fundamentals

Versi lengkap — pembahasan dalam + kasus beragam · Estimasi: ± 6 jam · Level: Pemula · Prasyarat: tidak ada

1. Apa yang Akan Anda Pelajari
2. Persiapan Lingkungan
3. Variabel, Tipe Data, Operator & I/O
4. Control Flow
5. Data Structures
6. Best Practice — PEP 8
7. Mini Project 1 — Sistem Data Mahasiswa (Dasar)
8. Mini Project 2 — Sistem Nilai & Statistik
9. Troubleshooting
10. Solusi Latihan (Cuplikan Utama)
11. Checklist Sebelum Modul 2

> **Catatan pengujian:** seluruh blok kode pada modul ini dijalankan (Python 3.11). Bagian **Output** adalah hasil eksekusi nyata. Blok yang memakai `input()` dijalankan dengan input simulasi yang ditulis di samping label output. Blok yang hanya berisi definisi/komentar diberi keterangan, dan "Uji pelengkap" ditambahkan agar function/class pada blok tersebut terbukti berjalan. Kasus error pada bagian Troubleshooting dijalankan nyata di "Demonstrasi error".

1. Apa yang Akan Anda Pelajari

- Memakai variabel, tipe data, casting, dan operator dengan tepat — termasuk kasus yang sering salah.
- Membangun logika program dengan if, for, while, break/continue, dan nested loop.
- Memilih dan memanipulasi list, tuple, dict, set — serta list of dict untuk data mahasiswa.
- Menulis kode yang lebih rapi (PEP 8) dan menyelesaikan 2 mini project berbasis menu.

> CATATAN
> **Cara memakai modul ini**
>
> Jangan hanya membaca. Ketik ulang contoh, lalu ubah angkanya / konteksnya.
>
> Kerjakan latihan sebelum membuka bagian solusi di akhir.
>
> Setiap topik punya pembahasan + kasus beragam + jebakan — jangan loncat langsung ke project.
>
> Simpan file latihan di portfolio/modul_01/.

2. Persiapan Lingkungan

Cek Python terpasang, lalu siapkan editor (VS Code, Cursor, atau Jupyter). Untuk uji cepat, REPL juga cukup.

```
python --version
# atau: python3 --version

python
>>> print("Halo, Python!")
>>> exit()
```

> TIPS
> **Tips**
>
> Google Colab bisa dipakai jika environment lokal bermasalah.
>
> Biasakan satu folder khusus modul ini agar file tidak bercampur.

3. Variabel, Tipe Data, Operator & I/O

3.1 Variabel — pembahasan

Variabel adalah nama yang merujuk ke nilai di memori. Di Python Anda tidak wajib menulis tipe saat membuat variabel; tipe mengikuti nilainya. Yang penting: nama jelas, dan Anda sadar nilai apa yang sedang dipegang.

Penamaan yang baik mengurangi bug. `n` atau `x` hanya cocok di konteks sangat sempit (misalnya indeks loop). Untuk data akademik, lebih aman memakai `nama`, `nim`, `ipk`, `total_sks`.

Kasus beragam

```
# 1) Data diri mahasiswa
nama = "Siti Aminah"
nim = "21012001"
ipk = 3.75
aktif = True
print(nama, nim, ipk, aktif)

# 2) Multiple assignment
matkul, sks, nilai = "Kalkulus", 3, 85
print(matkul, sks, nilai)

# 3) Pertukaran nilai (tanpa variabel bantu eksplisit)
a, b = 10, 20
a, b = b, a
print(a, b)  # 20 10

# 4) Konstanta konvensi (huruf besar — kesepakatan, bukan syntax)
MAX_SKS = 24
BIAYA_PER_SKS = 150000

# 5) Menghitung dari variabel lain
sks_diambil = 20
estimasi_biaya = sks_diambil * BIAYA_PER_SKS
print(estimasi_biaya)
```

OUTPUT:

```
Siti Aminah 21012001 3.75 True
Kalkulus 3 85
20 10
3000000
```

PERHATIAN

Jebakan umum

Memakai nama sebelum di-assign → NameError.

Menimpa nama built-in (list, str, input) membuat fungsi asli “hilang” di sesi itu.

Nomor HP / NIM sering lebih aman sebagai string agar nol di depan tidak hilang.

3.2 Tipe Data & Casting — pembahasan

Tipe menentukan operasi apa yang masuk akal. Menjumlah dua angka berbeda dengan menggabungkan dua teks. Casting mengubah tipe secara eksplisit; gagal jika isi tidak kompatibel.

```
print(type(21))
print(type(3.14))
print(type("Python"))
print(type(True))

# Casting yang sering dipakai
print(int("42"))
print(float("3.5"))
print(str(42))
print(int(3.9))      # 3 — truncasi, bukan pembulatan
print(bool(0), bool(1), bool(""), bool("teks"))
# Kasus form: input selalu string
# usia = int(input("Usia: "))  # gagal jika user ketik "dua puluh"

# Kasus nilai desimal dalam string
# int("3.5")  → ValueError; pakai float dulu lalu int jika perlu
print(int(float("3.5")))

# Kasus nomor identitas
nim = "00123"
print(nim, int(nim))  # int menghilangkan leading zero di tampilan numerik
```

OUTPUT:

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
42
3.5
42
3
False True False True
3
00123 123
```

3.3 Operator — pembahasan

Operator aritmatika menghasilkan nilai baru. Operator perbandingan menghasilkan boolean. Operator logika menggabungkan kondisi. Prioritas penting: eksponen lebih dulu, lalu kali/bagi, lalu tambah/kurang; pakai tanda kurung jika ragu.

```python
a, b = 10, 3
print(a + b, a - b, a * b, a / b)   # / selalu float di Python 3
print(a // b, a % b, a ** b)        # 3 1 1000

# Kasus genap/ganjil dan kelipatan
nilai = 85
print(nilai % 2 == 0)
print(nilai % 5 == 0)

# Kasus rata-rata
n1, n2, n3 = 80, 90, 70
print((n1 + n2 + n3) / 3)

# Perbandingan dan logika
ipk = 3.4
sks = 20
print(ipk >= 3.0 and sks >= 18)
print(ipk < 2.0 or sks < 12)
print(not aktif) if False else print("ok")

# Prioritas
print(2 + 3 * 4)    # 14
print((2 + 3) * 4)  # 20
```

OUTPUT:

```
13 7 30 3.3333333333333335
3 1 1000
False
True
80.0
True
False
ok
14
20
```

3.4 String & I/O — pembahasan

`input()` selalu mengembalikan str. Untuk hitung-hitungan, cast dulu. Untuk menampilkan, f-string biasanya paling jelas. String bisa digabung (`+`) atau diulang (`*`), tetapi `+` antara str dan int akan error.

```python
# Kasus sapaan formal
nama = "Rina"
print(f"Halo, {nama}. Selamat belajar.")

# Kasus struk sederhana
item, harga, qty = "Fotokopi", 500, 12
total = harga * qty
print(f"{item}: {qty} x {harga} = {total}")

# Kasus laporan satu baris
mk, huruf = "Basis Data", "A"
print(f"MK {mk} → grade {huruf}")

# Kasus input numerik
# berat = float(input("Berat (kg): "))
# tinggi = float(input("Tinggi (m): "))
# bmi = berat / (tinggi ** 2)

# Concat vs f-string
print("Nilai: " + str(90))
print("Nilai: {}".format(90))
```

OUTPUT:

```
Halo, Rina. Selamat belajar.
Fotokopi: 12 x 500 = 6000
MK Basis Data → grade A
Nilai: 90
Nilai: 90
```

3.5 Latihan — Variabel & I/O

Kerjakan tanpa melihat solusi di akhir modul. Setiap soal punya konteks berbeda.

1. Form singkat: minta nama dan tiga nilai, cetak rata-rata 2 desimal.
2. Konversi suhu C→F (F = C×9/5+32) dari input user.
3. Hitung estimasi biaya kuliah: input SKS × biaya per SKS (konstanta).
4. Tukar dua nilai variabel tanpa menulis angka literal di baris tukar.
5. Luas & keliling persegi panjang dari input panjang-lebar.
6. Deteksi genap/ganjil dari satu bilangan (pakai %).
7. Hitung BMI dari berat & tinggi; cetak juga kategori kasar (mis. <18.5 underweight).
8. Format struk 2 item berbeda (nama, harga, qty) dan total akhir.
9. Konversi jam ke menit dan detik (input jam desimal atau jam utuh).
10. Input nilai 0–100; cetak True/False apakah masuk rentang valid (tanpa if dulu boleh).

4. Control Flow

4.1 if / elif / else — pembahasan

Percabangan memilih blok kode berdasarkan kondisi. Urutan `elif` penting: kondisi dicek dari atas; yang pertama True dijalankan, sisanya dilewati. Rentang nilai (grade) harus dipikirkan agar tidak tumpang tindih atau bolong.

```python
# Kasus grade huruf
nilai = 78
if nilai >= 85:
    grade = "A"
elif nilai >= 70:
    grade = "B"
elif nilai >= 60:
    grade = "C"
elif nilai >= 50:
    grade = "D"
else:
    grade = "E"
print(grade)

# Kasus kelayakan beasiswa (kondisi majemuk)
ipk, penghasilan = 3.6, 3_000_000
if ipk >= 3.5 and penghasilan < 4_000_000:
    print("Kandidat beasiswa")
else:
    print("Belum memenuhi kriteria")

# Kasus status kehadiran
persen = 78
if persen >= 80:
    print("Boleh ujian")
elif persen >= 70:
    print("Boleh ujian dengan syarat")
else:
    print("Tidak boleh ujian")

# Ternary untuk ekspresi pendek
status = "lulus" if nilai >= 60 else "mengulang"
print(status)
```

OUTPUT:

```
B
Kandidat beasiswa
Boleh ujian dengan syarat
lulus
```

PERHATIAN

Miskonsepsi

Menulis if nilai >= 70 and < 85: → salah syntax; harus nilai < 85.

Mengandalkan urutan salah: if nilai >= 60 sebelum >= 85 membuat A tidak pernah tercapai jika tidak rapi.

Nested if dalam: berguna, tapi jika terlalu dalam, pertimbangkan merapikan kondisi.

4.2 for loop — pembahasan

`for` dipakai saat Anda mengiterasi sesuatu yang diketahui (range, list, string). Akumulator (total, hitung) adalah pola paling sering: mulai dari 0 atau list kosong, lalu update tiap putaran.

```python
# Kasus jumlah 1..n
n = 10
total = 0
for i in range(1, n + 1):
    total += i
print(total)

# Kasus rata-rata dari list nilai
nilai_list = [80, 70, 90, 60, 85]
jumlah = 0
for v in nilai_list:
    jumlah += v
print(jumlah / len(nilai_list))

# Kasus hitung yang lulus
lulus = 0
for v in nilai_list:
    if v >= 60:
        lulus += 1
print("Lulus:", lulus)

# Kasus enumerate (nomor urut MK)
mk = ["Kalkulus", "Fisika", "Python"]
for i, nama in enumerate(mk, start=1):
    print(f"{i}. {nama}")

# Kasus iterasi string
for ch in "NIM":
    print(ch)
```

OUTPUT:

```
55
77.0
Lulus: 5
1. Kalkulus
2. Fisika
3. Python
N
I
M
```

4.3 while loop — pembahasan

`while` cocok saat Anda belum tahu berapa kali mengulang — misalnya menunggu input valid. Bahaya utamanya: kondisi yang tidak pernah False (loop tak hingga). Pastikan ada perubahan variabel di dalam loop.

```python
# Kasus validasi angka 1–100
# while True:
#     x = int(input("Nilai 1-100: "))
#     if 1 <= x <= 100:
#         break
#     print("Di luar rentang.")

# Kasus menu sentinel
# pilihan = ""
# while pilihan != "0":
#     print("1. Lihat  0. Keluar")
#     pilihan = input("Pilih: ")

# Kasus akumulator dengan while
sisa = 5
while sisa > 0:
    print("Sisa kesempatan:", sisa)
    sisa -= 1
```

OUTPUT:

```
Sisa kesempatan: 5
Sisa kesempatan: 4
Sisa kesempatan: 3
Sisa kesempatan: 2
Sisa kesempatan: 1
```

4.4 break, continue, nested — pembahasan

`break` keluar dari loop terdekat. `continue` lompat ke iterasi berikutnya. Nested loop dipakai untuk pola 2 dimensi (tabel, grid), tetapi kompleksitas naik cepat — jangan memaksa nested jika satu loop + if sudah cukup.

```python
# Kasus cari nilai pertama yang < 60 lalu berhenti
nilai_list = [80, 90, 55, 70]
for v in nilai_list:
    if v < 60:
        print("Ketemu nilai rendah:", v)
        break

# Kasus skip nilai None / kosong (simulasi)
data = [80, None, 70, None, 90]
for v in data:
    if v is None:
        continue
    print(v)

# Kasus tabel perkalian kecil
for i in range(1, 4):
    for j in range(1, 4):
        print(i * j, end="\t")
    print()

# Kasus pola bintang
for baris in range(1, 5):
    print("*" * baris)
```

OUTPUT:

```
Ketemu nilai rendah: 55
80
70
90
1       2       3
2       4       6
3       6       9
*
**
***
****
```

4.5 Latihan — Control Flow

1. Grade huruf dari input nilai (rentang A–E lengkap).
2. Cetak bilangan genap 1…50.
3. Kasir: diskon 10% jika total ≥ 100000; tampilkan bayar.
4. Tebak angka 1–10 dengan while sampai benar.
5. Pola bintang n baris (n dari input).
6. Dari list nilai, hitung berapa yang A (≥85) dan berapa yang tidak lulus (<60).
7. Validasi input IPK 0–4; ulangi sampai valid.
8. Menu 3 opsi (tampilkan teks berbeda) + keluar.
9. Faktorial n dengan loop (bukan rekursi).
10. Cari angka maksimum di list tanpa pakai max().
11. Tabel perkalian 1…5 dalam bentuk baris-kolom.
12. Simulasi: saldo awal, beberapa transaksi (tambah/kurang), tolak jika saldo tidak cukup.

5. Data Structures

5.1 List — pembahasan

List menyimpan urutan nilai yang bisa berubah (mutable). Indeks mulai dari 0; indeks negatif dari belakang. Slicing menghasilkan list baru. Method seperti `append` mengubah list di tempat. List cocok untuk daftar MK, deret nilai, atau antrian sederhana.

```python
nilai = [80, 90, 75, 88]
print(nilai[0], nilai[-1])
print(nilai[1:3])
print(nilai[::-1])

nilai.append(95)
nilai.insert(1, 70)
nilai.remove(75)
terakhir = nilai.pop()
print(sorted(nilai))
print(sum(nilai), len(nilai), sum(nilai)/len(nilai))

# Kasus daftar MK
mk = ["Kalkulus", "Fisika"]
mk.append("Python")
print("Python" in mk)

# Kasus filter nilai >= 70 (buat list baru)
tinggi = []
for v in nilai:
    if v >= 70:
        tinggi.append(v)
print(tinggi)
```

OUTPUT:

```
80 88
[90, 75]
[88, 75, 90, 80]
[70, 80, 88, 90]
328 4 82.0
True
[80, 70, 90, 88]
```

5.2 Tuple — pembahasan

Tuple mirip list tetapi tidak bisa diubah setelah dibuat. Cocok untuk data yang seharusnya tetap (pasangan koordinat, rekaman singkat). Unpacking membuat kode lebih jelas.

```python
koordinat = (-6.2, 106.8)
lat, lon = koordinat
print(lat, lon)

# Kasus pasangan NIM-nama
rekaman = ("21012001", "Siti")
nim, nama = rekaman
print(nim, nama)

# Return multi-nilai natural (fungsi dibahas Modul 2)
def min_max(data):
    return min(data), max(data)
print(min_max([3, 1, 4, 2]))
```

OUTPUT:

```
-6.2 106.8
21012001 Siti
(1, 4)
```

5.3 Dictionary — pembahasan

Dict menyimpan pasangan key → value. Key harus unik dan bertipe immutable (str/int/tuple). Lookup by key biasanya lebih jelas daripada mencari di list of pair. Method `.get` aman saat key bisa absen.

```
# Kasus profil mahasiswa
mhs = {"nama": "Budi", "nim": "2201", "ipk": 3.6}
print(mhs["nama"])
print(mhs.get("prodi", "belum diisi"))
mhs["prodi"] = "Informatika"
mhs["ipk"] = 3.7
for k, v in mhs.items():
    print(k, "→", v)

# Kasus rekap nilai per MK
nilai_mk = {"Kalkulus": 80, "Python": 90, "Fisika": 70}
print(sum(nilai_mk.values()) / len(nilai_mk))

# Kasus frekuensi huruf grade
grade = ["A", "B", "A", "C", "B", "A"]
freq = {}
for g in grade:
    freq[g] = freq.get(g, 0) + 1
print(freq)

# Nested sederhana
data = {"2201": {"nama": "Ani", "ipk": 3.5}, "2202": {"nama": "Budi", "ipk": 3.1}}
print(data["2201"]["nama"])
```

OUTPUT:

```
Budi
belum diisi
nama → Budi
nim → 2201
ipk → 3.7
prodi → Informatika
80.0
{'A': 3, 'B': 2, 'C': 1}
Ani
```

5.4 Set — pembahasan

Set menyimpan nilai unik tanpa urutan tetap. Operasi himpunan (irisan, gabungan, selisih) membuatnya cocok untuk anggota kelas, tag, atau deduplikasi.

```
peserta_a = {"Ani", "Budi", "Cici"}
peserta_b = {"Budi", "Dodi", "Ani"}
print(peserta_a | peserta_b)  # gabungan
print(peserta_a & peserta_b)  # irisan
print(peserta_a - peserta_b)  # hanya di A

# Kasus deduplikasi list
raw = [1, 2, 2, 3, 1, 4]
unik = list(set(raw))
print(unik)
```

OUTPUT:

```
{'Cici', 'Ani', 'Dodi', 'Budi'}
{'Budi', 'Ani'}
{'Cici'}
[1, 2, 3, 4]
```

5.5 List of dict — pembahasan (penting)

Sebelum database atau Pandas, banyak program menyimpan data tabular sebagai list of dictionary. Satu dict = satu baris entitas (mahasiswa, tugas, buku). Pola ini menjadi dasar Mini Project modul ini.

```python
mahasiswa = [
    {"nama": "Ani", "nim": "2201", "ipk": 3.5, "prodi": "IF"},
    {"nama": "Budi", "nim": "2202", "ipk": 3.1, "prodi": "SI"},
    {"nama": "Cici", "nim": "2203", "ipk": 3.8, "prodi": "IF"},
]

# Tampilkan semua
for m in mahasiswa:
    print(m["nim"], m["nama"], m["ipk"])

# Cari by NIM
cari = "2202"
ketemu = None
for m in mahasiswa:
    if m["nim"] == cari:
        ketemu = m
        break
print(ketemu)

# Filter prodi IF
if_only = [m for m in mahasiswa if m["prodi"] == "IF"]
print(if_only)

# IPK tertinggi
tertinggi = mahasiswa[0]
for m in mahasiswa:
    if m["ipk"] > tertinggi["ipk"]:
        tertinggi = m
print(tertinggi["nama"], tertinggi["ipk"])
```

OUTPUT:

```
2201 Ani 3.5
2202 Budi 3.1
2203 Cici 3.8
{'nama': 'Budi', 'nim': '2202', 'ipk': 3.1, 'prodi': 'SI'}
[{'nama': 'Ani', 'nim': '2201', 'ipk': 3.5, 'prodi': 'IF'}, {'nama': 'Cici', 'nim': '2203', 'ipk':
3.8, 'prodi': 'IF'}]
Cici 3.8
```

5.6 Kapan memakai struktur yang mana

Struktur	Pilih jika…	Contoh kasus
List	Perlu urutan & bisa berubah	Daftar nilai, antrian MK
Tuple	Data tetap / pasangan singkat	Koordinat, (nim, nama)
Dict	Perlu label / lookup by key	Profil mhs, map MK→nilai
Set	Keunikan / operasi himpunan	Peserta unik, irisan kelas
List[dict]	Banyak record sejenis	Tabel mahasiswa di memori

5.7 Latihan — Data Structures

1. List MK: tambah, hapus, cetak jumlah.
2. Dict profil diri; cetak semua item.
3. Dari list nilai, hitung jumlah lulus (≥60).
4. Irisan dua set hobi.
5. List of dict mahasiswa: cetak nama dengan IPK tertinggi.
6. Update IPK satu mahasiswa by NIM di list of dict.
7. Filter semua mahasiswa prodi tertentu.
8. Hitung frekuensi angka di list (pakai dict).
9. Gabungkan dua list lalu buang duplikat (set).
10. Tuple (panjang, lebar) → hitung luas (unpacking).
11. Hapus mahasiswa by NIM dari list of dict (buat list baru atau hapus hati-hati).
12. Rata-rata IPK hanya untuk prodi IF dari list of dict.

6. Best Practice — PEP 8

PEP 8 adalah panduan gaya resmi Python. Bukan tentang “jalan atau tidak”, tetapi keterbacaan — penting saat kode dibaca orang lain atau Anda sendiri beberapa bulan kemudian.

- Indentasi 4 spasi; jangan campur tab.
- Nama variabel/function: snake_case; class nanti: PascalCase.
- Spasi di sekitar operator: `x = a + b`.
- Baris tidak terlalu panjang; pecah argumen jika perlu.
- Import di bagian atas file.
- Nama bermakna; hindari `a1`, `tmp2` tanpa alasan.

```
# Kurang baik
def H(x,y):
  return x+y

# Lebih baik
def hitung_jumlah(angka_satu, angka_dua):
    return angka_satu + angka_dua
```

OUTPUT:

```
(tidak ada output)
```

7. Mini Project 1 — Sistem Data Mahasiswa (Dasar)

Deskripsi

Program menu untuk mengelola list of dict mahasiswa di memori: tambah, tampil, cari, hapus, IPK tertinggi. Tidak perlu file/database.

Fitur wajib

1. Tambah mahasiswa (nama, NIM, prodi, IPK).
2. Tampilkan semua.
3. Cari by NIM.
4. Hapus by NIM.
5. Tampilkan IPK tertinggi.
6. Keluar.

Urutan kerjakan

- Siapkan list kosong `mahasiswa = []`.
- Buat fungsi tampil_menu, tambah, tampilkan, cari, hapus, ipk_tertinggi.
- Hubungkan ke loop `while True` + `input` pilihan.
- Uji setiap fitur manual sebelum lanjut.

Skeleton

```
mahasiswa = []

def tampil_menu():
    print("\n1. Tambah  2. Tampilkan  3. Cari")
    print("4. Hapus   5. IPK Tertinggi  6. Keluar")

def tambah():
    nama = input("Nama: ")
    nim = input("NIM: ")
    prodi = input("Prodi: ")
    ipk = float(input("IPK: "))
    mahasiswa.append({"nama": nama, "nim": nim, "prodi": prodi, "ipk": ipk})
    print("Ditambahkan.")

def tampilkan():
    if not mahasiswa:
        print("Data kosong.")
        return
    for m in mahasiswa:
        print(f"{m['nim']} | {m['nama']} | {m['prodi']} | {m['ipk']}")

def cari():
    nim = input("NIM: ")
    for m in mahasiswa:
        if m["nim"] == nim:
            print(m)
            return
    print("Tidak ditemukan.")

def hapus():
    nim = input("NIM: ")
    for i, m in enumerate(mahasiswa):
        if m["nim"] == nim:
            mahasiswa.pop(i)
            print("Dihapus.")
            return
    print("Tidak ditemukan.")

def ipk_tertinggi():
    if not mahasiswa:
        print("Data kosong.")
        return
    terbaik = mahasiswa[0]
    for m in mahasiswa:
        if m["ipk"] > terbaik["ipk"]:
            terbaik = m
    print(terbaik)

while True:
    tampil_menu()
    pilih = input("Pilih: ")
    if pilih == "1": tambah()
    elif pilih == "2": tampilkan()
    elif pilih == "3": cari()
    elif pilih == "4": hapus()
    elif pilih == "5": ipk_tertinggi()
    elif pilih == "6":
        print("Selesai.")
        break
```

```
    else:
        print("Pilihan tidak valid.")
```

OUTPUT (INPUT SIMULASI: 1 ⏎ ANI ⏎ 2201 ⏎ IF ⏎ 3.75 ⏎ 1 ⏎ BUDI ⏎ 2202 ⏎ SI ⏎ 3.2 ⏎ 2 ⏎ 3 ⏎ 2202 ⏎ 5 ⏎ 4 ⏎ 2201 ⏎

2 ⏎ 3 ⏎ 9999 ⏎ 7 ⏎ 6):

```
1. Tambah  2. Tampilkan  3. Cari
4. Hapus   5. IPK Tertinggi  6. Keluar
Pilih: 1
Nama: Ani
NIM: 2201
Prodi: IF
IPK: 3.75
Ditambahkan.

1. Tambah  2. Tampilkan  3. Cari
4. Hapus   5. IPK Tertinggi  6. Keluar
Pilih: 1
Nama: Budi
NIM: 2202
Prodi: SI
IPK: 3.2
Ditambahkan.

1. Tambah  2. Tampilkan  3. Cari
4. Hapus   5. IPK Tertinggi  6. Keluar
Pilih: 2
2201 | Ani | IF | 3.75
2202 | Budi | SI | 3.2

1. Tambah  2. Tampilkan  3. Cari
4. Hapus   5. IPK Tertinggi  6. Keluar
Pilih: 3
NIM: 2202
{'nama': 'Budi', 'nim': '2202', 'prodi': 'SI', 'ipk': 3.2}

1. Tambah  2. Tampilkan  3. Cari
4. Hapus   5. IPK Tertinggi  6. Keluar
Pilih: 5
{'nama': 'Ani', 'nim': '2201', 'prodi': 'IF', 'ipk': 3.75}

1. Tambah  2. Tampilkan  3. Cari
4. Hapus   5. IPK Tertinggi  6. Keluar
Pilih: 4
NIM: 2201
Dihapus.

1. Tambah  2. Tampilkan  3. Cari
4. Hapus   5. IPK Tertinggi  6. Keluar
Pilih: 2
2202 | Budi | SI | 3.2

1. Tambah  2. Tampilkan  3. Cari
4. Hapus   5. IPK Tertinggi  6. Keluar
Pilih: 3
NIM: 9999
Tidak ditemukan.

1. Tambah  2. Tampilkan  3. Cari
4. Hapus   5. IPK Tertinggi  6. Keluar
Pilih: 7
Pilihan tidak valid.

1. Tambah  2. Tampilkan  3. Cari
4. Hapus   5. IPK Tertinggi  6. Keluar
```

```
Pilih: 6
Selesai.
```

Catatan: program menu penuh dijalankan dengan skenario input di atas: tambah dua mahasiswa, tampilkan, cari, IPK tertinggi, hapus, cari NIM yang tidak ada, pilihan tidak valid, lalu keluar.

> **TIPS**
> **Tantangan opsional Project 1**
>
> Tolak NIM duplikat saat tambah.
>
> Validasi IPK hanya 0–4.
>
> Tambah fitur filter by prodi.

8. Mini Project 2 — Sistem Nilai & Statistik

Deskripsi

Setiap mahasiswa punya banyak nilai MK (dict di dalam dict/list). Hitung rata-rata per orang, statistik global, dan filter yang lulus ambang.

Fitur wajib

1. Tambah data: nama + nilai per MK.
2. Tampilkan semua + rata-rata tiap mahasiswa.
3. Statistik global (rata-rata kelas, max, min dari rata-rata individu).
4. Filter rata-rata ≥ ambang (mis. 70).
5. Menu interaktif.

Skeleton inti

```python
data = []  # [{"nama": str, "nilai": {"MK": angka, ...}}, ...]

def rata_rata(nilai_dict):
    if not nilai_dict:
        return 0
    return sum(nilai_dict.values()) / len(nilai_dict)

def tambah_data():
    nama = input("Nama: ")
    nilai = {}
    while True:
        mk = input("Nama MK (kosong=selesai): ").strip()
        if not mk:
            break
        nilai[mk] = float(input(f"Nilai {mk}: "))
    data.append({"nama": nama, "nilai": nilai})

def tampilkan_semua():
    for d in data:
        print(d["nama"], d["nilai"], "rata=", round(rata_rata(d["nilai"]), 2))

def statistik():
    if not data:
        print("Kosong")
        return
    rata_list = [rata_rata(d["nilai"]) for d in data]
    print("Rata kelas:", sum(rata_list)/len(rata_list))
    print("Max:", max(rata_list), "Min:", min(rata_list))

def filter_lulus(ambang=70):
    for d in data:
        r = rata_rata(d["nilai"])
        if r >= ambang:
            print(d["nama"], r)

# while menu: 1 tambah 2 tampil 3 statistik 4 filter 5 keluar
```

OUTPUT:

```
(tidak ada output)
```

Catatan: blok ini hanya mendefinisikan function; menu `while` dibiarkan sebagai tugas peserta (lihat uji pelengkap di bawah).

Uji pelengkap — menu yang dibiarkan kosong pada modul

```python
while True:
    print("\n1. Tambah  2. Tampil  3. Statistik  4. Filter  5. Keluar")
    pilih = input("Pilih: ")
    if pilih == "1": tambah_data()
    elif pilih == "2": tampilkan_semua()
    elif pilih == "3": statistik()
    elif pilih == "4": filter_lulus()
    elif pilih == "5":
        print("Selesai.")
        break
    else:
        print("Pilihan tidak valid.")
```

OUTPUT (INPUT SIMULASI: 1 ⏎ ANI ⏎ PYTHON ⏎ 85 ⏎ FISIKA ⏎ 70 ⏎ ⏎ 1 ⏎ BUDI ⏎ PYTHON ⏎ 60 ⏎ FISIKA ⏎ 55 ⏎ ⏎ 2 ⏎ 3 ⏎ 4 ⏎ 9 ⏎ 5):

```
1. Tambah  2. Tampil  3. Statistik  4. Filter  5. Keluar
Pilih: 1
Nama: Ani
Nama MK (kosong=selesai): Python
Nilai Python: 85
Nama MK (kosong=selesai): Fisika
Nilai Fisika: 70
Nama MK (kosong=selesai):

1. Tambah  2. Tampil  3. Statistik  4. Filter  5. Keluar
Pilih: 1
Nama: Budi
Nama MK (kosong=selesai): Python
Nilai Python: 60
Nama MK (kosong=selesai): Fisika
Nilai Fisika: 55
Nama MK (kosong=selesai):

1. Tambah  2. Tampil  3. Statistik  4. Filter  5. Keluar
Pilih: 2
Ani {'Python': 85.0, 'Fisika': 70.0} rata= 77.5
Budi {'Python': 60.0, 'Fisika': 55.0} rata= 57.5

1. Tambah  2. Tampil  3. Statistik  4. Filter  5. Keluar
Pilih: 3
Rata kelas: 67.5
Max: 77.5 Min: 57.5

1. Tambah  2. Tampil  3. Statistik  4. Filter  5. Keluar
Pilih: 4
Ani 77.5

1. Tambah  2. Tampil  3. Statistik  4. Filter  5. Keluar
Pilih: 9
Pilihan tidak valid.

1. Tambah  2. Tampil  3. Statistik  4. Filter  5. Keluar
Pilih: 5
Selesai.
```

Catatan: kode menu di atas ditambahkan hanya untuk menguji function pada blok sebelumnya; skenario: tambah dua mahasiswa (nama MK kosong = selesai), tampilkan, statistik, filter lulus (ambang 70), pilihan tidak valid, keluar.

9. Troubleshooting

NameError

```python
# Kasus 1: belum assign
# print(nama)  → NameError
nama = "Ani"
print(nama)

# Kasus 2: typo
nilai = 80
# print(niali)  → NameError

# Kasus 3: variabel di dalam blok if yang tidak jalan
x = 5
if x > 10:
    pesan = "besar"
# print(pesan)  bisa NameError jika kondisi False
```

OUTPUT:

```
Ani
```

TypeError

```python
# Kasus 1: str + int
# print("Nilai: " + 80)  → TypeError
print("Nilai: " + str(80))
print(f"Nilai: {80}")

# Kasus 2: len pada int
# len(100)  → TypeError

# Kasus 3: argumen fungsi kurang/lebih (Modul 2 lebih detail)
def tambah(a, b):
    return a + b
# tambah(1)  → TypeError
```

OUTPUT:

```
Nilai: 80
Nilai: 80
```

ValueError & ZeroDivisionError

```python
# int("abc")  → ValueError
# float("")    → ValueError
# 10 / 0      → ZeroDivisionError
pembagi = 2
if pembagi != 0:
    print(10 / pembagi)
```

OUTPUT:

```
5.0
```

IndexError & KeyError

```
data = [10, 20, 30]
# data[5]  → IndexError
if len(data) > 5:
    print(data[5])

mhs = {"nama": "Ani"}
# mhs["nim"]  → KeyError
print(mhs.get("nim", "-"))
if "nim" in mhs:
    print(mhs["nim"])
```

OUTPUT:

```
-
```

Demonstrasi error — output nyata

Bagian ini menjalankan setiap kasus pada tabel di atas secara terpisah, sehingga Anda melihat pesan error yang sebenarnya (pada contoh sebelumnya baris error sengaja dikomentari agar kode dapat dijalankan utuh).

NameError — tiga kasus

```
# Kasus 1: belum assign
print(nama)

# Kasus 2: typo
nilai = 80
print(niali)

# Kasus 3: variabel di blok if yang tidak jalan
x = 5
if x > 10:
    pesan = "besar"
print(pesan)
```

OUTPUT:

```
# Kasus 1: belum assign
  -> NameError: name 'nama' is not defined
# Kasus 2: typo
  -> NameError: name 'niali' is not defined
# Kasus 3: variabel di blok if yang tidak jalan
  -> NameError: name 'pesan' is not defined
```

TypeError — tiga kasus

```python
# Kasus 1: str + int
print("Nilai: " + 80)

# Kasus 2: len pada int
len(100)

# Kasus 3: argumen fungsi kurang
def tambah(a, b):
    return a + b
tambah(1)
```

OUTPUT:

```
# Kasus 1: str + int
  -> TypeError: can only concatenate str (not "int") to str
# Kasus 2: len pada int
  -> TypeError: object of type 'int' has no len()
# Kasus 3: argumen fungsi kurang
  -> TypeError: tambah() missing 1 required positional argument: 'b'
```

ValueError & ZeroDivisionError

```python
# int("abc")
int("abc")

# float("")
float("")

# int("3.5") — tidak bisa langsung; solusinya float dulu
print(int(float("3.5")))
int("3.5")

# 10 / 0
10 / 0
```

OUTPUT:

```
# int("abc")
  -> ValueError: invalid literal for int() with base 10: 'abc'
# float("")
  -> ValueError: could not convert string to float: ''
# int("3.5") — tidak bisa langsung; solusinya float dulu
3
  -> ValueError: invalid literal for int() with base 10: '3.5'
# 10 / 0
  -> ZeroDivisionError: division by zero
```

IndexError & KeyError

```python
data = [10, 20, 30]
data[5]

mhs = {"nama": "Ani"}
mhs["nim"]
```

OUTPUT:

```
data = [10, 20, 30]
  -> IndexError: list index out of range
mhs = {"nama": "Ani"}
  -> KeyError: 'nim'
```

IndentationError & SyntaxError

```
if True:
print("tanpa indentasi")

nilai = 80
if nilai >= 70 and < 85:
    print("B")
```

OUTPUT:

```
if True:
  -> IndentationError: expected an indented block after 'if' statement on line 1
nilai = 80
  -> SyntaxError: invalid syntax
```

10. Solusi Latihan (Cuplikan Utama)

10.1 Variabel & I/O

```
nama = input("Nama: ")
n1 = float(input("Nilai 1: "))
n2 = float(input("Nilai 2: "))
n3 = float(input("Nilai 3: "))
print(f"{nama}: {(n1+n2+n3)/3:.2f}")

c = float(input("Celsius: "))
print(c * 9/5 + 32)

BIAYA = 150000
sks = int(input("SKS: "))
print(sks * BIAYA)

a = input("a: "); b = input("b: ")
a, b = b, a
print(a, b)
```

OUTPUT (INPUT SIMULASI: SITI AMINAH ⏎ 80 ⏎ 90 ⏎ 85 ⏎ 100 ⏎ 20 ⏎ KIRI ⏎ KANAN):

```
Nama: Siti Aminah
Nilai 1: 80
Nilai 2: 90
Nilai 3: 85
Siti Aminah: 85.00
Celsius: 100
212.0
SKS: 20
3000000
a: kiri
b: kanan
kanan kiri
```

Catatan: urutan input: nama, tiga nilai, suhu Celsius, SKS, lalu dua string untuk swap.

10.2 Control Flow

```python
n = int(input("Nilai: "))
if n >= 85: print("A")
elif n >= 70: print("B")
elif n >= 60: print("C")
elif n >= 50: print("D")
else: print("E")

for i in range(2, 51, 2):
    print(i, end=" ")
print()

total = float(input("Total: "))
bayar = total * 0.9 if total >= 100000 else total
print(bayar)

rahasia = 7
while True:
    if int(input("Tebak: ")) == rahasia:
        print("Benar"); break

nilai_list = [80, 90, 55, 40, 85]
a = sum(1 for v in nilai_list if v >= 85)
g = sum(1 for v in nilai_list if v < 60)
print("A:", a, "tidak lulus:", g)
```

OUTPUT (INPUT SIMULASI: 78 ⏎ 150000 ⏎ 3 ⏎ 7):

```
Nilai: 78
B
2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50
Total: 150000
135000.0
Tebak: 3
Tebak: 7
Benar
A: 2 tidak lulus: 2
```

Catatan: urutan input: nilai ujian, total belanja, tebakan (3 salah, 7 benar).

10.3 Data Structures

```
mk = ["Kalkulus", "Fisika"]; mk.append("Python"); mk.remove("Fisika"); print(len(mk), mk)

mhs = [
    {"nama": "Ani", "nim": "2201", "ipk": 3.5, "prodi": "IF"},
    {"nama": "Budi", "nim": "2202", "ipk": 3.9, "prodi": "SI"},
    {"nama": "Cici", "nim": "2203", "ipk": 3.2, "prodi": "IF"},
]
terbaik = max(mhs, key=lambda x: x["ipk"])
print(terbaik["nama"])
if_only = [m for m in mhs if m["prodi"] == "IF"]
print(sum(m["ipk"] for m in if_only) / len(if_only))
```

OUTPUT:

```
2 ['Kalkulus', 'Python']
Budi
3.35
```

11. Checklist Sebelum Modul 2

- Contoh variabel, casting, operator, I/O diketik ulang & diubah-ubah
- Latihan control flow dikerjakan sebelum melihat solusi
- List of dict: cari, filter, IPK tertinggi sudah dipraktikkan
- Mini Project 1 berjalan; Project 2 diusahakan
- File tersimpan di portfolio/modul_01/

> **Modul 1 (lengkap) selesai? Lanjut ke Modul 2.**
>
> Python Intensive — Modul Belajar Mandiri

2

Modul 2 — Functions, Modules & File Handling

Versi lengkap — pembahasan dalam + kasus beragam · Estimasi: ± 6 jam · Prasyarat: Modul 1 selesai

1. Apa yang Akan Anda Pelajari
2. Functions — Pembahasan Dalam
3. Lambda, map, filter, Comprehension
4. Modules & Packages
5. Error Handling
6. File Handling
7. Mini Project 1 — Pencatat Kegiatan Akademik
8. Mini Project 2 — Sistem Perpustakaan Mini
9. Troubleshooting
10. Solusi Latihan (Cuplikan)
11. Checklist Sebelum Modul 3

> **Catatan pengujian:** seluruh blok kode pada modul ini dijalankan (Python 3.11). Bagian **Output** adalah hasil eksekusi nyata. Blok yang memakai `input()` dijalankan dengan input simulasi yang ditulis di samping label output. Blok yang hanya berisi definisi/komentar diberi keterangan, dan "Uji pelengkap" ditambahkan agar function/class pada blok tersebut terbukti berjalan. Kasus error pada bagian Troubleshooting dijalankan nyata di "Demonstrasi error". Blok yang menulis file (`catatan.txt` , `nilai.csv` , `kegiatan.csv` , `buku.csv` , dst.) dijalankan di folder sementara.

1. Apa yang Akan Anda Pelajari

- Merancang function: parameter, default, return, *args/**kwargs, docstring, dan scope.
- Memakai lambda, map/filter, dan list comprehension pada kasus yang berbeda.
- Membuat module sendiri dan memahami import.
- Menangani error dengan try/except dan menulis/membaca file teks serta CSV.

> CATATAN
> **Sebelum mulai**
>
> Modul 1 (variabel, control flow, list/dict) sebaiknya sudah dipraktikkan.
>
> Ketik contoh, lalu ubah konteksnya — jangan hanya membaca.
>
> Simpan latihan di portfolio/modul_02/.

2. Functions — Pembahasan Dalam

2.1 Mengapa function?

Function memecah masalah menjadi unit bernama yang bisa dipanggil ulang, diuji terpisah, dan dibaca seperti kosakata domain (“hitung_ipk”, “cek_lulus”). Tanpa function, logika yang sama sering disalin-tempel — sulit dirawat saat aturan berubah.

2.2 Dasar, parameter, default, return

Parameter adalah slot input. Default membuat argumen opsional. `return` mengirim hasil ke pemanggil; tanpa return, function mengembalikan `None`. Pisahkan “hitung nilai” dari “cetak nilai” agar function lebih fleksibel.

```python
# Kasus 1: sapaan
def sapa(nama, salam="Halo"):
    return f"{salam}, {nama}!"
print(sapa("Rina"))
print(sapa("Budi", salam="Selamat pagi"))

# Kasus 2: hitung luas — return, jangan hanya print
def luas_persegi(sisi):
    return sisi * sisi
luas = luas_persegi(5)
print("Luas:", luas)

# Kasus 3: grade dari skor
def nilai_huruf(skor):
    if skor >= 85: return "A"
    if skor >= 70: return "B"
    if skor >= 60: return "C"
    if skor >= 50: return "D"
    return "E"
print(nilai_huruf(78))

# Kasus 4: BMI
def hitung_bmi(berat_kg, tinggi_m):
    return berat_kg / (tinggi_m ** 2)
print(round(hitung_bmi(60, 1.65), 2))

# Kasus 5: rata-rata list (kosong = 0)
def rata_rata(angka):
    if not angka:
        return 0
    return sum(angka) / len(angka)
print(rata_rata([80, 90, 70]))
```

OUTPUT:

```
Halo, Rina!
Selamat pagi, Budi!
Luas: 25
B
22.04
80.0
```

2.3 *args dan **kwargs

`*args` menampung argumen posisional ekstra sebagai tuple. `**kwargs` menampung pasangan nama=nilai sebagai dict. Berguna saat jumlah input tidak tetap, tetapi jangan dipakai jika parameter eksplisit lebih jelas.

```python
# Kasus jumlah tak terbatas
def jumlahkan(*angka):
    total = 0
    for x in angka:
        total += x
    return total
print(jumlahkan(1, 2, 3, 4))

# Kasus max-min
def max_min(*angka):
    return max(angka), min(angka)
print(max_min(3, 1, 4, 2))

# Kasus profil fleksibel
def profil(nama, **info):
    print("Nama:", nama)
    for k, v in info.items():
        print(f"  {k}: {v}")
profil("Ani", prodi="IF", angkatan=2022, ipk=3.5)
```

OUTPUT:

```
10
(4, 1)
Nama: Ani
  prodi: IF
  angkatan: 2022
  ipk: 3.5
```

2.4 Scope (local vs global)

Nama di dalam function bersifat lokal kecuali Anda memakai `global` (jarang disarankan). Lebih aman: terima input lewat parameter dan kembalikan hasil lewat `return`.

```python
x = 10
def ubah():
    x = 5
    print("dalam:", x)
ubah()
print("luar:", x)  # 10

# Pola yang lebih baik
def tambah_satu(n):
    return n + 1
x = tambah_satu(x)
```

OUTPUT:

```
dalam: 5
luar: 10
```

PERHATIAN

Jebakan scope

UnboundLocalError: Anda assign ke nama yang sama dengan global di dalam function tanpa parameter/return yang jelas.

Mengubah list/dict mutabel lewat side effect bisa mengejutkan — dokumentasikan atau kembalikan objek baru.

2.5 Docstring

Docstring menjelaskan kontrak function: apa yang dilakukan, argumen, dan return. Berguna untuk Anda sendiri dan untuk `help()`.

```
def rata_rata(angka):
    """Menghitung rata-rata iterable numerik.

    Returns 0 jika kosong.
    """
    if not angka:
        return 0
    return sum(angka) / len(angka)
```

OUTPUT:

```
(tidak ada output)
```

2.6 Latihan Functions

1. hitung_bmi(berat, tinggi) + kategori kasar (under/normal/over).

```
Pseudocode:
  FUNCTION hitung_bmi(berat, tinggi)
    bmi ← berat / (tinggi × tinggi)
    IF bmi < 18.5 THEN kategori ← "under"
    ELSE IF bmi < 25 THEN kategori ← "normal"
    ELSE kategori ← "over"
    RETURN bmi dan kategori
```

2. nilai_huruf(skor) lengkap A–E.

```
Pseudocode:
  FUNCTION nilai_huruf(skor)
    IF skor ≥ 85 RETURN "A"
    ELSE IF skor ≥ 70 RETURN "B"
    ELSE IF skor ≥ 60 RETURN "C"
    ELSE IF skor ≥ 50 RETURN "D"
    ELSE RETURN "E"
```

3. max_min(*angka) → tuple.

```
Pseudocode:
  FUNCTION max_min(*angka)
    RETURN (nilai maksimum, nilai minimum)
```

4. is_palindrome(teks) — abaikan spasi & huruf besar.

```
Pseudocode:
  FUNCTION is_palindrome(teks)
    s ← teks dalam huruf kecil tanpa spasi
    RETURN (s sama dengan s terbalik)
```

5. hitung_diskon(total, persen=10) — hanya jika total ≥ 100000, else 0 diskon.

```
Pseudocode:
  FUNCTION hitung_diskon(total, persen default 10)
    IF total ≥ 100000 THEN RETURN total × persen / 100
    ELSE RETURN 0
```

6. rata_rata_ipk(list_of_dict) dari key "ipk".

```
Pseudocode:
  FUNCTION rata_rata_ipk(list_of_dict)
    IF list kosong RETURN 0
    jumlah ← sum semua elemen["ipk"]
    RETURN jumlah / banyak elemen
```

7. konversi_suhu(nilai, mode="c2f") mendukung c2f dan f2c.

```
Pseudocode:
  FUNCTION konversi_suhu(nilai, mode="c2f")
    IF mode = "c2f" RETURN nilai×9/5+32
    ELSE IF mode = "f2c" RETURN (nilai-32)×5/9
    ELSE RETURN None / error
```

8. ringkas_nilai(*nilai) → dict berisi rata, maks, min, jumlah.

```
Pseudocode:
  FUNCTION ringkas_nilai(*nilai)
    RETURN dict {rata, maks, min, jumlah}
```

3. Lambda, map, filter, Comprehension

3.1 Pembahasan

Lambda adalah function pendek sekali pakai. `map` / `filter` menerapkan function ke iterable. List comprehension sering lebih pythonic untuk transformasi & filter sederhana. Pilih yang paling mudah dibaca di konteks Anda.

```python
angka = [1, 2, 3, 4, 5]
kuadrat = list(map(lambda x: x ** 2, angka))
genap = list(filter(lambda x: x % 2 == 0, angka))
print(kuadrat, genap)

# Comprehension — kasus transformasi nilai
nilai = [55, 80, 90, 40, 70]
lulus = [v for v in nilai if v >= 70]
label = ["L" if v >= 60 else "TL" for v in nilai]
print(lulus, label)

# Kasus list nama → panjang
nama = ["Ani", "Budi", "Citra"]
panjang = [len(n) for n in nama]

# Kasus dict comprehension genap→kuadrat
d = {x: x ** 2 for x in angka if x % 2 == 0}
print(d)

# Kasus sort key
mhs = [{"nama": "Cici", "ipk": 3.2}, {"nama": "Ani", "ipk": 3.8}]
mhs_sorted = sorted(mhs, key=lambda m: m["ipk"], reverse=True)
print(mhs_sorted)
```

OUTPUT:

```
[1, 4, 9, 16, 25] [2, 4]
[80, 90, 70] ['TL', 'L', 'L', 'TL', 'L']
{2: 4, 4: 16}
[{'nama': 'Ani', 'ipk': 3.8}, {'nama': 'Cici', 'ipk': 3.2}]
```

3.2 Latihan

1. Dari list nilai: list baru yang ≥ 70.

```
Pseudocode:
  buat list kosong hasil
  FOR tiap v di list nilai:
    IF v ≥ 70 THEN tambahkan v ke hasil
  RETURN hasil
  # alternatif: list comprehension
```

2. List nama → list panjang karakter.

```
Pseudocode:
  hasil ← []
  FOR tiap nama n:
    tambahkan len(n) ke hasil
  RETURN hasil
```

3. Dict {angka: kuadrat} hanya untuk genap.

```
Pseudocode:
  hasil ← {}
  FOR x di angka:
    IF x genap THEN hasil[x] ← x²
  RETURN hasil
```

4. Sort list of dict mahasiswa by IPK naik memakai key=lambda.

```
Pseudocode:
  urutkan list dengan key = field ipk
  urutan naik (ascending)
  # sorted(mhs, key=lambda m: m["ipk"])
```

4. Modules & Packages

4.1 Pembahasan

Module adalah file `.py` yang bisa di-import. Memecah kode ke module mengurangi file raksasa dan memungkinkan pemakaian ulang (mis. `konversi.py`, `statistik.py`). Nama file module jangan menabrak modul standar (`random.py` berbahaya).

```python
import math
from random import randint, choice
import datetime as dt
print(math.sqrt(16), randint(1, 6), dt.date.today())

# matematika.py (file terpisah)
# def tambah(a, b): return a + b
# PI = 3.14159

# main.py
# import matematika
# from matematika import tambah, PI
# konversi.py — kasus suhu & jarak
def celsius_ke_fahrenheit(c):
    return c * 9 / 5 + 32

def fahrenheit_ke_celsius(f):
    return (f - 32) * 5 / 9

def km_ke_mil(km):
    return km * 0.621371
```

OUTPUT:

```
4.0 1 2026-10-01
```

4.2 Latihan

1. Buat konversi.py (suhu); import dan uji dari file lain.

```
Pseudocode:
  BUAT file konversi.py berisi function suhu
  DI main: import konversi
  panggil function; cetak hasil uji
```

2. Buat statistik.py berisi rata, maks, min; pakai di main.

```
Pseudocode:
  BUAT statistik.py: function rata, maks, min
  DI main: import dan uji dengan list contoh
```

5. Error Handling

5.1 Pembahasan

Error handling memisahkan jalur sukses dari jalur gagal. `try/except` menangkap exception spesifik. `else` jalan jika tidak ada error. `finally` selalu jalan (rapi resource). `raise` memunculkan error sendiri saat kontrak dilanggar.

```python
# Kasus input angka aman
def input_angka(prompt):
    while True:
        try:
            return float(input(prompt))
        except ValueError:
            print("Masukkan angka valid.")

# Kasus pembagian aman
def bagi(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        print("Tidak bisa dibagi nol")
        return None
    except TypeError:
        print("Tipe harus numerik")
        return None

# Kasus raise untuk validasi domain
def set_ipk(ipk):
    if not 0 <= ipk <= 4:
        raise ValueError("IPK harus 0–4")
    return ipk

try:
    set_ipk(4.5)
except ValueError as e:
    print("Gagal:", e)

# Pola lengkap
try:
    x = int("10")
except ValueError:
    print("bukan angka")
else:
    print("ok", x)
finally:
    print("selesai blok")
```

OUTPUT:

```
Gagal: IPK harus 0–4
ok 10
selesai blok
```

Uji pelengkap — memanggil function di atas

```
print(input_angka("Angka: "))
print(bagi(10, 4)); print(bagi(10, 0)); print(bagi(10, "a"))
```

OUTPUT (INPUT SIMULASI: ABC ⏎ 3.5):

```
Angka: abc
Masukkan angka valid.
Angka: 3.5
3.5
2.5
Tidak bisa dibagi nol
None
Tipe harus numerik
None
```

PERHATIAN

Jebakan

except: bare — menangkap semua, termasuk KeyboardInterrupt; lebih baik spesifik.

Menelan error tanpa log membuat bug sulit dilacak.

Jangan pakai try/except untuk kontrol alur normal jika if sudah cukup.

5.2 Latihan

1. bagi(a,b) aman (ZeroDivision + TypeError).

```
Pseudocode:
  FUNCTION bagi(a, b)
    TRY: RETURN a / b
    CATCH ZeroDivisionError: pesan; RETURN None
    CATCH TypeError: pesan; RETURN None
```

2. input_angka(prompt) sampai valid.

```
Pseudocode:
  FUNCTION input_angka(prompt)
    LOOP:
      TRY: RETURN float(input(prompt))
      CATCH ValueError: cetak peringatan
```

3. set_umur(n) raise ValueError jika n < 0 atau n > 120.

```
Pseudocode:
  FUNCTION set_umur(n)
    IF n < 0 ATAU n > 120 THEN RAISE ValueError
    RETURN n
```

6. File Handling

6.1 Teks & with

`with open(...)` memastikan file tertutup meski terjadi error. Mode `w` menimpa, `a` menambah, `r` membaca. Selalu pertimbangkan encoding UTF-8 untuk teks Indonesia.

```
# Tulis catatan
with open("catatan.txt", "w", encoding="utf-8") as f:
    f.write("Baris satu\n")
    f.write("Baris dua\n")

# Baca seluruh
with open("catatan.txt", "r", encoding="utf-8") as f:
    isi = f.read()
print(isi)

# Baca per baris
with open("catatan.txt", "r", encoding="utf-8") as f:
    for baris in f:
        print(baris.strip())

# Kasus: simpan daftar nama
nama_list = ["Ani", "Budi", "Cici"]
with open("nama.txt", "w", encoding="utf-8") as f:
    for n in nama_list:
        f.write(n + "\n")
```

OUTPUT:

```
Baris satu
Baris dua

Baris satu
Baris dua
```

6.2 pathlib

`pathlib.Path` membuat manipulasi path lebih jelas di Windows/Linux dan mendukung `read_text` / `write_text`.

```
from pathlib import Path
p = Path("data")
p.mkdir(exist_ok=True)
file = p / "hasil.txt"
file.write_text("Halo\n", encoding="utf-8")
print(file.read_text(encoding="utf-8"))
print(file.exists(), file.suffix)
```

OUTPUT:

```
Halo

True .txt
```

6.3 CSV

CSV adalah format tabular sederhana. `csv.DictReader` / `DictWriter` cocok untuk list of dict — selaras dengan pola Modul 1. Gunakan `newline=""` saat menulis di Windows.

```
import csv
from pathlib import Path

# Tulis
with open("nilai.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.DictWriter(f, fieldnames=["nama", "nim", "nilai"])
    w.writeheader()
    w.writerow({"nama": "Ani", "nim": "2201", "nilai": 90})
    w.writerow({"nama": "Budi", "nim": "2202", "nilai": 85})

# Baca
with open("nilai.csv", "r", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        print(row["nama"], row["nilai"])

# Kasus: muat ke list of dict
def muat_csv(path):
    path = Path(path)
    if not path.exists():
        return []
    with open(path, "r", encoding="utf-8") as f:
        return list(csv.DictReader(f))
```

OUTPUT:

```
Ani 90
Budi 85
```

6.4 Latihan File

1. Tulis 5 nama ke nama.txt; baca dan cetak.

```
Pseudocode:
  buka nama.txt mode tulis (with)
  FOR 5 nama: tulis baris ke file
  buka mode baca; cetak seluruh isi
```

2. Simpan list of dict mahasiswa ke CSV; baca ulang.

```
Pseudocode:
  buka CSV tulis + DictWriter + header
  tulis setiap dict sebagai baris
  baca ulang dengan DictReader; tampilkan
```

3. Append baris log ke kegiatan.log setiap kali fungsi dipanggil (mode a).

```
Pseudocode:
  FUNCTION catat_log(pesan)
    buka kegiatan.log mode append
    tulis pesan (+ opsional waktu)
  panggil beberapa kali untuk menguji
```

7. Mini Project 1 — Pencatat Kegiatan Akademik

Deskripsi

Catat kegiatan (tanggal, jenis, deskripsi, status). Persist ke CSV agar data tidak hilang saat program ditutup.

Fitur wajib

1. Tambah kegiatan; tampilkan semua; filter jenis/status.
2. Muat CSV di awal; simpan setiap ada perubahan.
3. Menu interaktif; function terpisah per aksi.

```python
import csv
from pathlib import Path

FILE = Path("kegiatan.csv")
kegiatan = []

def muat():
    global kegiatan
    if not FILE.exists():
        return
    with open(FILE, "r", encoding="utf-8") as f:
        kegiatan = list(csv.DictReader(f))

def simpan():
    if not kegiatan:
        return
    with open(FILE, "w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=kegiatan[0].keys())
        w.writeheader()
        w.writerows(kegiatan)

def tambah():
    row = {
        "tanggal": input("Tanggal: "),
        "jenis": input("Jenis (tugas/kuliah/ujian): "),
        "deskripsi": input("Deskripsi: "),
        "status": input("Status (belum/selesai): "),
    }
    kegiatan.append(row)
    simpan()

def tampilkan():
    for i, k in enumerate(kegiatan, 1):
        print(i, k)

def filter_kegiatan():
    kunci = input("Filter jenis/status: ")
    for k in kegiatan:
        if kunci in (k["jenis"], k["status"]):
            print(k)

muat()
while True:
    print("\n1.Tambah 2.Tampil 3.Filter 4.Keluar")
    p = input("Pilih: ")
    if p == "1": tambah()
    elif p == "2": tampilkan()
    elif p == "3": filter_kegiatan()
    elif p == "4": break
```

OUTPUT (INPUT SIMULASI: 1 ⏎ 2026-09-21 ⏎ TUGAS ⏎ LAPORAN PYTHON ⏎ BELUM ⏎ 1 ⏎ 2026-09-22 ⏎ KULIAH ⏎ PEMROGRAMAN ⏎ SELESAI ⏎ 2 ⏎ 3 ⏎ TUGAS ⏎ 4):

```
1.Tambah 2.Tampil 3.Filter 4.Keluar
Pilih: 1
Tanggal: 2026-09-21
Jenis (tugas/kuliah/ujian): tugas
Deskripsi: Laporan Python
Status (belum/selesai): belum

1.Tambah 2.Tampil 3.Filter 4.Keluar
Pilih: 1
Tanggal: 2026-09-22
Jenis (tugas/kuliah/ujian): kuliah
Deskripsi: Pemrograman
Status (belum/selesai): selesai

1.Tambah 2.Tampil 3.Filter 4.Keluar
Pilih: 2
1 {'tanggal': '2026-09-21', 'jenis': 'tugas', 'deskripsi': 'Laporan Python', 'status': 'belum'}
2 {'tanggal': '2026-09-22', 'jenis': 'kuliah', 'deskripsi': 'Pemrograman', 'status': 'selesai'}

1.Tambah 2.Tampil 3.Filter 4.Keluar
Pilih: 3
Filter jenis/status: tugas
{'tanggal': '2026-09-21', 'jenis': 'tugas', 'deskripsi': 'Laporan Python', 'status': 'belum'}

1.Tambah 2.Tampil 3.Filter 4.Keluar
Pilih: 4
```

Catatan: skenario: tambah dua kegiatan (data disimpan ke `kegiatan.csv`), tampilkan, filter `tugas`, keluar.

8. Mini Project 2 — Sistem Perpustakaan Mini

Deskripsi

Kelola buku: tambah, pinjam, kembalikan, cari. Status tersedia/dipinjam. Data di CSV.

Fitur wajib

1. CRUD status pinjam/kembali; cari keyword judul/penulis.
2. Persistensi CSV; function terpisah; validasi pinjam hanya jika tersedia.

```python
import csv
from pathlib import Path

FILE = Path("buku.csv")
buku = []

def muat():
    global buku
    if FILE.exists():
        with open(FILE, encoding="utf-8") as f:
            buku = list(csv.DictReader(f))

def simpan():
    if not buku: return
    with open(FILE, "w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=["judul", "penulis", "tahun", "status"])
        w.writeheader(); w.writerows(buku)

def tambah_buku():
    buku.append({
        "judul": input("Judul: "),
        "penulis": input("Penulis: "),
        "tahun": input("Tahun: "),
        "status": "tersedia",
    })
    simpan()

def pinjam(judul):
    for b in buku:
        if b["judul"].lower() == judul.lower() and b["status"] == "tersedia":
            b["status"] = "dipinjam"; simpan(); return True
    return False

def kembalikan(judul):
    for b in buku:
        if b["judul"].lower() == judul.lower():
            b["status"] = "tersedia"; simpan(); return True
    return False

def cari(keyword):
    key = keyword.lower()
    return [b for b in buku if key in b["judul"].lower() or key in b["penulis"].lower()]

muat()
# menu: tambah / pinjam / kembali / cari / tampil / keluar
```

OUTPUT:

```
(tidak ada output)
```

Catatan: blok ini hanya mendefinisikan function dan memanggil `muat()`; menu dibiarkan sebagai tugas peserta (lihat uji pelengkap).

Uji pelengkap — menu perpustakaan

```
while True:
    print("\n1.Tambah 2.Pinjam 3.Kembali 4.Cari 5.Tampil 6.Keluar")
    p = input("Pilih: ")
    if p == "1": tambah_buku()
    elif p == "2": print("Berhasil" if pinjam(input("Judul: ")) else "Gagal (tidak ada / sedang
dipinjam)")
    elif p == "3": print("Berhasil" if kembalikan(input("Judul: ")) else "Judul tidak ditemukan")
    elif p == "4":
        for b in cari(input("Kata kunci: ")):
            print(b)
    elif p == "5":
        for b in buku:
            print(b)
    elif p == "6": break
```

OUTPUT (INPUT SIMULASI: 1 ⏎ PYTHON DASAR ⏎ ANI ⏎ 2024 ⏎ 1 ⏎ BASIS DATA ⏎ BUDI ⏎ 2023 ⏎ 5 ⏎ 2 ⏎ PYTHON DASAR ⏎ 2 ⏎ PYTHON DASAR ⏎ 4 ⏎ BUDI ⏎ 3 ⏎ PYTHON DASAR ⏎ 5 ⏎ 6):

```
1.Tambah 2.Pinjam 3.Kembali 4.Cari 5.Tampil 6.Keluar
Pilih: 1
Judul: Python Dasar
Penulis: Ani
Tahun: 2024

1.Tambah 2.Pinjam 3.Kembali 4.Cari 5.Tampil 6.Keluar
Pilih: 1
Judul: Basis Data
Penulis: Budi
Tahun: 2023

1.Tambah 2.Pinjam 3.Kembali 4.Cari 5.Tampil 6.Keluar
Pilih: 5
{'judul': 'Python Dasar', 'penulis': 'Ani', 'tahun': '2024', 'status': 'tersedia'}
{'judul': 'Basis Data', 'penulis': 'Budi', 'tahun': '2023', 'status': 'tersedia'}

1.Tambah 2.Pinjam 3.Kembali 4.Cari 5.Tampil 6.Keluar
Pilih: 2
Judul: python dasar
Berhasil

1.Tambah 2.Pinjam 3.Kembali 4.Cari 5.Tampil 6.Keluar
Pilih: 2
Judul: python dasar
Gagal (tidak ada / sedang dipinjam)

1.Tambah 2.Pinjam 3.Kembali 4.Cari 5.Tampil 6.Keluar
Pilih: 4
Kata kunci: budi
{'judul': 'Basis Data', 'penulis': 'Budi', 'tahun': '2023', 'status': 'tersedia'}

1.Tambah 2.Pinjam 3.Kembali 4.Cari 5.Tampil 6.Keluar
Pilih: 3
Judul: Python Dasar
Berhasil

1.Tambah 2.Pinjam 3.Kembali 4.Cari 5.Tampil 6.Keluar
Pilih: 5
{'judul': 'Python Dasar', 'penulis': 'Ani', 'tahun': '2024', 'status': 'tersedia'}
{'judul': 'Basis Data', 'penulis': 'Budi', 'tahun': '2023', 'status': 'tersedia'}

1.Tambah 2.Pinjam 3.Kembali 4.Cari 5.Tampil 6.Keluar
Pilih: 6
```

Catatan: menu ditambahkan hanya untuk menguji function; data tersimpan di `buku.csv`. Pinjam kedua kali pada judul yang sama gagal karena sudah berstatus dipinjam.

TIPS

Tips project

Selalu simpan() setelah data berubah.

Bandingkan judul dengan .lower() agar tidak gagal karena kapitalisasi.

Project 1 dulu jika CSV masih terasa sulit.

9. Troubleshooting

```
# TypeError: missing argument → cek jumlah argumen saat panggil function
# UnboundLocalError → hindari assign ke nama global di dalam function; pakai return
# Function mengembalikan None → pastikan semua cabang yang perlu nilai punya return
# FileNotFoundError → cek path; Path.exists() sebelum baca
# PermissionError → file sedang dibuka aplikasi lain
# CSV berantakan → newline="" saat write; encoding utf-8
# ModuleNotFoundError → file module di folder yang sama; jangan namai random.py
```

> **OUTPUT: *TIDAK DIJALANKAN* — HANYA KOMENTAR (DAFTAR JEBAKAN); CONTOH ERROR NYATA DIJALANKAN DI BAGIAN "DEMONSTRASI ERROR" DI BAWAH**

Demonstrasi error — output nyata

Setiap kasus pada daftar di atas dijalankan terpisah di folder sementara agar pesan error aslinya terlihat. `PermissionError` tidak ditampilkan karena bergantung pada hak akses sistem file dan tidak dapat direproduksi andal di lingkungan pengujian (berjalan sebagai root).

TypeError — argument kurang / berlebih

```
def perkenalan(nama, prodi):
    print(f"Saya {nama} dari {prodi}")

perkenalan("Ani")
```

OUTPUT:

```
  -> TypeError: perkenalan() missing 1 required positional argument: 'prodi'
```

TypeError — argument kurang / berlebih (2)

```
def perkenalan(nama, prodi):
    print(f"Saya {nama} dari {prodi}")

perkenalan("Ani", "IF", 3)
```

OUTPUT:

```
  -> TypeError: perkenalan() takes 2 positional arguments but 3 were given
```

UnboundLocalError — assign ke nama global di dalam function

```
counter = 0

def tambah():
    counter += 1
    return counter

tambah()
```

OUTPUT:

```
  -> UnboundLocalError: cannot access local variable 'counter' where it is not associated with a
value
```

Solusi UnboundLocalError — pakai return

```
counter = 0

def tambah(c):
    return c + 1

counter = tambah(counter)
print(counter)
```

OUTPUT:

```
1
```

Function mengembalikan None — tidak ada return pada salah satu cabang

```
def nilai_huruf(skor):
    if skor >= 70:
        return "Lulus"
    # lupa return untuk cabang lain

hasil = nilai_huruf(50)
print(hasil)
print(hasil.upper())
```

OUTPUT:

```
None
  -> AttributeError: 'NoneType' object has no attribute 'upper'
```

FileNotFoundError — file belum ada

```
with open("tidak_ada.txt", "r", encoding="utf-8") as f:
    print(f.read())
```

OUTPUT:

```
  -> FileNotFoundError: [Errno 2] No such file or directory: 'tidak_ada.txt'
```

Solusi FileNotFoundError — cek dengan pathlib

```
from pathlib import Path

p = Path("tidak_ada.txt")
if p.exists():
    print(p.read_text(encoding="utf-8"))
else:
    print("File belum ada, membuat file kosong...")
    p.write_text("", encoding="utf-8")
print(p.exists())
```

OUTPUT:

```
File belum ada, membuat file kosong...
True
```

UnicodeEncodeError — encoding tidak mendukung karakter

```
with open("uji.txt", "w", encoding="ascii") as f:
    f.write("Café — Ünicode")
```

OUTPUT:

```
  -> UnicodeEncodeError: 'ascii' codec can't encode character '\xe9' in position 3: ordinal not in
range(128)
```

UnicodeDecodeError — dibaca dengan encoding berbeda

```
with open("uji.txt", "w", encoding="utf-8") as f:
    f.write("Café — Ünicode")

with open("uji.txt", "r", encoding="ascii") as f:
    print(f.read())
```

OUTPUT:

```
  -> UnicodeDecodeError: 'ascii' codec can't decode byte 0xc3 in position 3: ordinal not in
range(128)
```

CSV — isi byte file dengan newline=""

```
import csv

with open("a.csv", "w", newline="", encoding="utf-8") as f:
    csv.writer(f).writerows([["nama", "nilai"], ["Ani", "90"]])

with open("a.csv", "rb") as f:
    print(f.read())
```

OUTPUT:

```
b'nama,nilai\r\nAni,90\r\n'
```

ModuleNotFoundError — module tidak ditemukan

```
import matematika_utils
```

OUTPUT:

```
  -> ModuleNotFoundError: No module named 'matematika_utils'
```

ModuleNotFoundError teratasi setelah file module ada

```
import matematika_utils as mu
print(mu.cek_prima(17))
```

OUTPUT:

```
True
```

Menamai file seperti module bawaan — `random.py` menimpa module `random`

```
import subprocess, sys

# file random.py buatan sendiri berada di folder yang sama dengan main.py
hasil = subprocess.run([sys.executable, "main.py"], capture_output=True, text=True)
print(hasil.stderr.strip().splitlines()[-1])
```

OUTPUT:

```
AttributeError: module 'random' has no attribute 'randint'
```

10. Solusi Latihan (Cuplikan)

```
Pseudocode:
  FUNCTION is_palindrome(teks)
    s ← teks dalam huruf kecil tanpa spasi
    RETURN (s sama dengan s terbalik)
```

```
def hitung_bmi(berat, tinggi):
    return berat / (tinggi ** 2)

def nilai_huruf(skor):
    if skor >= 85: return "A"
    if skor >= 70: return "B"
    if skor >= 60: return "C"
    if skor >= 50: return "D"
    return "E"

def is_palindrome(teks):
    s = "".join(teks.lower().split())
    return s == s[::-1]

def bagi(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return None
    except TypeError:
        return None

def input_angka(prompt):
    while True:
        try:
            return float(input(prompt))
        except ValueError:
            print("Angka saja.")
```

OUTPUT:

```
(tidak ada output)
```

Catatan: blok hanya mendefinisikan function; uji pelengkap di bawah memanggilnya.

Uji pelengkap — memanggil function solusi

```
print(hitung_bmi(60, 1.7))
print([nilai_huruf(s) for s in (90, 75, 65, 55, 20)])
print(is_palindrome("Kasur ini rusak"), is_palindrome("Python"))
print(bagi(10, 4), bagi(10, 0), bagi(10, "a"))
print(input_angka("Angka: "))
```

OUTPUT (INPUT SIMULASI: DUA ⏎ 2.5):

```
20.761245674740486
['A', 'B', 'C', 'D', 'E']
True False
2.5 None None
Angka: dua
Angka saja.
Angka: 2.5
2.5
```

11. Checklist Sebelum Modul 3

- Function + docstring + latihan utama selesai
- Comprehension / lambda sudah dicoba di kasus berbeda
- try/except dipakai di input atau file
- CSV: tulis + baca berhasil
- Minimal Project 1 selesai dan datanya persist
- Siap Modul 3 (OOP)

> **Modul 2 (lengkap) selesai? Lanjut ke Modul 3 — OOP.**
>
> Python Intensive — Modul Belajar Mandiri

3

Modul 3 — Object-Oriented Programming

Penjelasan lebih jelas · Kasus beragam · Pseudocode di latihan ·
Estimasi: ± 6 jam · Prasyarat: Modul 1–2

1. Apa yang Akan Anda Pelajari
2. Class, Object, Attribute & Method
3. Inheritance (Pewarisan)
4. Encapsulation & Special Methods
5. Mini Project 1 — Manajemen Tugas Kuliah
6. Mini Project 2 — Perpustakaan dengan Inheritance
7. Troubleshooting
8. Solusi Latihan (Cuplikan)
9. Checklist Sebelum Modul 4

> **Catatan pengujian:** seluruh blok kode pada modul ini dijalankan (Python 3.11). Bagian **Output** adalah hasil eksekusi nyata. Blok yang memakai `input()` dijalankan dengan input simulasi yang ditulis di samping label output. Blok yang hanya berisi definisi/komentar diberi keterangan, dan "Uji pelengkap" ditambahkan agar function/class pada blok tersebut terbukti berjalan. Kasus error pada bagian Troubleshooting dijalankan nyata di "Demonstrasi error".

1. Apa yang Akan Anda Pelajari

- Memahami class dan object, serta cara menyimpan data dan perilaku dalam satu unit.
- Membedakan data milik satu object dengan data yang dibagi seluruh class.
- Membuat class anak yang mewarisi class induk, dan menyesuaikan perilakunya.
- Memakai **str**, encapsulation sederhana, dan menyelesaikan 2 mini project.

> **CATATAN**
> **Cara membaca modul ini**
>
> Bagian penjelasan sengaja dibuat lebih panjang agar alurnya mudah diikuti.
>
> Baca penjelasan dulu, baru ketik contoh. Ubah angka/nama agar terasa hidup.
>
> Di latihan: baca soal → ikuti pseudocode → tulis kode sendiri → baru bandingkan solusi.

2. Class, Object, Attribute & Method

2.1 Apa itu OOP, dengan bahasa sederhana

Sampai Modul 2, Anda banyak bekerja dengan function dan data terpisah. Misalnya list of dict mahasiswa di satu sisi, dan function yang mengolah list itu di sisi lain. Itu sah dan sering cukup.

Object-Oriented Programming (OOP) menawarkan cara lain: menggabungkan data dan operasi yang berkaitan ke dalam satu “benda” bernama object. Class adalah cetakannya. Object adalah hasil cetakan itu di dunia nyata program.

Analogi singkat: class Mahasiswa mirip formulir kosong “Data Mahasiswa”. Object m1 = Mahasiswa("Ani", ...) mirip formulir yang sudah diisi untuk Ani. Formulir kosong (class) bisa dipakai berkali-kali untuk orang berbeda; tiap isian (object) punya data sendiri.

Kapan OOP membantu? Saat Anda punya banyak entitas sejenis (banyak mahasiswa, banyak buku, banyak rekening) dan tiap entitas punya aturan yang sama (setor, tarik, update IPK). Kapan tidak wajib? Script pendek sekali jalan sering lebih jelas tetap memakai function biasa.

2.2 Bagian-bagian class

Di dalam class biasanya ada:

(1) **init** — method khusus yang dipanggil otomatis saat object dibuat. Isinya menyiapkan data awal.

(2) Attribute — data yang “nempel” di object, misalnya self.nama. Kata self merujuk ke object yang sedang dipakai.

(3) Method — function yang hidup di dalam class. Parameter pertamanya selalu self agar method tahu object mana yang sedang dilayani.

Alur membuat object: Anda menulis Mahasiswa("Ani", "2201"). Python membuat object kosong, memanggil **init**, mengisi self.nama dan self.nim, lalu menyerahkan object itu ke variabel Anda.

2.3 Contoh bertahap

Contoh 1 — Mahasiswa. Perhatikan: update_ipk memeriksa rentang sebelum mengubah data. Itu pola penting: method menjaga agar data tidak menjadi tidak masuk akal.

```
class Mahasiswa:
    def __init__(self, nama, nim, ipk=0.0):
        self.nama = nama
        self.nim = nim
        self.ipk = ipk

    def perkenalan(self):
        return f"{self.nama} ({self.nim}) — IPK {self.ipk}"

    def update_ipk(self, ipk_baru):
        if 0 <= ipk_baru <= 4:
            self.ipk = ipk_baru
        else:
            print("IPK tidak valid")

m1 = Mahasiswa("Ani", "2201", 3.5)
m2 = Mahasiswa("Budi", "2202")
print(m1.perkenalan())
m2.update_ipk(3.2)
print(m2.perkenalan())
```

OUTPUT:

```
Ani (2201) — IPK 3.5
Budi (2202) — IPK 3.2
```

Contoh 2 — Produk toko. Method total menghitung harga belanja; kurangi_stok mengembalikan True/False agar pemanggil tahu apakah stok cukup.

```python
class Produk:
    def __init__(self, nama, harga, stok):
        self.nama = nama
        self.harga = harga
        self.stok = stok

    def total(self, qty):
        return self.harga * qty

    def kurangi_stok(self, qty):
        if qty <= self.stok:
            self.stok -= qty
            return True
        return False

p = Produk("Buku Python", 75000, 10)
print(p.total(2))
print(p.kurangi_stok(3), p.stok)
```

OUTPUT:

```
150000
True 7
```

Contoh 3 — Titik pada bidang. Method jarak_ke menerima object Titik lain. Ini menunjukkan object bisa saling berinteraksi.

```python
import math

class Titik:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def jarak_ke(self, lain):
        return math.hypot(self.x - lain.x, self.y - lain.y)

a = Titik(0, 0)
b = Titik(3, 4)
print(a.jarak_ke(b))  # 5.0
```

OUTPUT:

```
5.0
```

2.4 Class attribute vs instance attribute

Instance attribute adalah data milik satu object: self.nama pada m1 tidak otomatis sama dengan self.nama pada m2.

Class attribute ditulis di tubuh class (sejajar dengan def), bukan di dalam **init**. Nilainya dibagi bersama. Contoh klasik: penghitung berapa object sudah dibuat.

```python
class Mahasiswa:
    jumlah = 0  # class attribute

    def __init__(self, nama):
        self.nama = nama  # instance attribute
        Mahasiswa.jumlah += 1

a = Mahasiswa("Ani")
b = Mahasiswa("Budi")
print(Mahasiswa.jumlah)  # 2
print(a.nama, b.nama)    # Ani Budi — berbeda per object
```

OUTPUT:

```
2
Ani Budi
```

PERHATIAN

Kesalahan yang sering muncul

Lupa menulis self di parameter method → TypeError saat method dipanggil.

Lupa self. saat mengisi attribute di **init** → variabel lokal hilang setelah **init** selesai.

Mengisi list/dict sebagai class attribute lalu mengubah isinya — perubahan bisa “bocor” ke semua instance. Untuk data per object, isi di **init**.

2.5 Latihan Class Dasar

Kerjakan satu per satu. Ikuti pseudocode jika masih bingung menyusun urutan.

1. Class RekeningBank: pemilik & saldo; method setor, tarik (hanya jika saldo cukup), dan info.

```
Pseudocode:
  CLASS RekeningBank
    SAAT dibuat: simpan pemilik dan saldo awal
    setor(jumlah): jika jumlah > 0, tambahkan ke saldo
    tarik(jumlah): jika jumlah valid dan saldo cukup, kurangi saldo
                   jika tidak, tampilkan pesan gagal
    info: kembalikan teks berisi pemilik dan saldo
```

2. Class Buku: judul, penulis, tahun; method ringkas() mengembalikan satu string rapi.

```
Pseudocode:
  CLASS Buku
    SAAT dibuat: simpan judul, penulis, tahun
    ringkas(): kembalikan "judul — penulis (tahun)"
```

3. Class Keranjang: menyimpan daftar item; method tambah, total_harga, jumlah_item.

```
Pseudocode:
  CLASS Keranjang
    SAAT dibuat: items ← list kosong
    tambah(nama, harga): masukkan pasangan nama-harga ke items
    total_harga(): jumlahkan semua harga di items
    jumlah_item(): banyak elemen di items
```

4. Class Mahasiswa dengan class attribute yang menghitung berapa instance sudah dibuat.

```
Pseudocode:
  CLASS Mahasiswa
    jumlah ← 0   (milik class)
    SAAT dibuat: simpan nama; jumlah bertambah 1
```

5. Class Suhu menyimpan nilai Celsius; method ke_fahrenheit() dan ke_kelvin().

```
Pseudocode:
  CLASS Suhu
    SAAT dibuat: simpan celsius
    ke_fahrenheit(): hitung c×9/5+32
    ke_kelvin(): hitung c+273.15
```

3. Inheritance (Pewarisan)

3.1 Ide dasar

Kadang dua class hampir sama, hanya berbeda di detail. Misalnya Mobil dan Motor sama-sama punya merek dan tahun, tetapi Mobil punya jumlah pintu, Motor punya jenis (bebek/sport).

Inheritance memungkinkan Anda menulis bagian yang sama sekali saja di class induk (Kendaraan), lalu class anak (Mobil, Motor) tinggal menambahkan yang khas. Hubungan yang cocok disebut “is-a”: mobil adalah sebuah kendaraan.

Yang tidak cocok: memaksa inheritance hanya karena ingin memakai ulang satu function kecil. Dalam kasus itu, function biasa atau komposisi (object A menyimpan object B) sering lebih sederhana.

3.2 super() dan override

Saat class anak punya **init** sendiri, bagian inisialisasi induk tidak otomatis jalan. Anda memanggil super().**init**(...) agar attribute induk tetap terisi.

Override berarti class anak menulis ulang method yang namanya sama dengan milik induk. Dari luar, pemanggil tetap menulis .info(), tetapi perilaku mengikuti jenis object yang sebenarnya (Mobil atau Motor).

```python
class Kendaraan:
    def __init__(self, merek, tahun):
        self.merek = merek
        self.tahun = tahun

    def info(self):
        return f"{self.merek} ({self.tahun})"

class Mobil(Kendaraan):
    def __init__(self, merek, tahun, pintu):
        super().__init__(merek, tahun)  # isi merek & tahun di induk
        self.pintu = pintu

    def info(self):
        # pakai info induk, lalu tambahkan detail
        return f"{super().info()} — {self.pintu} pintu"

class Motor(Kendaraan):
    def __init__(self, merek, tahun, jenis):
        super().__init__(merek, tahun)
        self.jenis = jenis

m = Mobil("Toyota", 2020, 4)
print(m.info())
print(isinstance(m, Kendaraan))  # True — Mobil adalah Kendaraan
```

OUTPUT:

```
Toyota (2020) — 4 pintu
True
```

Contoh lain di dunia akademik: Pegawai punya gaji pokok; Manager menambah tunjangan. Method gaji() di Manager mengoverride versi Pegawai.

```python
class Pegawai:
    def __init__(self, nama, gaji_pokok):
        self.nama = nama
        self.gaji_pokok = gaji_pokok

    def gaji(self):
        return self.gaji_pokok

class Manager(Pegawai):
    def __init__(self, nama, gaji_pokok, tunjangan):
        super().__init__(nama, gaji_pokok)
        self.tunjangan = tunjangan

    def gaji(self):
        return self.gaji_pokok + self.tunjangan

p = Pegawai("Siti", 5_000_000)
m = Manager("Rudi", 7_000_000, 2_000_000)
print(p.nama, p.gaji())
print(m.nama, m.gaji())
```

OUTPUT:

```
Siti 5000000
Rudi 9000000
```

3.3 Latihan Inheritance

1. Implementasikan Pegawai dan Manager seperti di atas; buat satu instance masing-masing dan cetak gajinya.

```
Pseudocode:
  TULIS class Pegawai dan Manager (lihat pola contoh)
  buat pegawai biasa dan seorang manager
  cetak nama dan hasil gaji() masing-masing
```

2. Class Hewan(nama) dengan method bersuara(); Kucing dan Anjing mengoverride bunyi yang berbeda.

```
Pseudocode:
  CLASS Hewan: simpan nama; bersuara() kembalikan teks generik
  CLASS Kucing(Hewan): bersuara() → "meong"
  CLASS Anjing(Hewan): bersuara() → "guk"
  uji: buat satu kucing dan satu anjing, panggil bersuara()
```

3. AkunBank → AkunTabungan yang punya suku bunga dan method tambah_bunga().

```
Pseudocode:
  CLASS AkunBank: saldo, setor, tarik
  CLASS AkunTabungan(AkunBank): simpan suku_bunga
    tambah_bunga(): saldo bertambah sebesar saldo × suku_bunga
```

4. BangunDatar dengan method luas(); Persegi dan PersegiPanjang mengisi rumus masing-masing.

```
Pseudocode:
  CLASS BangunDatar: luas() kembalikan 0 (placeholder)
  CLASS Persegi: simpan sisi; luas() → sisi × sisi
  CLASS PersegiPanjang: simpan panjang & lebar; luas() → panjang × lebar
```

4. Encapsulation & Special Methods

4.1 Encapsulation — menjaga data lewat pintu resmi

Encapsulation berarti menyembunyikan detail internal dan hanya membuka cara yang aman untuk mengubah data. Di Python ini lebih ke konvensi daripada kunci ketat.

Attribute yang diawali satu garis bawah (_saldo) dibaca sebagai: “ini internal, jangan diubah sembarangan dari luar.” Yang diawali dua garis bawah memicu name mangling (nama diubah di belakang layar). Untuk level modul ini, cukup pahami _ sebagai sinyal internal.

Praktik yang baik: sediakan method setor, tarik, atau get_saldo. Dengan begitu aturan “saldo tidak boleh negatif” bisa ditegakkan di satu tempat.

```python
class Rekening:
    def __init__(self, pemilik, saldo=0):
        self.pemilik = pemilik
        self._saldo = saldo  # internal

    def setor(self, jumlah):
        if jumlah > 0:
            self._saldo += jumlah

    def get_saldo(self):
        return self._saldo

r = Rekening("Ani", 1000)
r.setor(500)
print(r.get_saldo())  # 1500
```

OUTPUT:

```
1500
```

4.2 Special methods — str, repr, eq

Method dengan nama berungkung ganda (dunder) dipanggil Python pada situasi tertentu. Tiga yang paling sering Anda butuhkan di awal:

str — dipanggil saat print(object) atau str(object). Isinya teks yang enak dibaca manusia.

repr — representasi untuk debug; idealnya jelas menunjukkan cara membuat object itu.

eq — menentukan kapan dua object dianggap sama dengan operator ==.

```python
class Buku:
    def __init__(self, judul, penulis):
        self.judul = judul
        self.penulis = penulis

    def __str__(self):
        return f"{self.judul} — {self.penulis}"

    def __repr__(self):
        return f"Buku({self.judul!r}, {self.penulis!r})"

    def __eq__(self, other):
        if not isinstance(other, Buku):
            return False
        return self.judul == other.judul and self.penulis == other.penulis

b1 = Buku("Laskar Pelangi", "Andrea Hirata")
b2 = Buku("Laskar Pelangi", "Andrea Hirata")
print(b1)        # memakai __str__
print(b1 == b2)  # True karena __eq__
```

OUTPUT:

```
Laskar Pelangi — Andrea Hirata
True
```

4.3 Context manager (with) — sekilas

Pernyataan with memastikan resource (misalnya file) ditutup dengan benar. Object yang mendukung with perlu method **enter** dan **exit**. open() bawaan Python sudah mendukung ini; contoh di bawah hanya untuk memperlihatkan polanya.

```
class FileCatatan:
    def __init__(self, path, mode="w"):
        self.path = path
        self.mode = mode
        self.file = None

    def __enter__(self):
        self.file = open(self.path, self.mode, encoding="utf-8")
        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        if self.file:
            self.file.close()
        return False  # jangan menelan exception

with FileCatatan("log.txt") as f:
    f.write("Catatan penting\n")
```

OUTPUT:

```
(tidak ada output)
```

Uji pelengkap — membuktikan isi file dan file tertutup

```
from pathlib import Path
print(Path("log.txt").read_text(encoding="utf-8"), end="")
print(f.closed)
```

OUTPUT:

```
Catatan penting
True
```

4.4 Latihan

1. Tambahkan **str** pada class RekeningBank dari latihan sebelumnya.

```
Pseudocode:
  DI dalam RekeningBank
    definisikan __str__
    kembalikan teks yang memuat pemilik dan saldo
  uji dengan print(rekening)
```

2. Class Titik(x, y) dengan **str** berbentuk (x, y) dan method jarak_ke.

```
Pseudocode:
  CLASS Titik: simpan x, y
  jarak_ke(titik_lain): hitung jarak Euclidean
  __str__: kembalikan "(x, y)"
```

3. Class Fraksi(pembilang, penyebut): tolak penyebut 0; method nilai_nyata(); **str** sebagai "a/b".

```
Pseudocode:
  SAAT dibuat: jika penyebut = 0 → beritahu error
  nilai_nyata(): kembalikan pembilang / penyebut
  __str__: "pembilang/penyebut"
```

4. Pastikan dua object Buku dianggap sama dengan == jika judul dan penulis sama.

```
Pseudocode:
  DEF __eq__(self, other)
    jika other bukan Buku → False
    True hanya jika judul sama dan penulis sama
```

5. Mini Project 1 — Manajemen Tugas Kuliah

Anda akan membuat dua class yang bekerja sama. Class Tugas menyimpan data satu tugas. Class ManajerTugas menyimpan daftar tugas dan menyediakan operasi tambah, tampil, tandai selesai, serta filter yang belum selesai.

Pisahkan tanggung jawab itu dengan sengaja: jangan menaruh semua logika di satu class besar. Menu interaktif di bagian bawah hanya memanggil method manajer.

Fitur wajib

1. Class Tugas: matkul, deskripsi, deadline, status selesai, plus **str**.
2. Class ManajerTugas: tambah, tampilkan, tandai (index), filter belum selesai.
3. Menu teks sederhana di main.

```python
class Tugas:
    def __init__(self, matkul, deskripsi, deadline):
        self.matkul = matkul
        self.deskripsi = deskripsi
        self.deadline = deadline
        self.selesai = False

    def tandai_selesai(self):
        self.selesai = True

    def __str__(self):
        st = "✓" if self.selesai else "○"
        return f"[{st}] {self.matkul}: {self.deskripsi} ({self.deadline})"

class ManajerTugas:
    def __init__(self):
        self.daftar = []

    def tambah(self, tugas):
        self.daftar.append(tugas)

    def tampilkan(self):
        for i, t in enumerate(self.daftar, 1):
            print(f"{i}. {t}")

    def tandai(self, index):
        if 1 <= index <= len(self.daftar):
            self.daftar[index - 1].tandai_selesai()

    def belum_selesai(self):
        return [t for t in self.daftar if not t.selesai]

# while True: menu → panggil method yang sesuai
```

OUTPUT:

```
(tidak ada output)
```

Catatan: blok ini hanya mendefinisikan class; menu `while True` *dibiarkan sebagai tugas peserta (lihat uji pelengkap).*

Uji pelengkap — menjalankan ManajerTugas (menu dibiarkan kosong pada modul)

```python
m = ManajerTugas()
m.tambah(Tugas("Python", "Laporan Modul 3", "2026-10-05"))
m.tambah(Tugas("Basis Data", "ERD", "2026-10-07"))
m.tambah(Tugas("Jaringan", "Subnetting", "2026-10-09"))
m.tampilkan()
m.tandai(2)
m.tandai(9)  # indeks tidak valid diabaikan
print("--- belum selesai ---")
for t in m.belum_selesai():
    print(t)
```

OUTPUT:

```
1. [○] Python: Laporan Modul 3 (2026-10-05)
2. [○] Basis Data: ERD (2026-10-07)
3. [○] Jaringan: Subnetting (2026-10-09)
--- belum selesai ---
[○] Python: Laporan Modul 3 (2026-10-05)
[○] Jaringan: Subnetting (2026-10-09)
```

TIPS

Urutan mengerjakan Project 1

1. Selesaikan class Tugas dan uji dengan print(tugas).
2. Selesaikan ManajerTugas tanpa menu dulu (panggil method manual).
3. Baru pasang loop menu.

Ingat: index di menu biasanya mulai 1; index list mulai 0.

6. Mini Project 2 — Perpustakaan dengan Inheritance

Class ItemPerpustakaan menjadi induk: semua item punya judul dan status tersedia/dipinjam, plus method pinjam dan kembalikan. Buku dan Jurnal adalah jenis item yang menambahkan detail masing-masing dan menyesuaikan tampilan **str**.

Class Perpustakaan tidak mewarisi Item; ia memiliki koleksi (list) item. Ini contoh komposisi: perpustakaan “mempunyai” banyak item, bukan “adalah” sebuah item.

Fitur wajib

1. ItemPerpustakaan: judul, status, pinjam, kembalikan.
2. Buku dan Jurnal: attribute tambahan + **str** jelas.
3. Perpustakaan: tambah, cari, pinjam, kembalikan, tampilkan.

```python
class ItemPerpustakaan:
    def __init__(self, judul):
        self.judul = judul
        self.tersedia = True

    def pinjam(self):
        if self.tersedia:
            self.tersedia = False
            return True
        return False

    def kembalikan(self):
        self.tersedia = True

class Buku(ItemPerpustakaan):
    def __init__(self, judul, penulis, halaman):
        super().__init__(judul)
        self.penulis = penulis
        self.halaman = halaman

    def __str__(self):
        st = "tersedia" if self.tersedia else "dipinjam"
        return f"Buku: {self.judul} — {self.penulis} [{st}]"

class Jurnal(ItemPerpustakaan):
    def __init__(self, judul, volume, tahun):
        super().__init__(judul)
        self.volume = volume
        self.tahun = tahun

    def __str__(self):
        st = "tersedia" if self.tersedia else "dipinjam"
        return f"Jurnal: {self.judul} Vol.{self.volume} ({self.tahun}) [{st}]"

class Perpustakaan:
    def __init__(self):
        self.koleksi = []

    def tambah(self, item):
        self.koleksi.append(item)

    def cari(self, keyword):
        k = keyword.lower()
        return [i for i in self.koleksi if k in i.judul.lower()]

    def pinjam(self, judul):
        for i in self.koleksi:
            if i.judul.lower() == judul.lower():
                return i.pinjam()
        return False

    def tampilkan(self):
        for i in self.koleksi:
            print(i)
```

OUTPUT:

```
(tidak ada output)
```

Catatan: blok ini hanya mendefinisikan class (tanpa pemanggilan); lihat uji pelengkap.

Uji pelengkap — menjalankan Perpustakaan

```
p = Perpustakaan()
p.tambah(Buku("Python Dasar", "Ani", 250))
p.tambah(Buku("Basis Data", "Budi", 300))
p.tambah(Jurnal("Jurnal Informatika", 12, 2025))
p.tampilkan()
print(p.pinjam("python dasar"))
print(p.pinjam("python dasar"))
print(p.pinjam("tidak ada"))
print([str(i) for i in p.cari("data")])
p.tampilkan()
```

OUTPUT:

```
Buku: Python Dasar — Ani [tersedia]
Buku: Basis Data — Budi [tersedia]
Jurnal: Jurnal Informatika Vol.12 (2025) [tersedia]
True
False
False
['Buku: Basis Data — Budi [tersedia]']
Buku: Python Dasar — Ani [dipinjam]
Buku: Basis Data — Budi [tersedia]
Jurnal: Jurnal Informatika Vol.12 (2025) [tersedia]
```

7. Troubleshooting

TypeError: **init**() missing argument — object dibuat tanpa memberi argumen wajib. Isi semua parameter yang diminta **init** (kecuali yang berdefault).

AttributeError — nama attribute salah ketik, atau attribute belum di-set di **init**. Cetak object.**dict** saat debug untuk melihat apa yang tersimpan.

Attribute induk “kosong” di class anak — hampir selalu karena lupa super().**init**(...) di **init** anak.

print(object) masih jelek — tambahkan **str** dan pastikan return-nya string, bukan print di dalam **str**.

Demonstrasi error — output nyata

Bagian Troubleshooting di atas berupa penjelasan teks. Berikut setiap kasusnya dijalankan nyata agar pesan error dan perilakunya terlihat. Alamat memori pada `<... object at 0x...>` berbeda di setiap eksekusi.

TypeError — argument `__init__` kurang

```
class Mahasiswa:
    def __init__(self, nama, nim, prodi):
        self.nama = nama

m = Mahasiswa("Ani", "2201")
```

OUTPUT:

```
  -> TypeError: Mahasiswa.__init__() missing 1 required positional argument: 'prodi'
```

TypeError — lupa `self` pada method

```
class Mahasiswa:
    def sapa():
        print("Halo")

Mahasiswa().sapa()
```

OUTPUT:

```
  -> TypeError: Mahasiswa.sapa() takes 0 positional arguments but 1 was given
```

AttributeError — attribute tidak ada

```
class Mahasiswa:
    def __init__(self, nama):
        self.nama = nama

m = Mahasiswa("Ani")
print(m.__dict__)
print(m.nim)
```

OUTPUT:

```
{'nama': 'Ani'}
  -> AttributeError: 'Mahasiswa' object has no attribute 'nim'
```

AttributeError — attribute dibuat di method lain yang belum dipanggil

```
class Keranjang:
    def isi(self):
        self.item = ["buku"]

k = Keranjang()
print(k.__dict__)
print(k.item)
```

OUTPUT:

```
{}
  -> AttributeError: 'Keranjang' object has no attribute 'item'
```

Lupa `super().__init__()` — attribute induk "kosong"

```
class Pegawai:
    def __init__(self, nama, gaji):
        self.nama = nama
        self.gaji = gaji

class Dosen(Pegawai):
    def __init__(self, nama, gaji, bidang):
        self.bidang = bidang    # super().__init__ tidak dipanggil

d = Dosen("Dr. Sari", 12000000, "ML")
print(d.__dict__)
print(d.nama)
```

OUTPUT:

```
{'bidang': 'ML'}
  -> AttributeError: 'Dosen' object has no attribute 'nama'
```

Perbaikan — panggil `super().__init__()`

```python
class Pegawai:
    def __init__(self, nama, gaji):
        self.nama = nama
        self.gaji = gaji

class Dosen(Pegawai):
    def __init__(self, nama, gaji, bidang):
        super().__init__(nama, gaji)
        self.bidang = bidang

d = Dosen("Dr. Sari", 12000000, "ML")
print(d.__dict__)
```

OUTPUT:

```
{'nama': 'Dr. Sari', 'gaji': 12000000, 'bidang': 'ML'}
```

`print(object)` jelek — tanpa `__str__`

```python
class Buku:
    def __init__(self, judul):
        self.judul = judul

print(Buku("Python"))
```

OUTPUT:

```
<__main__.Buku object at 0x7faae7140550>
```

`__str__` harus mengembalikan string, bukan mencetak

```python
class Buku:
    def __init__(self, judul):
        self.judul = judul
    def __str__(self):
        print(self.judul)     # salah: mencetak, tidak return

print(Buku("Python"))
```

OUTPUT:

```
Python
  -> TypeError: __str__ returned non-string (type NoneType)
```

`__str__` yang benar

```
class Buku:
    def __init__(self, judul):
        self.judul = judul
    def __str__(self):
        return f"Buku: {self.judul}"

print(Buku("Python"))
```

OUTPUT:

```
Buku: Python
```

Mutable default argument — semua object berbagi list yang sama

```
class Kelas:
    def __init__(self, nama, mahasiswa=[]):
        self.nama = nama
        self.mahasiswa = mahasiswa

a, b = Kelas("A"), Kelas("B")
a.mahasiswa.append("Ani")
print(a.mahasiswa, b.mahasiswa, a.mahasiswa is b.mahasiswa)
```

OUTPUT:

```
['Ani'] ['Ani'] True
```

Mutable default — perbaikan dengan `None`

```
class Kelas:
    def __init__(self, nama, mahasiswa=None):
        self.nama = nama
        self.mahasiswa = mahasiswa if mahasiswa is not None else []

a, b = Kelas("A"), Kelas("B")
a.mahasiswa.append("Ani")
print(a.mahasiswa, b.mahasiswa, a.mahasiswa is b.mahasiswa)
```

OUTPUT:

```
['Ani'] [] False
```

Atribut `__saldo` — name mangling

```
class Rekening:
    def __init__(self, saldo=0):
        self.__saldo = saldo

r = Rekening(100)
print(vars(r))
print(r._Rekening__saldo)
print(r.__saldo)
```

OUTPUT:

```
{'_Rekening__saldo': 100}
100
  -> AttributeError: 'Rekening' object has no attribute '__saldo'
```

8. Solusi Latihan (Cuplikan)

```python
class RekeningBank:
    def __init__(self, pemilik, saldo=0):
        self.pemilik = pemilik
        self.saldo = saldo

    def setor(self, jumlah):
        if jumlah > 0:
            self.saldo += jumlah

    def tarik(self, jumlah):
        if 0 < jumlah <= self.saldo:
            self.saldo -= jumlah
        else:
            print("Gagal tarik")

    def __str__(self):
        return f"{self.pemilik}: Rp{self.saldo:,}"

class Pegawai:
    def __init__(self, nama, gaji_pokok):
        self.nama = nama
        self.gaji_pokok = gaji_pokok
    def gaji(self):
        return self.gaji_pokok

class Manager(Pegawai):
    def __init__(self, nama, gaji_pokok, tunjangan):
        super().__init__(nama, gaji_pokok)
        self.tunjangan = tunjangan
    def gaji(self):
        return self.gaji_pokok + self.tunjangan

import math
class Titik:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def jarak_ke(self, lain):
        return math.hypot(self.x - lain.x, self.y - lain.y)
    def __str__(self):
        return f"({self.x}, {self.y})"
```

OUTPUT:

```
(tidak ada output)
```

Catatan: blok hanya mendefinisikan class solusi; lihat uji pelengkap.

Uji pelengkap — memanggil class solusi

```
r = RekeningBank("Ani", 1_000_000)
r.setor(250_000)
r.tarik(2_000_000)
r.tarik(300_000)
print(r)
for p in (Pegawai("Rina", 7_000_000), Manager("Budi", 9_000_000, 3_000_000)):
    print(p.nama, p.gaji())
print(Titik(0, 0).jarak_ke(Titik(3, 4)), Titik(1, 2))
```

OUTPUT:

```
Gagal tarik
Ani: Rp950,000
Rina 7000000
Budi 12000000
5.0 (1, 2)
```

9. Checklist Sebelum Modul 4

- Sudah bisa menjelaskan class vs object dengan kata sendiri
- Inheritance + super() pernah ditulis sendiri, bukan hanya dibaca
- **str** dipasang setidaknya di satu class buatan sendiri
- Mini Project 1 berjalan; Project 2 diusahakan
- Siap lanjut Modul 4 (NumPy & Pandas)

> **Modul 3 selesai? Lanjut ke Modul 4 — NumPy & Pandas.**
>
> Python Intensive — Modul Belajar Mandiri

4

Modul 4 — NumPy & Pandas

Pembahasan jelas · Banyak kasus · Pseudocode latihan · 5 Mini Project · Estimasi: ± 6–8 jam · Prasyarat: Modul 1–3 · pip install numpy pandas

1. Apa yang Akan Anda Pelajari
2. NumPy — Array Numerik
3. Pandas — Series & DataFrame
4. Cleaning & Operasi Lanjutan
5. Mini Project (5 Berjenjang)
6. Troubleshooting
7. Solusi Latihan (Cuplikan)
8. Checklist Sebelum Modul 5

> **Catatan pengujian:** seluruh blok kode pada modul ini dijalankan (Python 3.11, numpy 2.4.6, pandas 3.0.6, openpyxl 3.1.5). Bagian **Output** adalah hasil eksekusi nyata. Blok yang memakai `input()` dijalankan dengan input simulasi yang ditulis di samping label output. Blok yang hanya berisi definisi/komentar diberi keterangan, dan "Uji pelengkap" ditambahkan agar function/class pada blok tersebut terbukti berjalan. Kasus error pada bagian Troubleshooting dijalankan nyata di "Demonstrasi error". Pada pandas 3.x kolom teks bertipe `str` (di pandas 2.x tampil sebagai `object`).

1. Apa yang Akan Anda Pelajari

- Memakai NumPy untuk array, indexing, masking, broadcasting, dan agregasi.
- Membangun workflow Pandas: inspeksi, filter, cleaning, groupby, merge, pivot.
- Menyelesaikan minimal 2–3 dari 5 mini project berjenjang.

> **CATATAN**
> **Persiapan**
>
> pip install numpy pandas
>
> Jupyter Notebook sangat membantu karena output tabel langsung terlihat.
>
> Simpan kerja di portfolio/modul_04/.

2. NumPy — Array Numerik

2.1 Mengapa bukan list saja?

List Python fleksibel: bisa campur tipe, mudah diubah ukurannya. Namun untuk ribuan atau jutaan angka, operasi dengan loop Python terasa lambat. NumPy menyediakan array bertipe seragam dan operasi vektorisasi — satu perintah diterapkan ke seluruh elemen tanpa Anda menulis for secara eksplisit.

Anda tidak harus membuang list. Pakai NumPy saat data utamanya numerik dan Anda butuh hitung cepat, slicing multidimensi, atau fondasi untuk Pandas/visualisasi.

2.2 Membuat array

Array bisa dibuat dari list, atau dengan helper seperti zeros, ones, arange, linspace, dan generator acak. Perhatikan shape (bentuk) dan dtype (tipe elemen) — dua atribut yang paling sering dicek saat debug.

```python
import numpy as np

a = np.array([1, 2, 3, 4])
b = np.array([[1, 2], [3, 4], [5, 6]])
print(a.ndim, a.shape, a.dtype)
print(b.ndim, b.shape)

print(np.zeros((2, 3)))
print(np.ones((2, 2)))
print(np.arange(0, 10, 2))
print(np.linspace(0, 1, 5))

rng = np.random.default_rng(42)  # seed agar hasil bisa diulang
print(rng.integers(1, 10, size=5))
```

OUTPUT:

```
1 (4,) int64
2 (3, 2)
[[0. 0. 0.]
 [0. 0. 0.]]
[[1. 1.]
 [1. 1.]]
[0 2 4 6 8]
[0.   0.25 0.5  0.75 1.  ]
[1 7 6 4 4]
```

2.3 Indexing, slicing, boolean masking

Indexing 2D memakai [baris, kolom]. Slicing menyerupai list, tetapi hasilnya sering berupa view: mengubah slice bisa mengubah array asli. Boolean masking memilih elemen yang memenuhi kondisi — sangat berguna untuk filter nilai ujian, outlier, atau data di luar rentang.

```python
m = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])
print(m[0, 1])     # 20
print(m[:, 1])     # kolom tengah
print(m[1:, :2])   # submatriks

nilai = np.array([55, 80, 90, 40, 70, 95])
print(nilai[nilai >= 70])
print(nilai[(nilai >= 60) & (nilai < 90)])

label = np.where(nilai >= 70, "lulus", "mengulang")
print(label)
adjusted = np.where(nilai < 60, 60, nilai)  # floor 60
print(adjusted)
```

OUTPUT:

```
20
[20 50 80]
[[40 50]
 [70 80]]
[80 90 70 95]
[80 70]
['mengulang' 'lulus' 'lulus' 'mengulang' 'lulus' 'lulus']
[60 80 90 60 70 95]
```

PERHATIAN

Perhatian slicing

b = a[1:3] sering berbagi memori dengan a. Jika perlu salinan independen: b = a[1:3].copy()

Operator and/or Python biasa tidak dipakai antar array; pakai & | dengan tanda kurung.

2.4 Operasi, agregasi, broadcasting

Operasi + * ** pada array diterapkan per elemen. Agregasi (sum, mean, std) bisa global atau per sumbu: axis=0 biasanya “turun per kolom”, axis=1 “menyamping per baris” (untuk array 2D).

Broadcasting memungkinkan operasi antara array berbeda ukuran jika bentuknya kompatibel. Contoh penting: menormalisasi setiap kolom data dengan (data - mean) / std tanpa menulis loop.

```
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])
print(a + b, a * 2)

m = np.array([[1, 2, 3], [4, 5, 6]])
print(m.sum(axis=0))  # per kolom
print(m.sum(axis=1))  # per baris
print(m.mean(), m.std())

data = np.array([[10., 200.], [20., 300.], [30., 400.]])
mean = data.mean(axis=0)
std = data.std(axis=0)
print((data - mean) / std)
```

OUTPUT:

```
[11 22 33] [2 4 6]
[5 7 9]
[ 6 15]
3.5 1.707825127659933
[[-1.22474487 -1.22474487]
 [ 0.          0.        ]
 [ 1.22474487  1.22474487]]
```

2.5 Reshape

```
a = np.arange(12)
print(a.reshape(3, 4))
print(a.reshape(2, -1))  # -1 = otomatis
print(np.eye(3))
print(np.diag(np.eye(3)))
```

OUTPUT:

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]]
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
[1. 1. 1.]
```

2.6 Latihan NumPy

1. Buat array 6×6 berisi 1…36; ambil blok tengah 2×2; hitung mean, min, max blok itu.

```
Pseudocode:
  a ← arange 1..36, reshape 6×6
  blok ← a[baris tengah, kolom tengah]  # mis. [2:4, 2:4]
  cetak mean, min, max dari blok
```

2. Normalisasi min-max sebuah array 1D ke rentang 0…1.

```
Pseudocode:
  norm ← (x - min(x)) / (max(x) - min(x))
  hati-hati jika max = min (pembagi nol)
```

3. Ganti semua nilai < 60 menjadi 60 dengan np.where.

```
Pseudocode:
  hasil ← where(nilai < 60, 60, nilai)
```

4. Matriks identitas 5×5; ambil diagonal utamanya.

```
Pseudocode:
  I ← eye(5)
  cetak diag(I)
```

3. Pandas — Series & DataFrame

3.1 Ide dasar

Jika NumPy andal untuk angka murni, Pandas andal untuk tabel berlabel: baris dan kolom punya nama. Series mirip satu kolom; DataFrame mirip spreadsheet atau tabel database di memori.

Alur kerja tipikal: muat data → inspeksi (head, info, describe) → bersihkan → filter/groupby → simpan hasil. Modul ini membangun fondasi itu langkah demi langkah.

3.2 Series dan DataFrame

```
import pandas as pd
import numpy as np

s = pd.Series([80, 90, 75], index=["Ani", "Budi", "Cici"])
print(s["Ani"], s.mean())

df = pd.DataFrame({
    "nama": ["Ani", "Budi", "Cici", "Dodi", "Eka"],
    "nim": ["2201", "2202", "2203", "2204", "2205"],
    "prodi": ["IF", "SI", "IF", "SI", "IF"],
    "ipk": [3.50, 3.20, 3.80, 2.90, 3.60],
})
print(df.head())
print(df.info())
print(df.describe())
print(df["prodi"].value_counts())
```

OUTPUT:

```
80 81.66666666666667
   nama   nim prodi  ipk
0   Ani  2201    IF  3.5
1  Budi  2202    SI  3.2
2  Cici  2203    IF  3.8
3  Dodi  2204    SI  2.9
4   Eka  2205    IF  3.6
<class 'pandas.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   nama    5 non-null      str
 1   nim     5 non-null      str
 2   prodi   5 non-null      str
 3   ipk     5 non-null      float64
dtypes: float64(1), str(3)
memory usage: 292.0 bytes
None
            ipk
count  5.000000
mean   3.400000
std    0.353553
min    2.900000
25%    3.200000
50%    3.500000
75%    3.600000
max    3.800000
prodi
IF    3
SI    2
Name: count, dtype: int64
```

3.3 Selection: [], loc, iloc

Ambil kolom dengan df["nama"] atau df[["nama", "ipk"]]. loc memakai label (termasuk filter boolean). iloc memakai posisi angka seperti indeks list. Membiasakan ketiganya mencegah kebingungan saat index tidak berurutan 0,1,2,...

```
print(df["nama"])
print(df.loc[0, "ipk"])
print(df.loc[df["ipk"] >= 3.5, ["nama", "ipk"]])
print(df.iloc[0:3, 0:2])
print(df.query("ipk >= 3.5 and prodi == 'IF'"))
```

OUTPUT:

```
0     Ani
1    Budi
2    Cici
3    Dodi
4     Eka
Name: nama, dtype: str
3.5
   nama  ipk
0   Ani  3.5
2  Cici  3.8
4   Eka  3.6
   nama   nim
0   Ani  2201
1  Budi  2202
2  Cici  2203
   nama   nim prodi  ipk
0   Ani  2201    IF  3.5
2  Cici  2203    IF  3.8
4   Eka  2205    IF  3.6
```

3.4 Menambah kolom, filter, sort

```
df = df.copy()
df["status"] = df["ipk"].apply(lambda x: "unggul" if x >= 3.5 else "baik")
df["skor"] = df["ipk"] * 25
print(df[df["ipk"] >= 3.5].sort_values("ipk", ascending=False))
```

OUTPUT:

```
   nama   nim prodi  ipk  status  skor
2  Cici  2203    IF  3.8  unggul  95.0
4   Eka  2205    IF  3.6  unggul  90.0
0   Ani  2201    IF  3.5  unggul  87.5
```

4. Cleaning & Operasi Lanjutan

4.1 Missing values dan duplikat

Data nyata sering berlubang atau berulang. isna() memetakan lubang; dropna menghapus; fillna mengisi (mean, median, atau nilai tetap). Pilihan strategi bergantung pada konteks: menghapus baris mahasiswa karena satu nilai kosong bisa membuang informasi penting.

```python
df = pd.DataFrame({
    "nama": ["Ani", "Budi", None, "Ani"],
    "nilai": [80, np.nan, 70, 80],
})
print(df.isna().sum())
df["nilai"] = df["nilai"].fillna(df["nilai"].median())
df = df.dropna(subset=["nama"])
df = df.drop_duplicates()
print(df)
```

OUTPUT:

```
nama     1
nilai    1
dtype: int64
   nama  nilai
0   Ani   80.0
1  Budi   80.0
```

4.2 GroupBy

groupby memecah tabel menurut kunci (misalnya prodi), lalu menerapkan agregasi (mean, count, sum) pada tiap kelompok. Ini padanan “pivot sederhana” di spreadsheet.

```python
df = pd.DataFrame({
    "prodi": ["IF", "IF", "SI", "SI", "IF"],
    "nilai": [80, 90, 70, 85, 75],
})
print(df.groupby("prodi")["nilai"].mean())
print(df.groupby("prodi").agg(rata=("nilai", "mean"), n=("nilai", "count")))
```

OUTPUT:

```
prodi
IF    81.666667
SI    77.500000
Name: nilai, dtype: float64
            rata  n
prodi
IF     81.666667  3
SI     77.500000  2
```

4.3 Merge dan concat

Merge menggabungkan tabel berdasarkan kolom kunci (mirip JOIN SQL). how="left" mempertahankan semua baris kiri; inner hanya yang cocok di kedua sisi. Concat menumpuk tabel secara vertikal atau horizontal.

```python
mhs = pd.DataFrame({"nim": ["2201", "2202"], "nama": ["Ani", "Budi"]})
nilai = pd.DataFrame({"nim": ["2201", "2202"], "ipk": [3.5, 3.1]})
print(mhs.merge(nilai, on="nim", how="left"))
```

OUTPUT:

```
    nim  nama  ipk
0  2201   Ani  3.5
1  2202  Budi  3.1
```

4.4 Pivot table

```
df = pd.DataFrame({
    "prodi": ["IF", "IF", "SI", "SI"],
    "semester": [1, 2, 1, 2],
    "nilai": [80, 85, 70, 90],
})
print(df.pivot_table(values="nilai", index="prodi", columns="semester", aggfunc="mean"))
```

OUTPUT:

```
semester     1     2
prodi
IF        80.0  85.0
SI        70.0  90.0
```

4.5 Baca dan simpan file

```
df = pd.read_csv("data.csv")
df.to_csv("hasil.csv", index=False)
# pd.read_excel / df.to_excel memerlukan openpyxl
```

OUTPUT:

```
(tidak ada output)
```

Catatan: dijalankan dengan file contoh `data.csv` *(3 baris) yang dibuat untuk pengujian; blok ini sengaja tidak mencetak apa pun — hasil baca/tulis dapat dilihat pada file* `hasil.csv`*.*

Uji pelengkap — memeriksa hasil baca/tulis CSV

```
from pathlib import Path
print(df)
print(Path("hasil.csv").read_text())
```

OUTPUT:

```
    nama  ipk
0    Ani  3.7
1   Budi  3.2
2  Citra  3.9
nama,ipk
Ani,3.7
Budi,3.2
Citra,3.9
```

4.6 Latihan Pandas

1. DataFrame 8 mahasiswa (nama, prodi, ipk, sks). Filter IPK ≥ 3.5; sort IPK menurun.

```
Pseudocode:
  buat DataFrame
  subset ← baris dengan ipk ≥ 3.5
  urutkan subset by ipk descending
  cetak hasil
```

2. Groupby prodi: rata-rata IPK, total SKS, jumlah mahasiswa.

```
Pseudocode:
  groupby prodi
  agg: mean(ipk), sum(sks), count(nama)
```

3. Sengaja buat missing & duplikat; bersihkan; bandingkan shape sebelum/sesudah.

```
Pseudocode:
  catat shape dan jumlah na awal
  fillna / dropna / drop_duplicates sesuai kebutuhan
  catat shape akhir
```

4. Merge mahasiswa + nilai (how="left"); hitung berapa yang tidak punya nilai.

```
Pseudocode:
  gabung ← merge left on nim
  jumlah_na ← gabung["ipk"].isna().sum()
```

5. Mini Project (5 Berjenjang)

Minimal selesaikan 2–3 project. Kerjakan berurutan jika memungkinkan.

Project 1 — Kalkulator Nilai Akhir (Dasar)

Dari komponen tugas, UTS, UAS, hitung nilai akhir berbobot dan huruf mutu; tampilkan ringkasan kelas.

```python
import pandas as pd
df = pd.DataFrame({
    "nama": ["Ani", "Budi", "Cici", "Dodi"],
    "tugas": [80, 70, 90, 60],
    "uts": [85, 75, 88, 65],
    "uas": [90, 80, 85, 70],
})
df["nilai_akhir"] = 0.3*df["tugas"] + 0.3*df["uts"] + 0.4*df["uas"]

def ke_huruf(n):
    if n >= 85: return "A"
    if n >= 70: return "B"
    if n >= 60: return "C"
    if n >= 50: return "D"
    return "E"

df["huruf"] = df["nilai_akhir"].apply(ke_huruf)
print(df)
print("Rata kelas:", df["nilai_akhir"].mean())
print(df["huruf"].value_counts())
```

OUTPUT:

```
   nama  tugas  uts  uas  nilai_akhir huruf
0   Ani     80   85   90         85.5     A
1  Budi     70   75   80         75.5     B
2  Cici     90   88   85         87.4     A
3  Dodi     60   65   70         65.5     C
Rata kelas: 78.475
huruf
A    2
B    1
C    1
Name: count, dtype: int64
```

Project 2 — Cleaning Dataset Kotor

Rapikan missing, duplikat, dan tipe data; simpan CSV bersih; bandingkan shape.

```
df = pd.DataFrame({
    "nama": ["Ani", "Budi", None, "Ani", "Cici"],
    "nilai": ["80", "90", None, "80", "70"],
    "prodi": ["IF", "SI", "IF", "IF", None],
})
print("Sebelum:", df.shape)
df["nilai"] = pd.to_numeric(df["nilai"], errors="coerce")
df["nilai"] = df["nilai"].fillna(df["nilai"].median())
df["prodi"] = df["prodi"].fillna("unknown")
df = df.dropna(subset=["nama"]).drop_duplicates()
print("Sesudah:", df.shape)
df.to_csv("data_bersih.csv", index=False)
```

OUTPUT:

```
Sebelum: (5, 3)
Sesudah: (3, 3)
```

Project 3 — Merge Multi-Tabel

```
mhs = pd.DataFrame({"nim": ["2201", "2202", "2203"], "nama": ["Ani", "Budi", "Cici"],
"kode_prodi": ["IF", "SI", "IF"]})
nilai = pd.DataFrame({"nim": ["2201", "2202"], "ipk": [3.5, 3.1]})
prodi = pd.DataFrame({"kode_prodi": ["IF", "SI"], "nama_prodi": ["Informatika", "Sistem
Informasi"]})
df = mhs.merge(nilai, on="nim", how="left").merge(prodi, on="kode_prodi", how="left")
print(df)
print(df.groupby("nama_prodi")["ipk"].mean())
print("Tanpa nilai:", df["ipk"].isna().sum())
```

OUTPUT:

```
    nim  nama kode_prodi  ipk        nama_prodi
0  2201   Ani         IF  3.5       Informatika
1  2202  Budi         SI  3.1  Sistem Informasi
2  2203  Cici         IF  NaN       Informatika
nama_prodi
Informatika         3.5
Sistem Informasi    3.1
Name: ipk, dtype: float64
Tanpa nilai: 1
```

Project 4 — Pivot & Reporting

```
df = pd.DataFrame({
    "prodi": ["IF", "IF", "SI", "SI", "IF", "SI"],
    "semester": [1, 1, 1, 1, 2, 2],
    "nilai": [80, 85, 70, 75, 90, 88],
})
print(df.pivot_table(values="nilai", index="prodi", columns="semester", aggfunc="mean"))
print(df.groupby("prodi")["nilai"].agg(["mean", "count", "std"]))
```

OUTPUT:

```
semester     1     2
prodi
IF        82.5  90.0
SI        72.5  88.0
            mean  count       std
prodi
IF     85.000000      3  5.000000
SI     77.666667      3  9.291573
```

Project 5 — EDA Dataset Publik

Ambil dataset dari seaborn (penguins, tips, titanic) atau CSV publik. Inspeksi, cleaning ringan, minimal 2 groupby bermakna, tulis 3–5 insight, simpan ringkasan CSV.

```
import seaborn as sns
df = sns.load_dataset("penguins")
print(df.head()); print(df.info()); print(df.isna().sum())
print(df.groupby("species")["body_mass_g"].mean())
# tulis insight; simpan tabel ringkas
```

OUTPUT:

```
  species     island  bill_length_mm  bill_depth_mm  flipper_length_mm  body_mass_g     sex
0  Adelie  Torgersen            39.1           18.7              181.0       3750.0    Male
1  Adelie  Torgersen            39.5           17.4              186.0       3800.0  Female
2  Adelie  Torgersen            40.3           18.0              195.0       3250.0  Female
3  Adelie  Torgersen             NaN            NaN                NaN          NaN     NaN
4  Adelie  Torgersen            36.7           19.3              193.0       3450.0  Female
<class 'pandas.DataFrame'>
RangeIndex: 344 entries, 0 to 343
Data columns (total 7 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   species            344 non-null    str
 1   island             344 non-null    str
 2   bill_length_mm     342 non-null    float64
 3   bill_depth_mm      342 non-null    float64
 4   flipper_length_mm  342 non-null    float64
 5   body_mass_g        342 non-null    float64
 6   sex                333 non-null    str
dtypes: float64(4), str(3)
memory usage: 18.9 KB
None
species               0
island                0
bill_length_mm        2
bill_depth_mm         2
flipper_length_mm     2
body_mass_g           2
sex                  11
dtype: int64
species
Adelie       3700.662252
Chinstrap    3733.088235
Gentoo       5076.016260
Name: body_mass_g, dtype: float64
```

TIPS

Sumber cepat

sns.load_dataset("tips"), "penguins", "titanic"

Fokus ke pertanyaan analisis, bukan hanya memanggil fungsi.

6. Troubleshooting

```
# KeyError kolom → cek df.columns; perhatikan huruf besar/kecil
# SettingWithCopyWarning → pakai .copy() setelah filter, atau .loc[]
# Merge menghasilkan NaN → normal pada how="left" jika kunci tidak cocok
# dtype object padahal angka → pd.to_numeric(..., errors="coerce")
# Broadcasting error NumPy → cek shape; reshape bila perlu
```

> **OUTPUT:** *TIDAK DIJALANKAN* — HANYA KOMENTAR (DAFTAR JEBAKAN); CONTOH ERROR NYATA DIJALANKAN DI BAGIAN "DEMONSTRASI ERROR" DI BAWAH

Demonstrasi error — output nyata

Daftar jebakan di atas hanya berupa komentar. Berikut setiap kasusnya dijalankan nyata (pandas 3.x / numpy 2.x). Pada pandas 3.x chained assignment tidak mengubah `df` dan memunculkan peringatan `ChainedAssignmentError`; pada pandas 2.x hasilnya bisa tidak konsisten.

KeyError — nama kolom salah / ada spasi tersembunyi

```
import pandas as pd
import numpy as np
df = pd.DataFrame({"nama ": ["Ani"], "ipk": [3.7]})
print(df.columns.tolist())
print(df["nama"])
```

OUTPUT:

```
['nama ', 'ipk']
  -> KeyError: 'nama'
```

Solusi KeyError — bersihkan nama kolom

```
import pandas as pd
import numpy as np
df = pd.DataFrame({"nama ": ["Ani"], "ipk": [3.7]})
df.columns = df.columns.str.strip()
print(df["nama"])
```

OUTPUT:

```
0    Ani
Name: nama, dtype: str
```

Chained assignment — nilai tidak berubah

```
import pandas as pd
import numpy as np
df = pd.DataFrame({"a": [1, 2, 3], "b": [10, 20, 30]})
df[df["a"] > 1]["b"] = 0
print(df)
```

OUTPUT:

```
   a   b
0  1  10
1  2  20
2  3  30
[WARNING] ChainedAssignmentError: A value is being set on a copy of a DataFrame or Series through
chained assignment.
```

Perbaikan — pakai `.loc`

```
import pandas as pd
import numpy as np
df = pd.DataFrame({"a": [1, 2, 3], "b": [10, 20, 30]})
df.loc[df["a"] > 1, "b"] = 0
print(df)
```

OUTPUT:

```
   a   b
0  1  10
1  2   0
2  3   0
```

SettingWithCopy — gunakan `.copy()` pada subset yang diubah

```
import pandas as pd
import numpy as np
df = pd.DataFrame({"IPK": [3.2, 3.8, 3.6]})
subset = df[df["IPK"] > 3.5].copy()
subset["flag"] = 1
print(subset)
print(df.columns.tolist())
```

OUTPUT:

```
   IPK  flag
1  3.8     1
2  3.6     1
['IPK']
```

Merge menghasilkan NaN — normal pada `how="left"`

```
import pandas as pd
import numpy as np
a = pd.DataFrame({"nim": ["2201", "2202", "2203"], "nama": ["Ani", "Budi", "Citra"]})
b = pd.DataFrame({"nim": ["2201", "2202"], "skor": [85, 90]})
print(a.merge(b, on="nim", how="left"))
print(a.merge(b, on="nim", how="inner"))
```

OUTPUT:

```
    nim   nama  skor
0  2201    Ani  85.0
1  2202   Budi  90.0
2  2203  Citra   NaN
    nim  nama  skor
0  2201   Ani    85
1  2202  Budi    90
```

ValueError — merge pada kolom dengan tipe berbeda

```
import pandas as pd
import numpy as np
a = pd.DataFrame({"nim": [2201, 2202], "nama": ["Ani", "Budi"]})
b = pd.DataFrame({"nim": ["2201", "2202"], "skor": [85, 90]})
print(a.merge(b, on="nim"))
```

OUTPUT:

```
  -> ValueError: You are trying to merge on int64 and str columns for key 'nim'. If you wish to
proceed you should use pd.concat
```

Solusi — samakan tipe kunci merge

```
import pandas as pd
import numpy as np
a = pd.DataFrame({"nim": [2201, 2202], "nama": ["Ani", "Budi"]})
b = pd.DataFrame({"nim": ["2201", "2202"], "skor": [85, 90]})
a["nim"] = a["nim"].astype(str)
print(a.merge(b, on="nim"))
```

OUTPUT:

```
    nim  nama  skor
0  2201   Ani    85
1  2202  Budi    90
```

dtype object padahal angka — `to_numeric`

```
import pandas as pd
import numpy as np
s = pd.Series(["80", "90", "n/a"])
print(s.dtype)
print(pd.to_numeric(s, errors="coerce").tolist())
print(s.astype(int))
```

OUTPUT:

```
str
[80.0, 90.0, nan]
  -> ValueError: invalid literal for int() with base 10: 'n/a'
```

Broadcasting error NumPy — cek shape

```
import pandas as pd
import numpy as np
a = np.ones((3, 4))
b = np.array([1, 2, 3])
print(a.shape, b.shape)
print(a + b)
```

OUTPUT:

```
(3, 4) (3,)
  -> ValueError: operands could not be broadcast together with shapes (3,4) (3,)
```

Solusi — reshape agar broadcasting berhasil

```
import pandas as pd
import numpy as np
a = np.ones((3, 4))
b = np.array([1, 2, 3]).reshape(3, 1)
print(a + b)
```

OUTPUT:

```
[[2. 2. 2. 2.]
 [3. 3. 3. 3.]
 [4. 4. 4. 4.]]
```

ValueError — `and` / `or` pada filter DataFrame

```
import pandas as pd
import numpy as np
df = pd.DataFrame({"ipk": [3.7, 2.8], "prodi": ["IF", "SI"]})
print(df[(df["ipk"] > 3) and (df["prodi"] == "IF")])
```

OUTPUT:

```
  -> ValueError: The truth value of a Series is ambiguous. Use a.empty, a.bool(), a.item(),
a.any() or a.all().
```

Perbaikan — `&` dengan kurung

```
import pandas as pd
import numpy as np
df = pd.DataFrame({"ipk": [3.7, 2.8], "prodi": ["IF", "SI"]})
print(df[(df["ipk"] > 3) & (df["prodi"] == "IF")])
```

OUTPUT:

```
   ipk prodi
0  3.7    IF
```

7. Solusi Latihan (Cuplikan)

```
import numpy as np
a = np.arange(1, 37).reshape(6, 6)
blok = a[2:4, 2:4]
print(blok.mean(), blok.min(), blok.max())

x = np.array([2., 4., 6., 8.])
print((x - x.min()) / (x.max() - x.min()))

import pandas as pd
df = pd.DataFrame({
    "nama": list("ABCDEFGH"),
    "prodi": ["IF", "SI"] * 4,
    "ipk": [3.6, 3.1, 3.9, 2.8, 3.4, 3.7, 3.0, 3.5],
    "sks": [20, 18, 21, 19, 20, 22, 18, 19],
})
print(df[df["ipk"] >= 3.5].sort_values("ipk", ascending=False))
print(df.groupby("prodi").agg(rata=("ipk", "mean"), total_sks=("sks", "sum"), n=("nama",
"count")))
```

OUTPUT:

```
18.5 15 22
[0.         0.33333333 0.66666667 1.        ]
  nama prodi  ipk  sks
2    C    IF  3.9   21
5    F    SI  3.7   22
0    A    IF  3.6   20
7    H    SI  3.5   19
        rata  total_sks  n
prodi
IF     3.475         79  4
SI     3.275         78  4
```

8. Checklist Sebelum Modul 5

- Array NumPy: indexing, mask, agregasi sudah dicoba sendiri
- Pandas: loc/filter/groupby/merge pernah dijalankan
- Minimal 2–3 mini project selesai
- Siap Modul 5 — Visualisasi, API & Scraping

> **Modul 4 selesai? Lanjut ke Modul 5.**
>
> Python Intensive — Modul Belajar Mandiri

5

Modul 5 — Visualisasi, API & Scraping

Penjelasan jelas · Kasus beragam · Pseudocode · 4 Mini Project ·
Estimasi: ± 6 jam · Prasyarat: Modul 4 · Butuh koneksi internet (API)

1. Apa yang Akan Anda Pelajari
2. Matplotlib — Visualisasi Dasar
3. Seaborn — Visualisasi Statistik
4. Bekerja dengan API (requests)
5. Scraping & arXiv API
6. Mini Project
7. Troubleshooting
8. Solusi Latihan (Cuplikan)
9. Checklist Sebelum Modul 6

Catatan pengujian: seluruh blok kode pada modul ini dijalankan (Python 3.11, matplotlib 3.11.2, seaborn 0.13.2, pandas 3.0.6, requests, beautifulsoup4 4.15, feedparser 6.0.14). Bagian **Output** adalah hasil eksekusi nyata. Blok yang memakai `input()` dijalankan dengan input simulasi yang ditulis di samping label output. Blok yang hanya berisi definisi/komentar diberi keterangan, dan "Uji pelengkap" ditambahkan agar function/class pada blok tersebut terbukti berjalan. Kasus error pada bagian Troubleshooting dijalankan nyata di "Demonstrasi error". Grafik hasil eksekusi disimpan sebagai gambar di `materi/images/modul05/` dan ditampilkan di bawah kodenya. Peringatan `MatplotlibDeprecationWarning` (boxplot/violinplot) berasal dari kombinasi seaborn 0.13.2 dengan matplotlib 3.11 dan tidak memengaruhi hasil. Host API selain GitHub diblokir di lingkungan pengujian: blok yang memanggil jsonplaceholder, open-meteo, arXiv, dan quotes.toscrape dijalankan dengan respons tiruan dan diberi catatan; kasus error `requests` memakai server HTTP lokal.

1. Apa yang Akan Anda Pelajari

- Membuat plot informatif dengan Matplotlib dan Seaborn (label, judul, simpan file).
- Mengambil data dari API publik dengan requests dan mengubahnya ke DataFrame.
- Memahami etika scraping; memakai BeautifulSoup dan arXiv API.
- Menyelesaikan minimal 2 dari 4 mini project.

CATATAN

Persiapan

pip install matplotlib seaborn requests beautifulsoup4 feedparser lxml

Simpan figure di portfolio/modul_05/figures/

API membutuhkan internet; siapkan cadangan CSV jika jaringan tidak stabil.

2. Matplotlib — Visualisasi Dasar

2.1 Ide dasar

Visualisasi mengubah angka menjadi bentuk yang lebih cepat ditangkap mata: tren, perbandingan, sebaran, proporsi. Matplotlib adalah fondasi plotting di Python. Pola umumnya: siapkan data → pilih jenis plot → beri judul dan label sumbu → tampilkan atau simpan ke file.

Plot tanpa label sulit dibaca orang lain (dan Anda sendiri nanti). Biasakan title, xlabel, ylabel, dan bila perlu legend.

2.2 Line plot

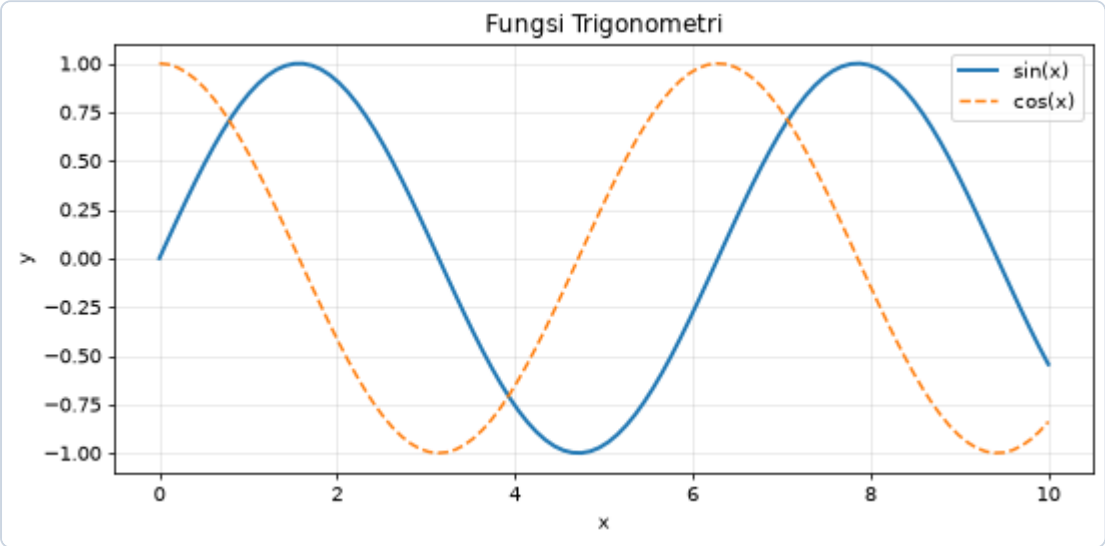
Line cocok untuk data berurutan: waktu, iterasi, atau fungsi matematika. Anda bisa menumpuk beberapa garis di satu figure.

```python
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)
plt.figure(figsize=(8, 4))
plt.plot(x, np.sin(x), label="sin(x)", linewidth=2)
plt.plot(x, np.cos(x), label="cos(x)", linestyle="--")
plt.title("Fungsi Trigonometri")
plt.xlabel("x"); plt.ylabel("y")
plt.legend(); plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("line_plot.png", dpi=150)
plt.show()
```

OUTPUT:

```
(tidak ada output)
```



2.3 Bar, scatter, histogram, pie

Bar membandingkan kategori (jumlah mhs per prodi). Scatter melihat hubungan dua variabel numerik. Histogram melihat sebaran satu variabel. Pie menampilkan proporsi — gunakan hemat, karena sulit dibanding-bandingkan sudut.

```
prodi = ["IF", "SI", "TI", "DKV"]
jumlah = [120, 95, 80, 60]
plt.figure(figsize=(7, 4))
bars = plt.bar(prodi, jumlah, color="steelblue")
plt.title("Jumlah Mahasiswa per Prodi")
plt.ylabel("Jumlah")
for bar, j in zip(bars, jumlah):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 2,
             str(j), ha="center")
plt.tight_layout(); plt.show()

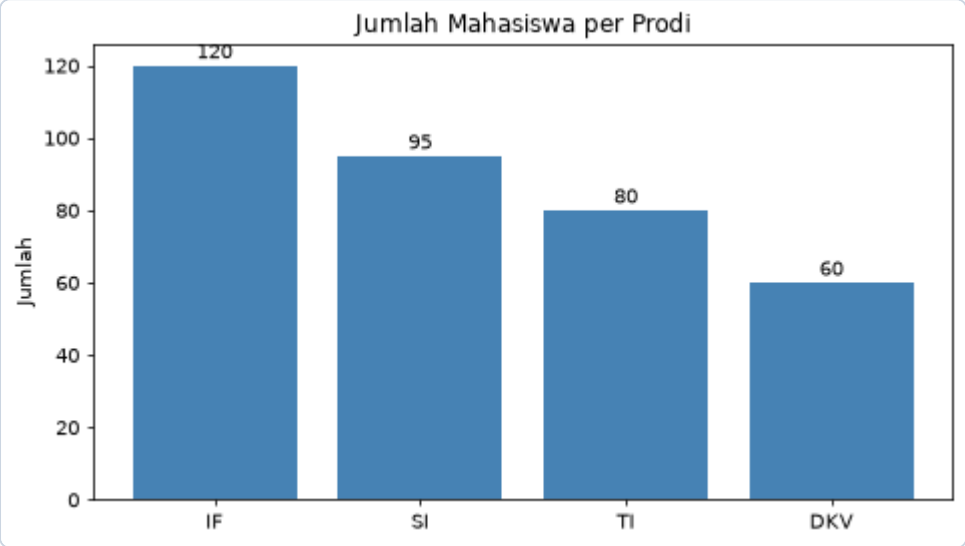
rng = np.random.default_rng(42)
x = rng.normal(70, 10, 80)
y = 0.8 * x + rng.normal(0, 5, 80)
plt.scatter(x, y, alpha=0.7)
plt.xlabel("Nilai X"); plt.ylabel("Nilai Y")
plt.title("Scatter"); plt.show()

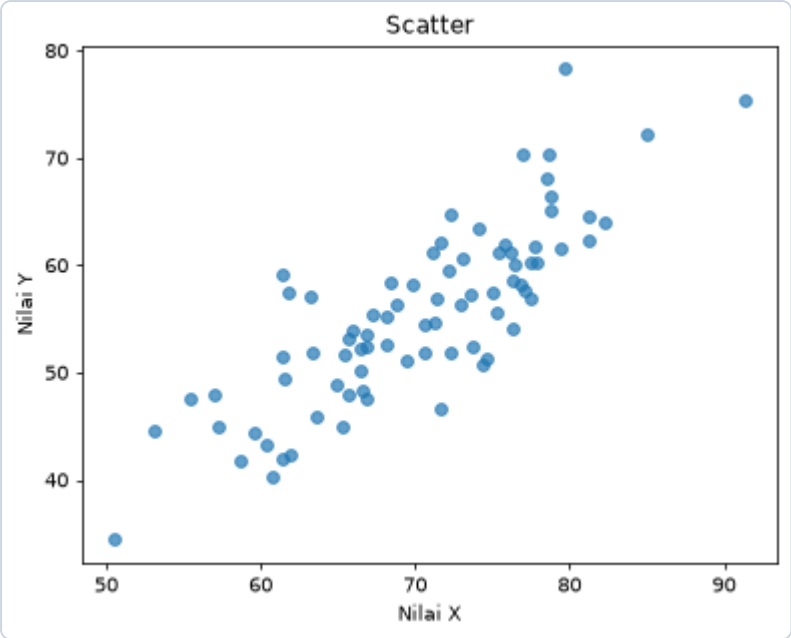
plt.hist(x, bins=15, edgecolor="white")
plt.title("Distribusi Nilai"); plt.show()

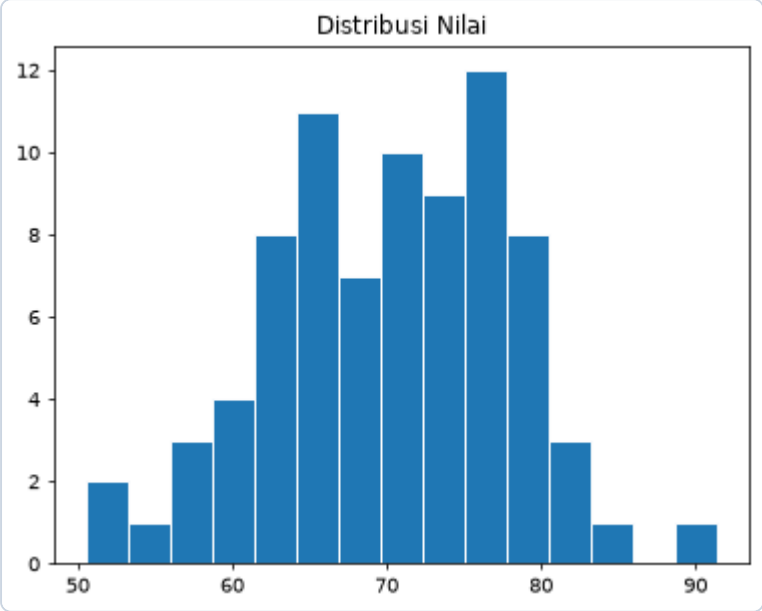
plt.pie([30, 25, 25, 20], labels=list("ABCD"), autopct="%1.1f%%")
plt.title("Proporsi Grade"); plt.axis("equal"); plt.show()
```

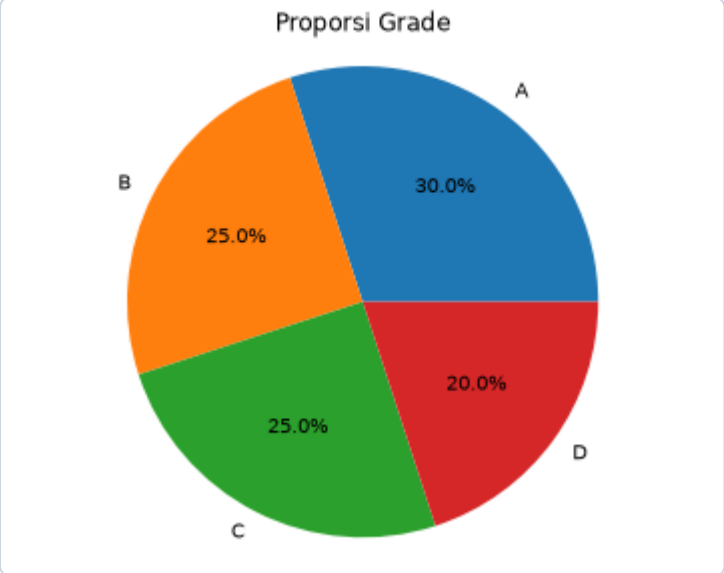
OUTPUT:

```
(tidak ada output)
```




Scatter
Nilai Y
Nilai X
80
70
60
50
40
50
60
70
80
90


Distribusi Nilai
12
10
8
6
4
2
0
50
60
70
80
90


Proporsi Grade
A
B
C
D
30.0%
25.0%
25.0%
20.0%

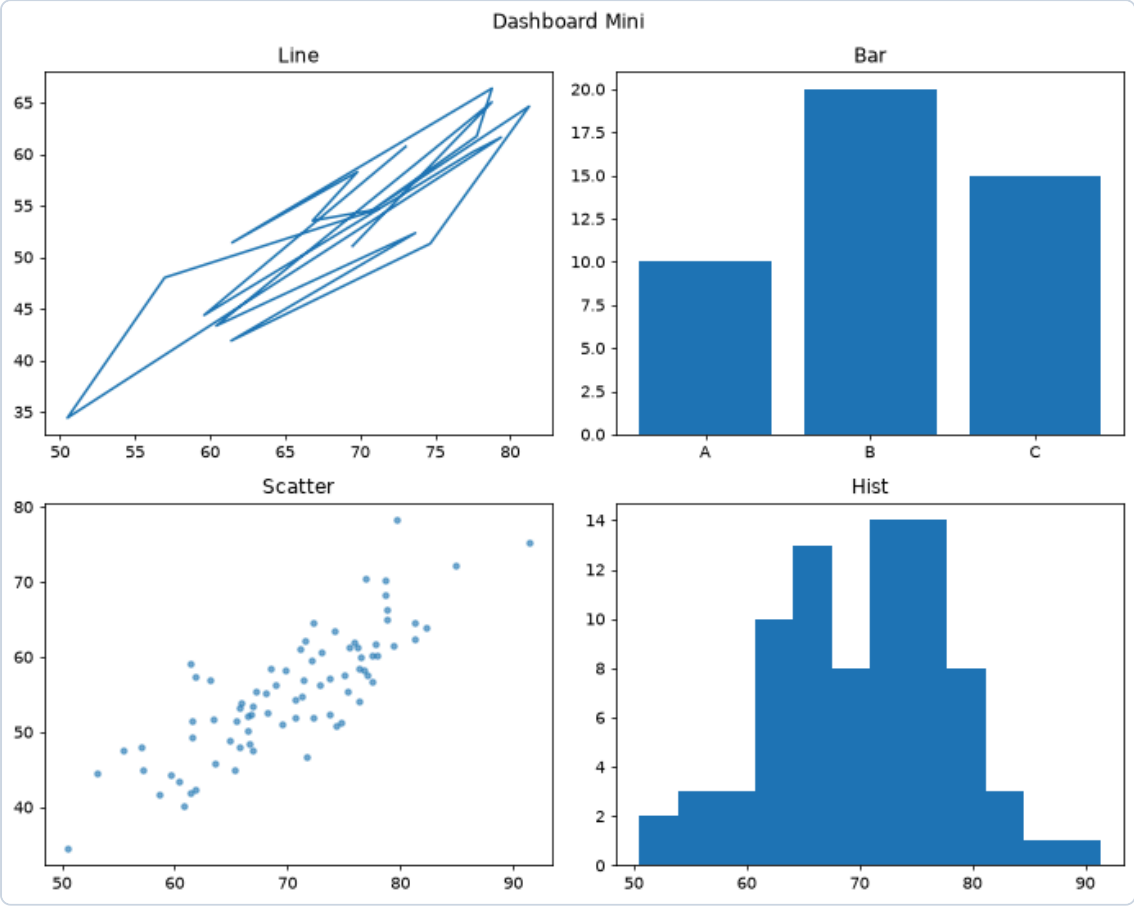
2.4 Subplot

Subplot menaruh beberapa plot dalam satu figure — berguna untuk dashboard mini laporan.

```
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
axes[0, 0].plot(x[:20], y[:20]); axes[0, 0].set_title("Line")
axes[0, 1].bar(["A", "B", "C"], [10, 20, 15]); axes[0, 1].set_title("Bar")
axes[1, 0].scatter(x, y, s=12, alpha=0.6); axes[1, 0].set_title("Scatter")
axes[1, 1].hist(x, bins=12); axes[1, 1].set_title("Hist")
fig.suptitle("Dashboard Mini")
plt.tight_layout()
plt.savefig("dashboard.png", dpi=150)
plt.show()
```

OUTPUT:

```
(tidak ada output)
```



TIPS

Tips Matplotlib

plt.tight_layout() mengurangi label terpotong.

figsize dalam inci; dpi mengatur ketajaman file PNG.

Di script .py, jangan lupa plt.show() atau savefig.

2.5 Latihan Matplotlib

1. Line plot y = x² untuk x dari -10 sampai 10; beri judul dan label.

```
Pseudocode:
  x ← linspace -10..10
  plot(x, x**2); judul; xlabel; ylabel; tampilkan/simpan
```

2. Bar chart 5 mata kuliah vs rata-rata nilai (data bebas); angka di atas bar.

```
Pseudocode:
  siapkan list matkul dan list nilai
  bar(...); loop untuk text di atas tiap bar
```

3. Histogram 200 angka random normal; bins=20.

```
Pseudocode:
  data ← random normal size 200
  hist(data, bins=20); judul
```

4. Subplot 1×2: scatter kiri, histogram kanan (data sama).

```
Pseudocode:
  fig, axes ← subplots(1, 2)
  axes[0].scatter(...); axes[1].hist(...)
  tight_layout; show/save
```

3. Seaborn — Visualisasi Statistik

3.1 Ide dasar

Seaborn dibangun di atas Matplotlib dan memudahkan plot statistik: distribusi, hubungan antar variabel, perbandingan kategori. Banyak dataset contoh bisa dimuat dengan sns.load_dataset — ini data nyata untuk latihan, bukan angka fiktif semata.

```python
import seaborn as sns
import pandas as pd
sns.set_theme(style="whitegrid")
tips = sns.load_dataset("tips")
penguins = sns.load_dataset("penguins")
print(tips.head())
```

OUTPUT:

```
   total_bill   tip     sex smoker  day    time  size
0       16.99  1.01  Female     No  Sun  Dinner     2
1       10.34  1.66    Male     No  Sun  Dinner     3
2       21.01  3.50    Male     No  Sun  Dinner     3
3       23.68  3.31    Male     No  Sun  Dinner     2
4       24.59  3.61  Female     No  Sun  Dinner     4
```

3.2 Distribusi dan relasi

```
sns.histplot(data=tips, x="total_bill", kde=True)
plt.title("Distribusi Total Bill"); plt.show()

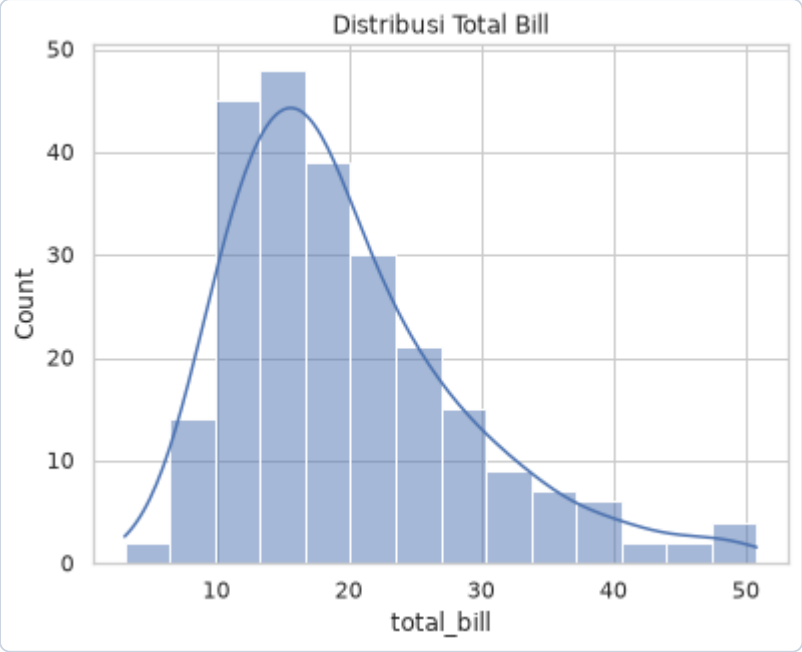
sns.boxplot(data=tips, x="day", y="total_bill")
plt.title("Total Bill per Hari"); plt.show()

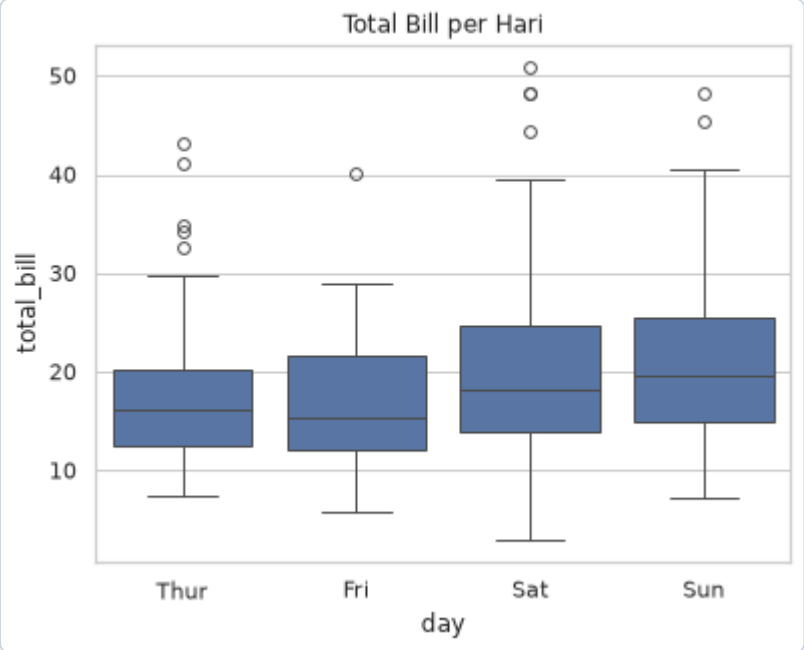
sns.scatterplot(data=tips, x="total_bill", y="tip", hue="time")
plt.show()

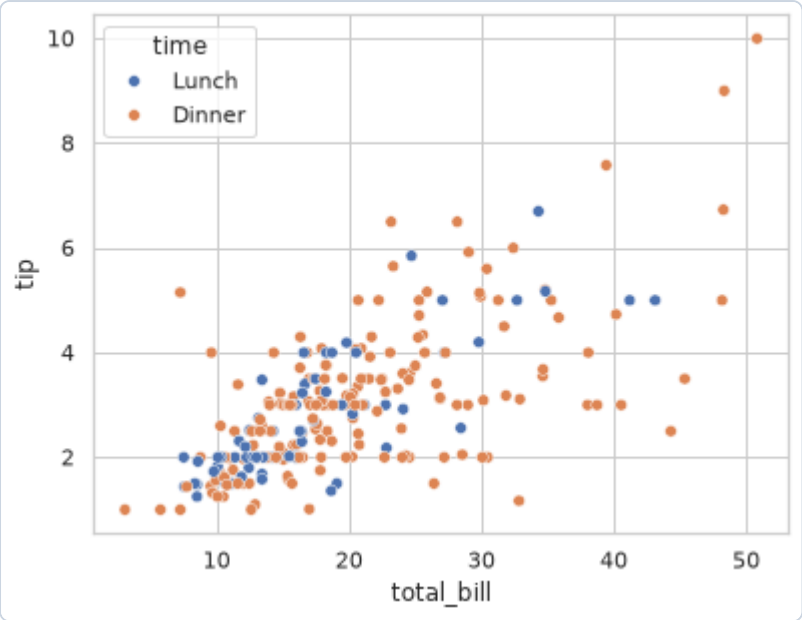
sns.regplot(data=tips, x="total_bill", y="tip")
plt.title("Tip vs Total Bill"); plt.show()
```

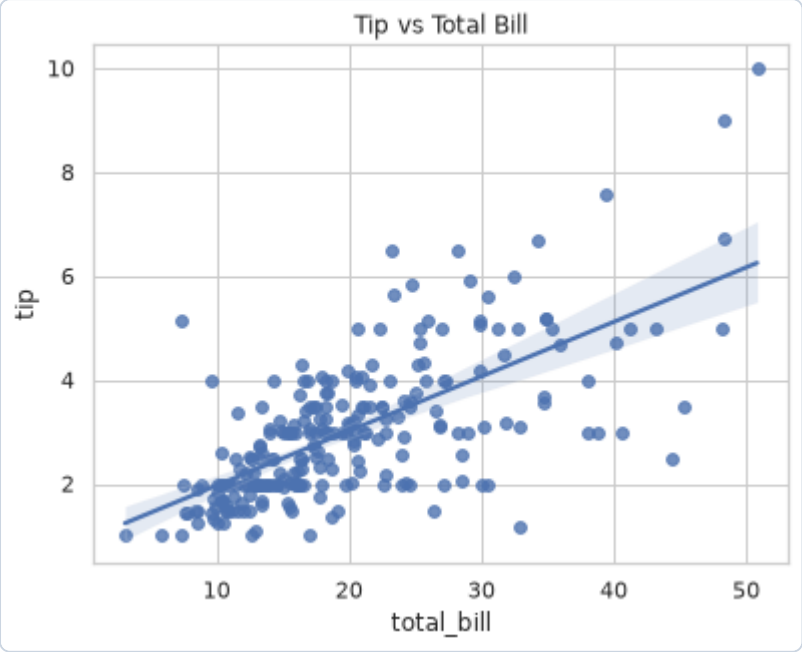
OUTPUT:

```
(tidak ada output)
```









3.3 Categorical & heatmap

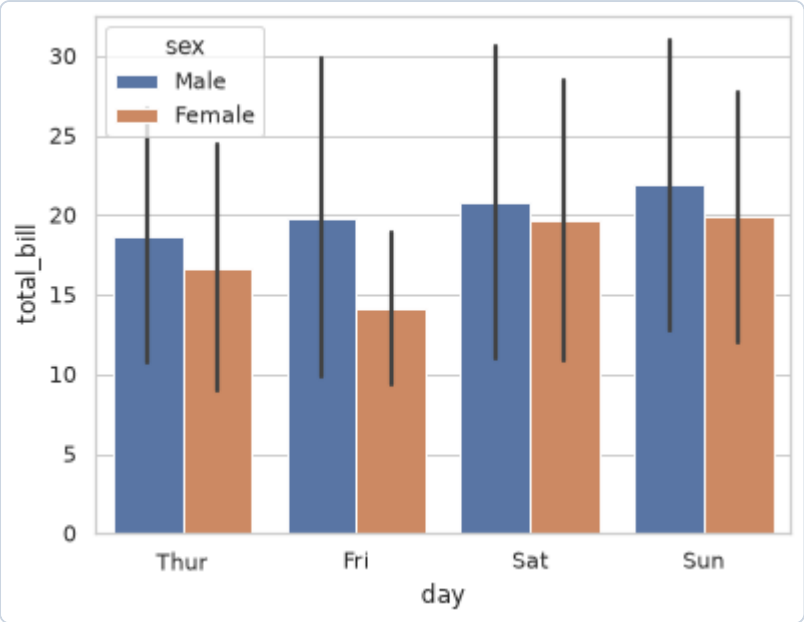
```
sns.barplot(data=tips, x="day", y="total_bill", hue="sex", errorbar="sd")
plt.show()

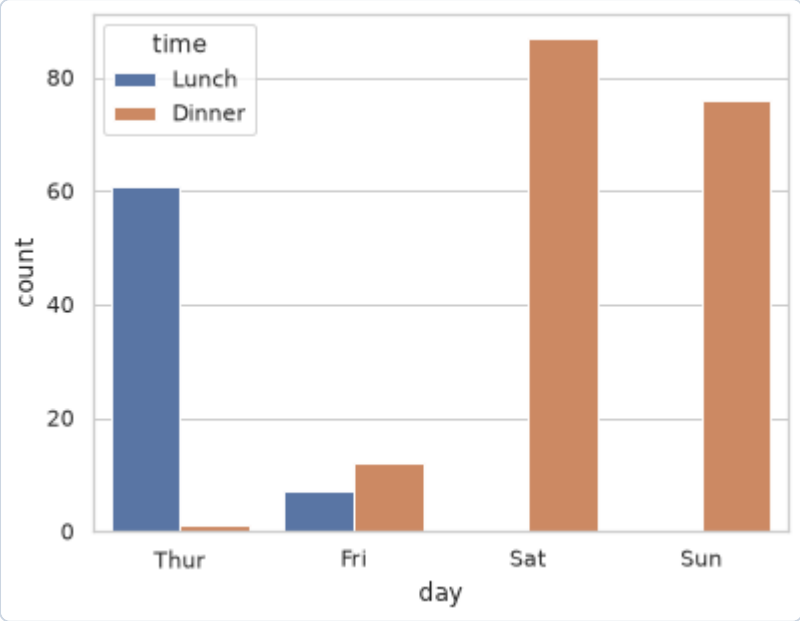
sns.countplot(data=tips, x="day", hue="time")
plt.show()

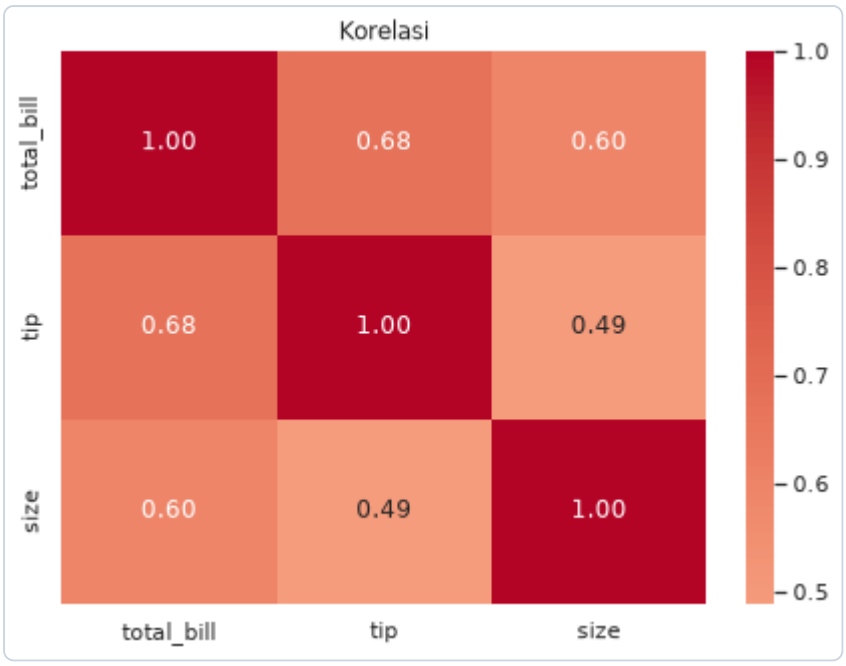
num = tips.select_dtypes(include="number")
sns.heatmap(num.corr(), annot=True, cmap="coolwarm", center=0, fmt=".2f")
plt.title("Korelasi"); plt.tight_layout(); plt.show()
```

OUTPUT:

```
(tidak ada output)
```


sex
Male
Female
total_bill
0
5
10
15
20
25
30
Thur
Fri
Sat
Sun
day


time
Lunch
Dinner
count
0
20
40
60
80
Thur
Fri
Sat
Sun
day



3.4 Latihan Seaborn

1. histplot + kde untuk body_mass_g pada dataset penguins.

```
Pseudocode:
  muat penguins; dropna bila perlu
  histplot(x="body_mass_g", kde=True); judul
```

2. boxplot body_mass_g berdasarkan species.

```
Pseudocode:
  boxplot(data=penguins, x="species", y="body_mass_g")
```

3. heatmap korelasi kolom numerik penguins (setelah dropna).

```
Pseudocode:
  num ← select_dtypes number; dropna
  heatmap(num.corr(), annot=True)
```

4. Bekerja dengan API (requests)

4.1 Apa itu API, singkatnya

API (Application Programming Interface) di konteks web sering berarti endpoint HTTP yang mengembalikan data (biasanya JSON) saat Anda mengirim permintaan. library requests memudahkan panggilan itu dari Python.

Alur tipikal: susun URL (+ parameter) → requests.get → cek status → .json() → ubah ke DataFrame bila berbentuk tabel → analisis/visualisasi.

4.2 GET dan JSON

```
import requests
import pandas as pd

url = "https://jsonplaceholder.typicode.com/users"
resp = requests.get(url, timeout=10)
print(resp.status_code)
data = resp.json()
df = pd.DataFrame(data)
print(df[["id", "name", "email"]].head())
```

OUTPUT:

```
200
   id              name                      email
0   1     Leanne Graham          Sincere@april.biz
1   2      Ervin Howell          Shanna@melissa.tv
2   3  Clementine Bauch         Nathan@yesenia.net
3   4  Patricia Lebsack  Julianne.OConner@kory.org
4   5  Chelsey Dietrich   Lucio_Hettinger@annie.ca
```

Catatan: host jsonplaceholder.typicode.com tidak dapat diakses dari lingkungan pengujian (kebijakan jaringan), sehingga kode dijalankan dengan respons tiruan yang bentuknya sama dengan API asli. ***Nilai/teks pada output ini contoh, bukan data nyata.***

4.3 Parameter & cuaca (Open-Meteo)

Banyak API memakai query parameter. Open-Meteo menyediakan data cuaca tanpa API key — cocok untuk latihan.

```
url = "https://api.open-meteo.com/v1/forecast"
params = {"latitude": -6.2, "longitude": 106.8, "current_weather": True}
r = requests.get(url, params=params, timeout=10)
r.raise_for_status()
print(r.json().get("current_weather"))
```

OUTPUT:

```
{'temperature': 31.2, 'windspeed': 9.4}
```

Catatan: host api.open-meteo.com tidak dapat diakses dari lingkungan pengujian (kebijakan jaringan), sehingga kode dijalankan dengan respons tiruan yang bentuknya sama dengan API asli. ***Nilai/teks pada output ini contoh, bukan data nyata.***

4.4 Error handling

Jaringan bisa timeout; server bisa mengembalikan 404/500. Bungkus panggilan dengan try/except agar program tidak berhenti kasar.

```python
def ambil_json(url, params=None):
    try:
        r = requests.get(url, params=params, timeout=15)
        r.raise_for_status()
        return r.json()
    except requests.exceptions.Timeout:
        print("Timeout")
    except requests.exceptions.HTTPError as e:
        print("HTTP error:", e)
    except requests.exceptions.RequestException as e:
        print("Gagal:", e)
    return None

data = ambil_json("https://jsonplaceholder.typicode.com/posts")
if data:
    df = pd.DataFrame(data)
    df.to_csv("posts.csv", index=False)
```

OUTPUT:

```
(tidak ada output)
```

Catatan: host jsonplaceholder.typicode.com tidak dapat diakses dari lingkungan pengujian (kebijakan jaringan), sehingga kode dijalankan dengan respons tiruan yang bentuknya sama dengan API asli. ***Nilai/teks pada output ini contoh, bukan data nyata.***

4.5 Latihan API

1. Ambil /comments dari JSONPlaceholder; DataFrame; jumlah komentar per postId.

```
Pseudocode:
  GET .../comments → json → DataFrame
  groupby postId → size/count
  cetak head hasil
```

2. Panggil Open-Meteo untuk kota Anda; cetak suhu dan angin saat ini.

```
Pseudocode:
  siapkan latitude/longitude kota
  GET open-meteo dengan current_weather
  cetak field suhu dan windspeed
```

5. Scraping & arXiv API

5.1 Etika

Scraping berarti mengekstrak data dari HTML halaman web. Lakukan dengan tanggung jawab: hormati robots.txt, jangan membebani server (beri jeda), utamakan API resmi bila ada, dan jangan mengambil data pribadi tanpa izin. Untuk tugas kuliah, utamakan API publik atau HTML contoh lokal.

5.2 BeautifulSoup (pola)

```
from bs4 import BeautifulSoup

html = """
<html><body>
  <h1>Daftar Buku</h1>
  <ul>
    <li class="buku">Python Dasar</li>
    <li class="buku">Data Science</li>
  </ul>
</body></html>
"""
soup = BeautifulSoup(html, "lxml")
print(soup.h1.text)
for item in soup.select("li.buku"):
    print(item.get_text())
```

OUTPUT:

```
Daftar Buku
Python Dasar
Data Science
```

5.3 arXiv API

arXiv menyediakan API terbuka untuk metadata paper ilmiah — sangat relevan untuk mahasiswa S2/S3. Respons berformat Atom/XML; feedparser memudahkan parsing.

```
import requests
import feedparser
import pandas as pd

url = "http://export.arxiv.org/api/query"
params = {
    "search_query": "all:machine learning",
    "start": 0,
    "max_results": 5,
    "sortBy": "submittedDate",
    "sortOrder": "descending",
}
r = requests.get(url, params=params, timeout=20)
feed = feedparser.parse(r.text)

rows = []
for e in feed.entries:
    rows.append({
        "title": e.title.replace("\n", " ").strip(),
        "authors": ", ".join(a.name for a in e.authors[:3]),
        "published": e.published[:10],
        "link": e.link,
    })
df = pd.DataFrame(rows)
print(df[["title", "published"]])
df.to_csv("arxiv_ml.csv", index=False)
```

OUTPUT:

```
                                              title   published
0  Attention Mechanisms for Medical Image Segment...  2025-12-15
1  Graph Neural Networks for Traffic Forecasting ...  2025-10-15
2  Self-Supervised Learning with Limited Labels  ...  2025-08-15
3  Federated Learning in Educational Data Mining ...  2025-06-15
4  Transformer Models for Time Series Prediction ...  2024-12-15
```

Catatan: host export.arxiv.org tidak dapat diakses dari lingkungan pengujian (kebijakan jaringan), sehingga kode dijalankan dengan respons tiruan yang bentuknya sama dengan API asli. ***Nilai/teks pada output ini contoh, bukan data nyata.***

TIPS

Tips arXiv

Jangan spam request; beri jeda jika mengambil banyak halaman.

Field pencarian: ti: (judul), au: (penulis), abs: (abstrak), all:

Contoh: ti:"neural network" AND au:hinton

5.4 Latihan

1. Parse HTML contoh di atas; kumpulkan semua teks li ke list Python.

```
Pseudocode:
  soup ← BeautifulSoup(html)
  hasil ← [teks tiap li.buku]
  cetak hasil
```

2. Cari 5 paper arXiv dengan kata kunci bidang Anda; simpan title + published ke CSV.

```
Pseudocode:
  susun search_query sesuai bidang
  GET arXiv API max_results=5
  parse feed; bangun DataFrame; to_csv
```

6. Mini Project

Kerjakan minimal 2 project. Project 3 dan 4 sengaja lebih menantang — cocok jika API dan Pandas sudah nyaman.

Project 1 — Dashboard Visualisasi Nilai (Dasar)

Dari DataFrame nilai mahasiswa, buat minimal 3 plot: histogram nilai, bar rata-rata per prodi, dan pie proporsi huruf mutu. Simpan semua figure ke folder figures/.

Fitur wajib

1. Data minimal 8 mahasiswa, 2 prodi atau lebih.
2. Kolom huruf mutu (A–E) dari nilai.
3. 3 jenis plot + judul/label; simpan PNG.

```python
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt

FIG = Path("figures")
FIG.mkdir(exist_ok=True)

df = pd.DataFrame({
    "nama": list("ABCDEFGH"),
    "prodi": ["IF", "SI", "IF", "SI", "IF", "SI", "IF", "SI"],
    "nilai": [80, 70, 90, 65, 85, 75, 95, 60],
})

def ke_huruf(n):
    if n >= 85: return "A"
    if n >= 70: return "B"
    if n >= 60: return "C"
    if n >= 50: return "D"
    return "E"

df["huruf"] = df["nilai"].apply(ke_huruf)

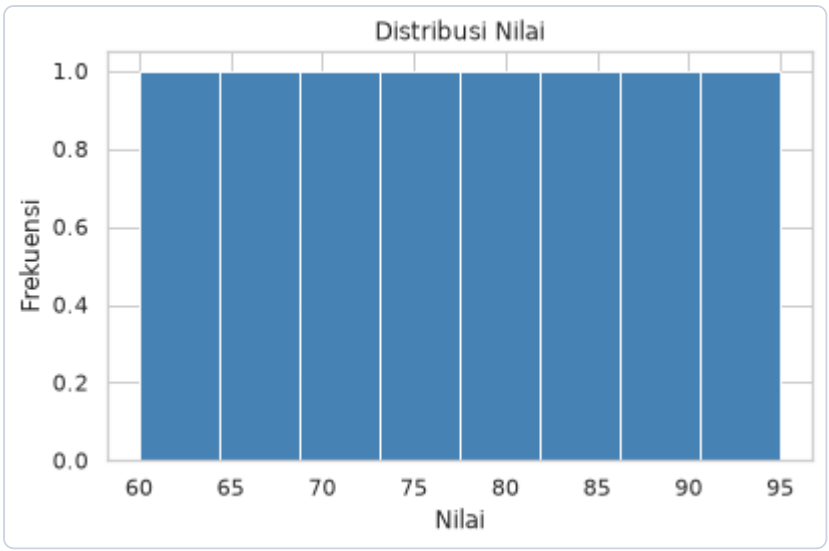
# 1) Histogram nilai
plt.figure(figsize=(6, 4))
plt.hist(df["nilai"], bins=8, edgecolor="white", color="steelblue")
plt.title("Distribusi Nilai"); plt.xlabel("Nilai"); plt.ylabel("Frekuensi")
plt.tight_layout(); plt.savefig(FIG / "hist_nilai.png", dpi=150); plt.show()

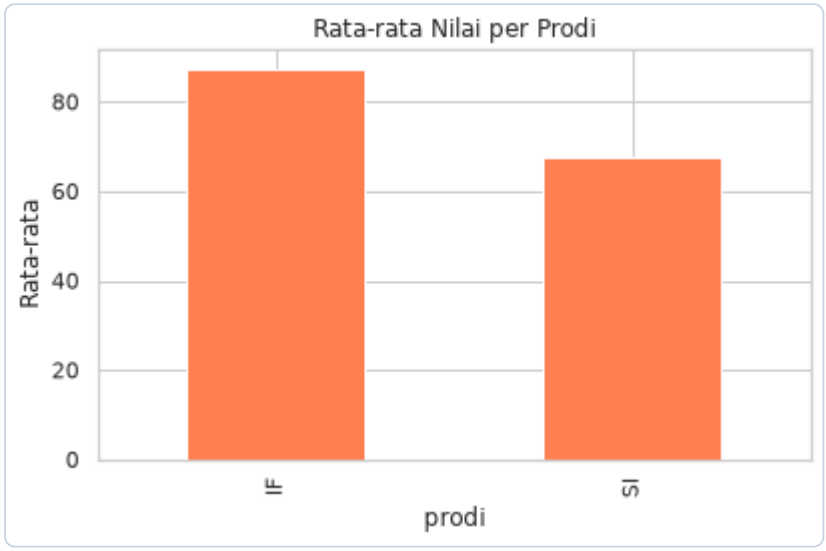
# 2) Bar rata-rata per prodi
rata = df.groupby("prodi")["nilai"].mean()
plt.figure(figsize=(6, 4))
rata.plot(kind="bar", color="coral")
plt.title("Rata-rata Nilai per Prodi"); plt.ylabel("Rata-rata")
plt.tight_layout(); plt.savefig(FIG / "bar_prodi.png", dpi=150); plt.show()

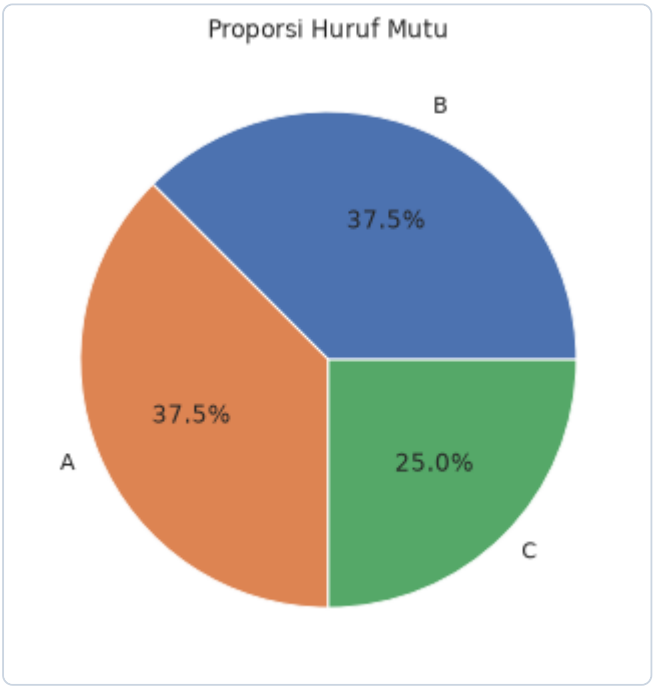
# 3) Pie huruf mutu
plt.figure(figsize=(5, 5))
df["huruf"].value_counts().plot(kind="pie", autopct="%1.1f%%")
plt.title("Proporsi Huruf Mutu"); plt.ylabel("")
plt.tight_layout(); plt.savefig(FIG / "pie_huruf.png", dpi=150); plt.show()
```

OUTPUT:

```
(tidak ada output)
```


Distribusi Nilai
Frekuensi
1.0
0.8
0.6
0.4
0.2
0.0
60
65
70
75
80
85
90
95
Nilai


Rata-rata Nilai per Prodi
Rata-rata
80
60
40
20
0
IF
SI
prodi


Proporsi Huruf Mutu
B
37.5%
37.5%
A
25.0%
C

Project 2 — API + Visualisasi (Menengah)

Ambil data posts (dan opsional users) dari JSONPlaceholder. Gabungkan bila perlu, lalu buat minimal 2 visualisasi yang bermakna.

Fitur wajib

1. GET dengan timeout + cek status / error handling.
2. DataFrame + simpan CSV mentah.
3. Minimal 2 plot (mis. bar post per user; panjang title rata-rata).

```
from pathlib import Path
import requests
import pandas as pd
import matplotlib.pyplot as plt

OUT = Path("output"); OUT.mkdir(exist_ok=True)
FIG = OUT / "figures"; FIG.mkdir(exist_ok=True)

def ambil(url):
    r = requests.get(url, timeout=15)
    r.raise_for_status()
    return r.json()

posts = ambil("https://jsonplaceholder.typicode.com/posts")
users = ambil("https://jsonplaceholder.typicode.com/users")
df_posts = pd.DataFrame(posts)
df_users = pd.DataFrame(users)
df_posts.to_csv(OUT / "posts.csv", index=False)

# Agregasi
per_user = df_posts.groupby("userId").size().reset_index(name="jumlah_post")
df_posts["title_len"] = df_posts["title"].str.len()
len_per_user = df_posts.groupby("userId")["title_len"].mean()

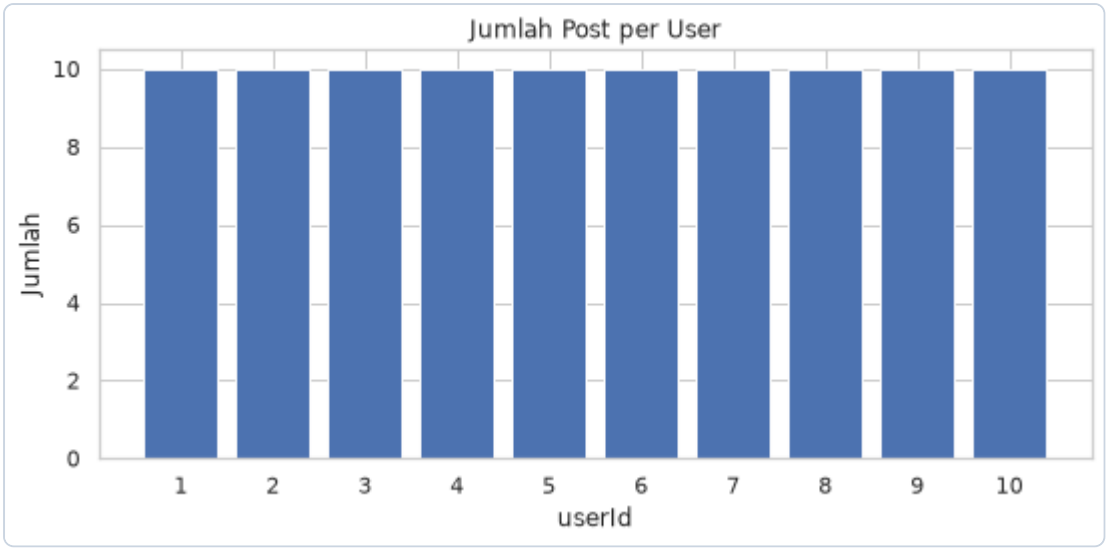
# Plot 1
plt.figure(figsize=(8, 4))
plt.bar(per_user["userId"].astype(str), per_user["jumlah_post"])
plt.title("Jumlah Post per User"); plt.xlabel("userId"); plt.ylabel("Jumlah")
plt.tight_layout(); plt.savefig(FIG / "posts_per_user.png", dpi=150); plt.show()

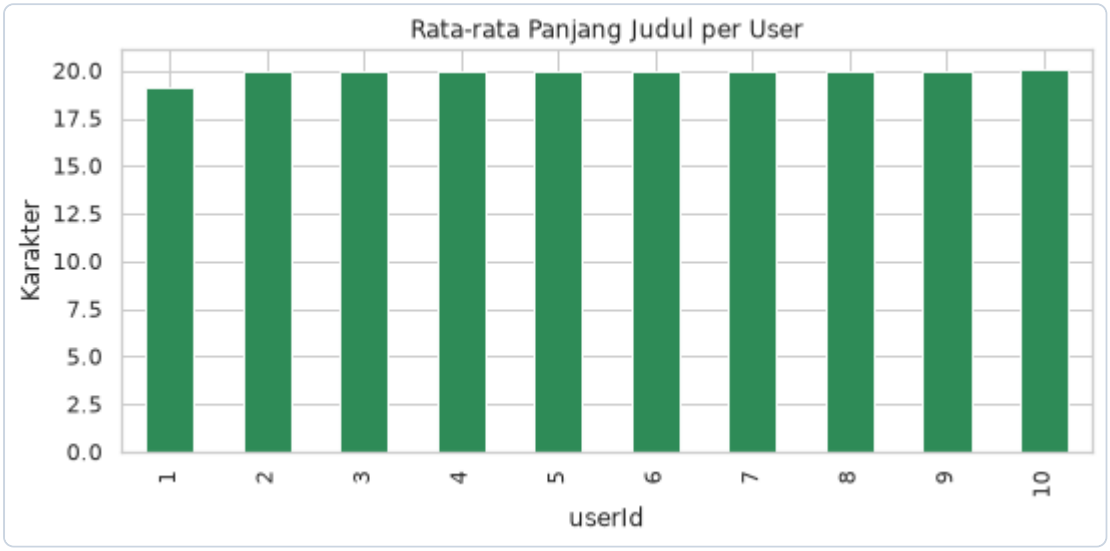
# Plot 2
plt.figure(figsize=(8, 4))
len_per_user.plot(kind="bar", color="seagreen")
plt.title("Rata-rata Panjang Judul per User"); plt.ylabel("Karakter")
plt.tight_layout(); plt.savefig(FIG / "title_len.png", dpi=150); plt.show()

# Opsional: merge nama user
# df = df_posts.merge(df_users[["id", "name"]], left_on="userId", right_on="id")
```

OUTPUT:

```
(tidak ada output)
```





Catatan: host jsonplaceholder.typicode.com tidak dapat diakses dari lingkungan pengujian (kebijakan jaringan), sehingga kode dijalankan dengan respons tiruan yang bentuknya sama dengan API asli. ***Nilai/teks pada output ini contoh, bukan data nyata.***

Project 3 — Pipeline Paper arXiv (Menengah–Tinggi)

Bangun pipeline pencarian paper yang lebih lengkap: beberapa kata kunci atau satu query majemuk, ambil lebih banyak hasil (paginasi sederhana), bersihkan judul, ekstrak tahun, analisis frekuensi, dan visualisasi. Ini melampaui “ambil 5 paper lalu print”.

Fitur wajib

1. Function fetch_arxiv(query, start, max_results) dengan error handling.
2. Paginasi: minimal 2 request (mis. start=0 dan start=10) digabung jadi satu DataFrame.
3. Kolom: title, authors, published, year, link, summary_short.
4. Analisis: jumlah paper per tahun; opsional kata paling sering di judul.
5. Minimal 2 visualisasi + simpan CSV + jeda antar request (time.sleep).

```python
import time
import re
from pathlib import Path
from collections import Counter
import requests
import feedparser
import pandas as pd
import matplotlib.pyplot as plt

OUT = Path("output_arxiv"); OUT.mkdir(exist_ok=True)
FIG = OUT / "figures"; FIG.mkdir(exist_ok=True)
API = "http://export.arxiv.org/api/query"

def fetch_arxiv(query, start=0, max_results=10):
    params = {
        "search_query": query,
        "start": start,
        "max_results": max_results,
        "sortBy": "submittedDate",
        "sortOrder": "descending",
    }
    try:
        r = requests.get(API, params=params, timeout=30)
        r.raise_for_status()
    except requests.RequestException as e:
        print("Gagal fetch:", e)
        return []
    feed = feedparser.parse(r.text)
    rows = []
    for e in feed.entries:
        published = e.get("published", "")[:10]
        rows.append({
            "title": e.title.replace("\n", " ").strip(),
            "authors": ", ".join(a.name for a in e.authors[:5]) if e.get("authors") else "",
            "published": published,
            "year": published[:4] if published else "",
            "link": e.get("link", ""),
            "summary_short": e.get("summary", "").replace("\n", " ")[:240],
        })
    return rows

def ambil_banyak(query, batch=10, halaman=2):
    semua = []
    for i in range(halaman):
        print(f"Mengambil halaman {i+1}...")
        semua.extend(fetch_arxiv(query, start=i * batch, max_results=batch))
        time.sleep(3)  # hormati server
    return pd.DataFrame(semua)

def kata_sering_di_judul(df, n=15):
    stop = {"the", "a", "an", "of", "and", "for", "in", "on", "to", "with", "from"}
    tokens = []
    for t in df["title"].dropna():
        for w in re.findall(r"[a-zA-Z]{3,}", t.lower()):
            if w not in stop:
                tokens.append(w)
    return Counter(tokens).most_common(n)

def main():
    # Ganti query sesuai bidang Anda
    query = 'all:"machine learning" OR all:"deep learning"'
```

```
    df = ambil_banyak(query, batch=10, halaman=2)
    if df.empty:
        print("Data kosong"); return
    df.to_csv(OUT / "arxiv_papers.csv", index=False)
    print(df.head())

    # Analisis per tahun
    per_tahun = df["year"].value_counts().sort_index()
    plt.figure(figsize=(8, 4))
    per_tahun.plot(kind="bar")
    plt.title("Jumlah Paper per Tahun"); plt.xlabel("Tahun"); plt.ylabel("Jumlah")
    plt.tight_layout(); plt.savefig(FIG / "per_tahun.png", dpi=150); plt.show()

    # Kata sering di judul
    top = kata_sering_di_judul(df)
    words, counts = zip(*top) if top else ([], [])
    plt.figure(figsize=(8, 4))
    plt.barh(list(words)[::-1], list(counts)[::-1])
    plt.title("Kata Sering Muncul di Judul")
    plt.tight_layout(); plt.savefig(FIG / "kata_judul.png", dpi=150); plt.show()

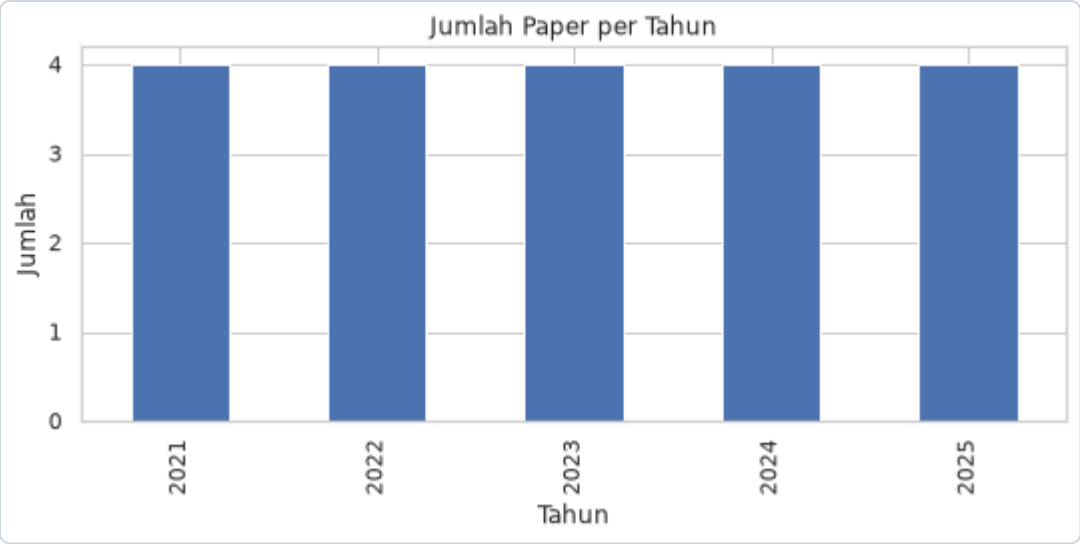
    print("Selesai. File di", OUT)

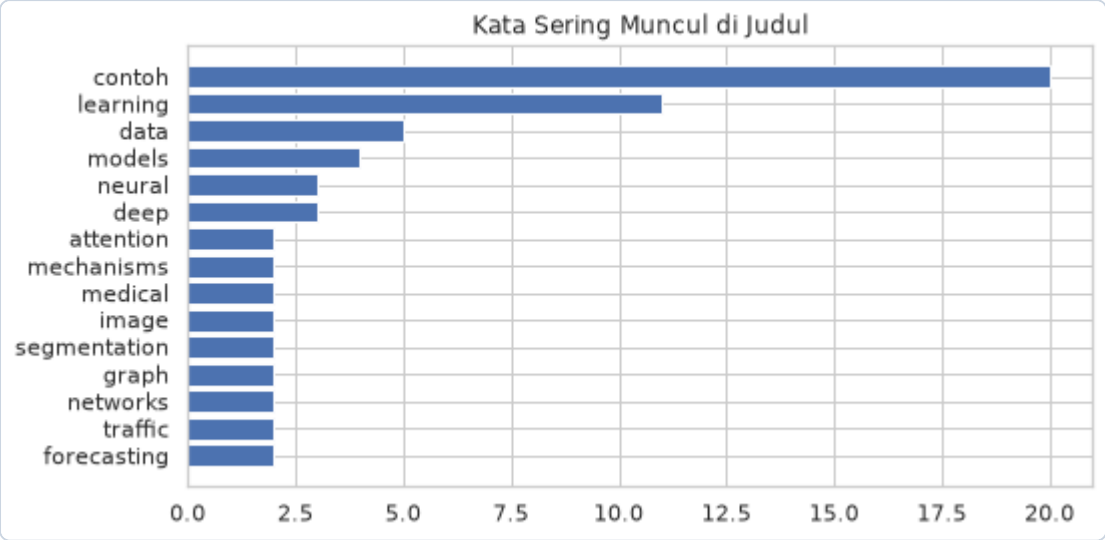
if __name__ == "__main__":
    main()
```

OUTPUT:

```
Mengambil halaman 1...
Mengambil halaman 2...
                                          title                 authors   published  year  \
0  Attention Mechanisms for Medical Image Segment...  Penulis 1A, Penulis 1B  2025-12-15  2025
1  Graph Neural Networks for Traffic Forecasting ...  Penulis 2A, Penulis 2B  2025-10-15  2025
2  Self-Supervised Learning with Limited Labels  ...  Penulis 3A, Penulis 3B  2025-08-15  2025
3  Federated Learning in Educational Data Mining ...  Penulis 4A, Penulis 4B  2025-06-15  2025
4  Transformer Models for Time Series Prediction ...  Penulis 5A, Penulis 5B  2024-12-15  2024

                              link                   summary_short
0  http://arxiv.org/abs/2512.1000v1  Ringkasan contoh paper nomor 1.
1  http://arxiv.org/abs/2510.1001v1  Ringkasan contoh paper nomor 2.
2  http://arxiv.org/abs/2508.1002v1  Ringkasan contoh paper nomor 3.
3  http://arxiv.org/abs/2506.1003v1  Ringkasan contoh paper nomor 4.
4  http://arxiv.org/abs/2412.1004v1  Ringkasan contoh paper nomor 5.
Selesai. File di output_arxiv
```





Catatan: `time.sleep(3)` *antar halaman dihilangkan saat pengujian agar cepat; pada pemakaian nyata jeda tetap diperlukan untuk menghormati server arXiv.*

Catatan: host export.arxiv.org tidak dapat diakses dari lingkungan pengujian (kebijakan jaringan), sehingga kode dijalankan dengan respons tiruan yang bentuknya sama dengan API asli. ***Nilai/teks pada output ini contoh, bukan data nyata.***

TIPS

Project 3 — kompleksitas ekstra (opsional)

Bandingkan dua query berbeda (mis. "climate change" vs "renewable energy") lalu gabungkan dengan kolom source_query.

Filter paper hanya 3 tahun terakhir sebelum plot.

Tulis 3 insight singkat di file insight_arxiv.txt.

Project 4 — EDA Lengkap Dataset Nyata (Tinggi)

Pilih dataset publik (disarankan sns.load_dataset: titanic, penguins, atau tips). Lakukan EDA terstruktur: inspeksi, cleaning, feature sederhana, beberapa agregasi, minimal 5 visualisasi berbeda, dan laporan insight tertulis. Bukan sekadar memanggil plot acak.

Fitur wajib

1. Function terpisah: load_and_inspect, clean, analyze, visualize, tulis_insight.
2. Cleaning: missing, tipe, (opsional) outlier kasar atau kategori jarang.
3. Minimal 3 agregasi groupby yang menjawab pertanyaan berbeda.
4. Minimal 5 plot berbeda (hist/box/bar/scatter/heatmap/count — pilih yang relevan).
5. File insight.md atau insight.txt berisi 5 temuan + batasan analisis.

```python
from pathlib import Path
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

OUT = Path("output_eda"); OUT.mkdir(exist_ok=True)
FIG = OUT / "figures"; FIG.mkdir(exist_ok=True)

def load_and_inspect(name="titanic"):
    df = sns.load_dataset(name)
    print("Shape:", df.shape)
    print(df.head())
    print(df.info())
    print("Missing:\n", df.isna().sum())
    print(df.describe(include="all"))
    return df

def clean(df):
    df = df.copy()
    # Contoh strategi — sesuaikan dataset:
    # - drop kolom terlalu kosong
    # - isi median untuk numerik penting
    # - dropna pada kolom kunci analisis
    if "age" in df.columns:
        df["age"] = df["age"].fillna(df["age"].median())
    if "embarked" in df.columns:
        df["embarked"] = df["embarked"].fillna(df["embarked"].mode().iloc[0])
    # df = df.dropna(subset=["..."])
    print("Shape setelah clean:", df.shape)
    return df

def analyze(df):
    ringkas = {}
    # 1) agregasi contoh — Titanic: survival by class
    if {"survived", "class"}.issubset(df.columns):
        ringkas["survive_by_class"] = df.groupby("class")["survived"].mean()
        print(ringkas["survive_by_class"])
    if {"survived", "sex"}.issubset(df.columns):
        ringkas["survive_by_sex"] = df.groupby("sex")["survived"].mean()
        print(ringkas["survive_by_sex"])
    if "age" in df.columns and "survived" in df.columns:
        ringkas["age_by_survived"] = df.groupby("survived")["age"].mean()
        print(ringkas["age_by_survived"])
    # Untuk penguins: groupby species / island, dll.
    return ringkas

def visualize(df):
    # Plot 1 — distribusi numerik utama
    num_cols = df.select_dtypes(include="number").columns.tolist()
    if num_cols:
        col = "age" if "age" in df.columns else num_cols[0]
        plt.figure(figsize=(6, 4))
        sns.histplot(data=df, x=col, kde=True)
        plt.title(f"Distribusi {col}")
        plt.tight_layout(); plt.savefig(FIG / "01_hist.png", dpi=150); plt.show()

    # Plot 2 — boxplot by kategori
    if "class" in df.columns and "age" in df.columns:
        plt.figure(figsize=(6, 4))
        sns.boxplot(data=df, x="class", y="age")
        plt.title("Age by Class")
```

```
        plt.tight_layout(); plt.savefig(FIG / "02_box.png", dpi=150); plt.show()

    # Plot 3 — bar agregasi
    if "survived" in df.columns and "sex" in df.columns:
        plt.figure(figsize=(6, 4))
        sns.barplot(data=df, x="sex", y="survived", errorbar=None)
        plt.title("Survival Rate by Sex")
        plt.tight_layout(); plt.savefig(FIG / "03_bar.png", dpi=150); plt.show()

    # Plot 4 — countplot
    cat = "class" if "class" in df.columns else df.select_dtypes(exclude="number").columns[0]
    plt.figure(figsize=(6, 4))
    sns.countplot(data=df, x=cat)
    plt.title(f"Count of {cat}")
    plt.tight_layout(); plt.savefig(FIG / "04_count.png", dpi=150); plt.show()

    # Plot 5 — heatmap korelasi numerik
    num = df.select_dtypes(include="number")
    if num.shape[1] >= 2:
        plt.figure(figsize=(6, 5))
        sns.heatmap(num.corr(), annot=True, fmt=".2f", cmap="coolwarm", center=0)
        plt.title("Korelasi Numerik")
        plt.tight_layout(); plt.savefig(FIG / "05_heatmap.png", dpi=150); plt.show()

def tulis_insight(path, baris):
    path.write_text("\n".join(baris), encoding="utf-8")

def main():
    df0 = load_and_inspect("titanic")  # ganti "penguins" / "tips" bila ingin
    df = clean(df0)
    ringkas = analyze(df)
    visualize(df)
    df.to_csv(OUT / "data_bersih.csv", index=False)
    tulis_insight(OUT / "insight.txt", [
        "1. ... (temuan dari groupby/plot)",
        "2. ...",
        "3. ...",
        "4. ...",
        "5. ...",
        "",
        "Batasan: ... (missing, bias sampel, dll.)",
    ])
    print("Selesai →", OUT)

if __name__ == "__main__":
    main()
```

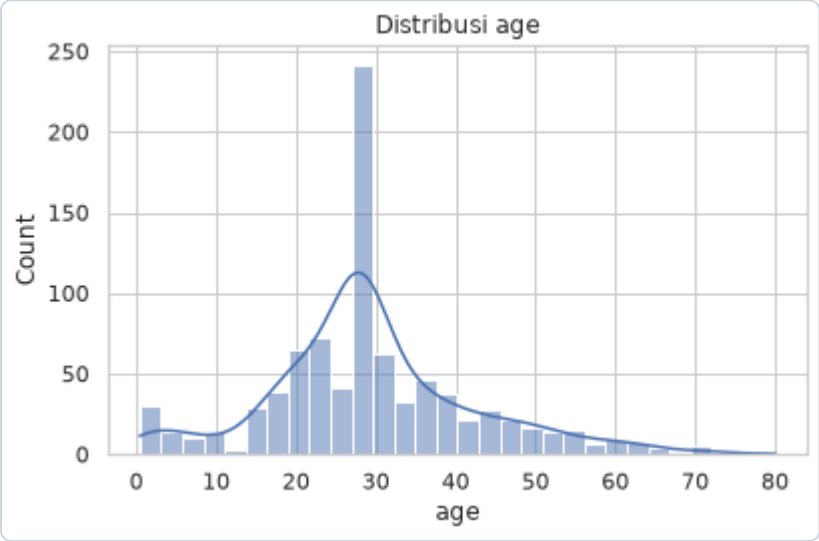
OUTPUT:

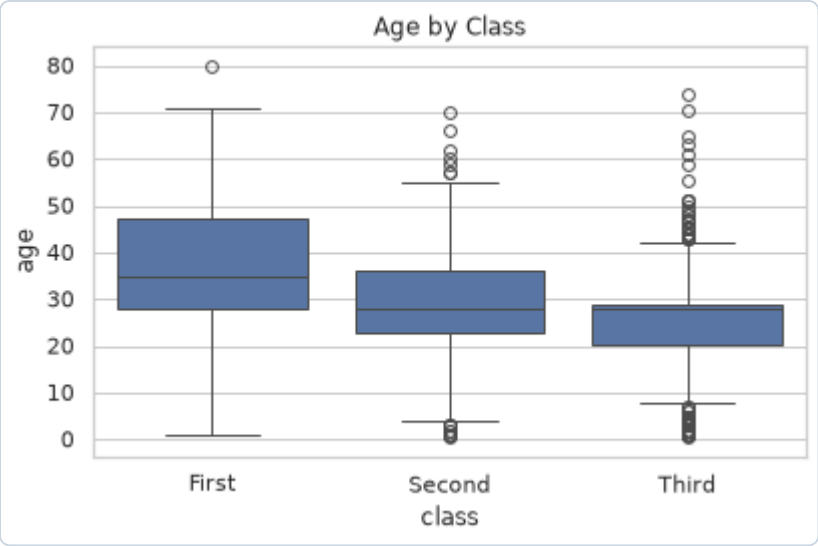
```
Shape: (891, 15)
   survived  pclass     sex   age  sibsp  parch     fare embarked  class    who  adult_male deck
\
0         0       3    male  22.0      1      0   7.2500        S  Third    man        True  NaN
1         1       1  female  38.0      1      0  71.2833        C  First  woman       False    C
2         1       3  female  26.0      0      0   7.9250        S  Third  woman       False  NaN
3         1       1  female  35.0      1      0  53.1000        S  First  woman       False    C
4         0       3    male  35.0      0      0   8.0500        S  Third    man        True  NaN

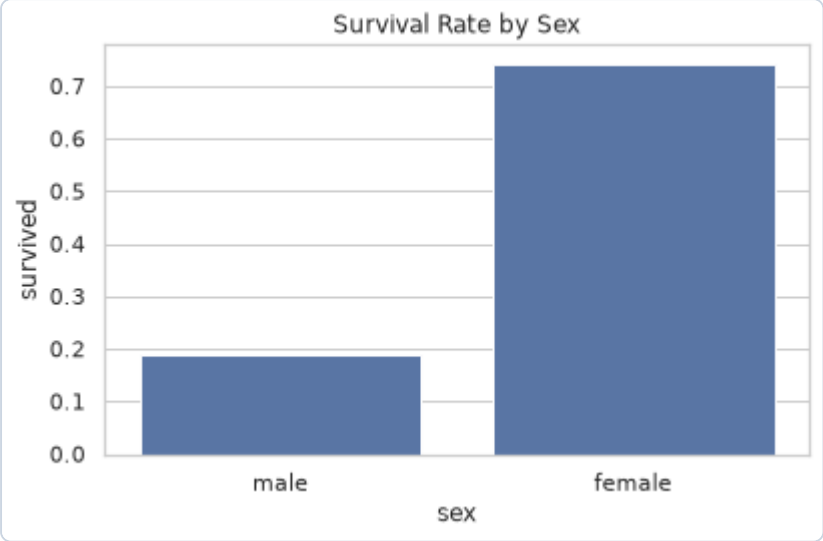
   embark_town alive  alone
0  Southampton    no  False
1    Cherbourg   yes  False
2  Southampton   yes   True
3  Southampton   yes  False
4  Southampton    no   True
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   survived     891 non-null    int64
 1   pclass       891 non-null    int64
 2   sex          891 non-null    str
 3   age          714 non-null    float64
 4   sibsp        891 non-null    int64
 5   parch        891 non-null    int64
 6   fare         891 non-null    float64
 7   embarked     889 non-null    str
 8   class        891 non-null    category
 9   who          891 non-null    str
 10  adult_male   891 non-null    bool
 11  deck         203 non-null    category
 12  embark_town  889 non-null    str
 13  alive        891 non-null    str
 14  alone        891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), str(5)
memory usage: 80.7 KB
None
Missing:
 survived         0
pclass           0
sex              0
age            177
sibsp            0
parch            0
fare             0
embarked         2
class            0
who              0
adult_male       0
deck           688
embark_town      2
alive            0
alone            0
dtype: int64
          survived      pclass   sex         age       sibsp       parch        fare embarked  \
count   891.000000  891.000000   891  714.000000  891.000000  891.000000  891.000000      889
unique         NaN         NaN     2         NaN         NaN         NaN         NaN        3
top            NaN         NaN  male         NaN         NaN         NaN         NaN        S
freq           NaN         NaN   577         NaN         NaN         NaN         NaN      644
```

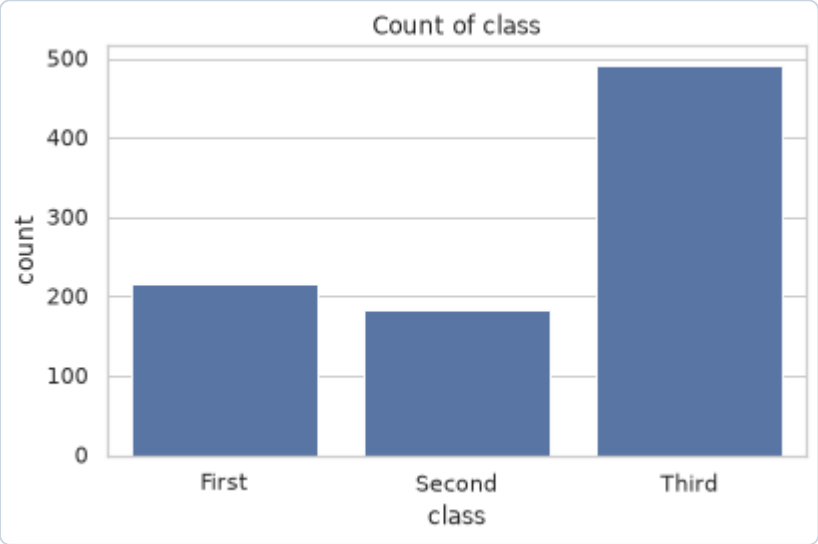
```
mean      0.383838    2.308642   NaN   29.699118    0.523008    0.381594   32.204208      NaN
std       0.486592    0.836071   NaN   14.526497    1.102743    0.806057   49.693429      NaN
min       0.000000    1.000000   NaN    0.420000    0.000000    0.000000    0.000000      NaN
25%       0.000000    2.000000   NaN   20.125000    0.000000    0.000000    7.910400      NaN
50%       0.000000    3.000000   NaN   28.000000    0.000000    0.000000   14.454200      NaN
75%       1.000000    3.000000   NaN   38.000000    1.000000    0.000000   31.000000      NaN
max       1.000000    3.000000   NaN   80.000000    8.000000    6.000000  512.329200      NaN

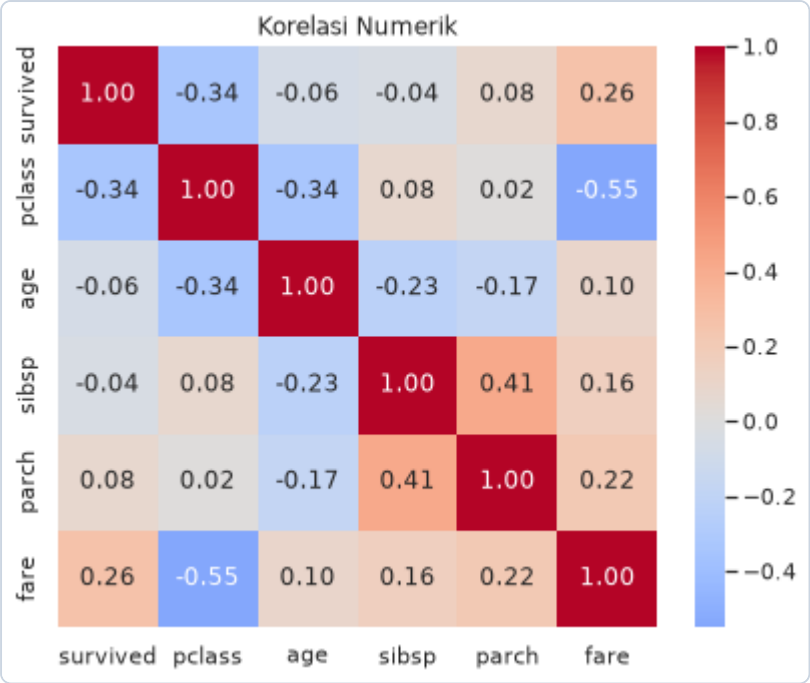
        class  who adult_male deck  embark_town alive alone
count     891  891        891  203          889   891   891
unique      3    3          2    7            3     2     2
top     Third  man       True    C  Southampton    no  True
freq      491  537        537   59          644   549   537
mean      NaN  NaN        NaN  NaN          NaN   NaN   NaN
std       NaN  NaN        NaN  NaN          NaN   NaN   NaN
min       NaN  NaN        NaN  NaN          NaN   NaN   NaN
25%       NaN  NaN        NaN  NaN          NaN   NaN   NaN
50%       NaN  NaN        NaN  NaN          NaN   NaN   NaN
75%       NaN  NaN        NaN  NaN          NaN   NaN   NaN
max       NaN  NaN        NaN  NaN          NaN   NaN   NaN
Shape setelah clean: (891, 15)
class
First     0.629630
Second    0.472826
Third     0.242363
Name: survived, dtype: float64
sex
female    0.742038
male      0.188908
Name: survived, dtype: float64
survived
0    30.028233
1    28.291433
Name: age, dtype: float64
Selesai → output_eda
```




Age by Class
80
70
60
50
40
30
20
10
0
age
First
Second
Third
class


Survival Rate by Sex
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0
survived
male
female
sex


Count of class
500
400
300
200
100
0
count
First
Second
Third
class



Catatan: proyek EDA ini memakai dataset seaborn bawaan (diunduh dari GitHub seaborn-data) sehingga hasil dijalankan dengan data nyata; `time.sleep()` *untuk jeda server dipersingkat saat pengujian.*

TIPS

Project 4 — menaikkan kualitas

Rumuskan 2–3 pertanyaan analisis sebelum plotting (bukan plot dulu, cerita kemudian).

Setiap figure disebut di insight.txt (jangan biarkan plot “yatim”).

Jika memakai penguins: bandingkan body_mass antar species/island; korelasi bill_length vs flipper_length.

7. Troubleshooting

```
# Plot tidak muncul di .py → plt.show() atau savefig
# Gambar terpotong → tight_layout sebelum save
# requests Timeout / SSL → cek jaringan; timeout lebih panjang; User-Agent
# HTTP 429 → kurangi frekuensi; sleep antar request
# feedparser kosong → cek status_code dan cuplikan r.text
# BeautifulSoup kosong → selector salah; coba parser lxml / html.parser
```

OUTPUT: *TIDAK DIJALANKAN* — HANYA KOMENTAR (DAFTAR JEBAKAN); CONTOH ERROR NYATA DIJALANKAN DI BAGIAN "DEMONSTRASI ERROR" DI BAWAH

Demonstrasi error — output nyata

Daftar jebakan di atas berupa komentar. Berikut setiap kasusnya dijalankan nyata. Kasus `requests` memakai server HTTP lokal sederhana agar error aslinya muncul dan dapat diulang tanpa internet.

Persiapan — server uji lokal (agar error `requests` asli dan dapat diulang tanpa internet):

```
import threading, time, json
from http.server import BaseHTTPRequestHandler, HTTPServer

class H(BaseHTTPRequestHandler):
    def log_message(self, *a): pass
    def do_GET(self):
        if self.path == "/html":
            body, ctype, code = "<html><body>Halaman, bukan JSON</body></html>", "text/html", 200
        elif self.path == "/cuaca-error":
            body, ctype, code = json.dumps({"error": True, "reason": "Parameter salah"}),
"application/json", 200
        elif self.path == "/lambat":
            time.sleep(2); body, ctype, code = "{}", "application/json", 200
        elif self.path == "/limit":
            body, ctype, code = "Too Many Requests", "text/plain", 429
        else:
            body, ctype, code = "tidak ada", "text/plain", 404
        self.send_response(code); self.send_header("Content-Type", ctype); self.end_headers()
        self.wfile.write(body.encode())

server = HTTPServer(("127.0.0.1", 8765), H)
threading.Thread(target=server.serve_forever, daemon=True).start()
import requests
BASE = "http://127.0.0.1:8765"
```

Matplotlib — ukuran x dan y tidak sama

```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3], [10, 20])
```

OUTPUT:

```
  -> ValueError: x and y must have same first dimension, but have shapes (3,) and (2,)
```

Solusi — samakan panjang

```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3], [10, 20, 30])
print("OK:", len(plt.gca().lines), "garis")
```

OUTPUT:

```
OK: 1 garis
```

Plot tidak muncul di skrip `.py` — simpan dengan `savefig`

```
import matplotlib.pyplot as plt
from pathlib import Path
plt.plot([1, 2, 3])
plt.tight_layout()
plt.savefig("gambar.png", dpi=150, bbox_inches="tight")
print(Path("gambar.png").exists(), Path("gambar.png").stat().st_size > 0)
```

OUTPUT:

```
True True
```

Seaborn — nama kolom salah

```
import seaborn as sns
tips = sns.load_dataset("tips")
sns.histplot(data=tips, x="total")
```

OUTPUT:

```
  -> ValueError: Could not interpret value `total` for `x`. An entry with this name does not
appear in `data`.
```

Solusi — periksa nama kolom

```
import seaborn as sns
tips = sns.load_dataset("tips")
print(tips.columns.tolist())
sns.histplot(data=tips, x="total_bill")
print("OK")
```

OUTPUT:

```
['total_bill', 'tip', 'sex', 'smoker', 'day', 'time', 'size']
OK
```

requests — HTTP 404 dan `raise_for_status()`

```
r = requests.get(BASE + "/tidak-ada")
print(r.status_code, r.text)
r.raise_for_status()
```

OUTPUT:

```
404 tidak ada
  -> requests.exceptions.HTTPError: 404 Client Error: Not Found for url:
http://127.0.0.1:8765/tidak-ada
```

requests — HTTP 429 (terlalu sering)

```
r = requests.get(BASE + "/limit")
print(r.status_code, r.text)
# solusi: kurangi frekuensi, beri jeda time.sleep(...) antar request
```

OUTPUT:

```
429 Too Many Requests
```

requests — JSONDecodeError saat respons bukan JSON

```
r = requests.get(BASE + "/html")
print(r.status_code, r.headers["Content-Type"])
r.json()
```

OUTPUT:

```
200 text/html
  -> requests.exceptions.JSONDecodeError: Expecting value: line 1 column 1 (char 0)
```

requests — Timeout

```
requests.get(BASE + "/lambat", timeout=0.5)
```

OUTPUT:

```
  -> requests.exceptions.ReadTimeout: HTTPConnectionPool(host='127.0.0.1', port=8765): Read timed
out. (read timeout=0.5)
```

requests — ConnectionError (server tidak ada)

```
requests.get("http://127.0.0.1:9/", timeout=2)
```

OUTPUT:

```
  -> requests.exceptions.ConnectionError: HTTPConnectionPool(host='127.0.0.1', port=9): Max
retries exceeded with url: / (Caused by NewConnectionError("HTTPConnection(host='127.0.0.1',
port=9): Failed to establish a new connection: [Errno 111] Connection refused"))
```

Solusi — tangkap `RequestException`

```
try:
    requests.get(BASE + "/lambat", timeout=0.5)
except requests.exceptions.RequestException as e:
    print("Gagal:", type(e).__name__)
```

OUTPUT:

```
Gagal: ReadTimeout
```

KeyError — respons API berisi error, bukan data

```
data = requests.get(BASE + "/cuaca-error").json()
print(data["current_weather"])
```

OUTPUT:

```
  -> KeyError: 'current_weather'
```

Solusi — periksa key error lebih dulu

```
data = requests.get(BASE + "/cuaca-error").json()
if data.get("error"):
    print("API error:", data["reason"])
else:
    print(data["current_weather"])
```

OUTPUT:

```
API error: Parameter salah
```

BeautifulSoup — selector salah, `find()` mengembalikan None

```
from bs4 import BeautifulSoup
soup = BeautifulSoup("<div><span class='judul'>Halo</span></div>", "html.parser")
print(soup.find("span", class_="text").get_text())
```

OUTPUT:

```
  -> AttributeError: 'NoneType' object has no attribute 'get_text'
```

Solusi — periksa None dulu

```
from bs4 import BeautifulSoup
soup = BeautifulSoup("<div><span class='judul'>Halo</span></div>", "html.parser")
el = soup.find("span", class_="text")
print(el.get_text() if el else "Elemen tidak ditemukan")
```

OUTPUT:

```
Elemen tidak ditemukan
```

feedparser/arXiv — hasil kosong menyebabkan KeyError

```
import feedparser, pandas as pd
feed = feedparser.parse('<?xml version="1.0"?><feed xmlns="http://www.w3.org/2005/Atom"></feed>')
df = pd.DataFrame([{"title": e.title} for e in feed.entries])
print(df.shape)
print(df["title"])
```

OUTPUT:

```
(0, 0)
  -> KeyError: 'title'
```

Solusi — cek `df.empty`

```
import feedparser, pandas as pd
feed = feedparser.parse('<?xml version="1.0"?><feed xmlns="http://www.w3.org/2005/Atom"></feed>')
df = pd.DataFrame([{"title": e.title} for e in feed.entries])
if df.empty:
    print("Tidak ada hasil — cek query (all:kata) dan status_code / cuplikan r.text")
```

OUTPUT:

```
Tidak ada hasil — cek query (all:kata) dan status_code / cuplikan r.text
```

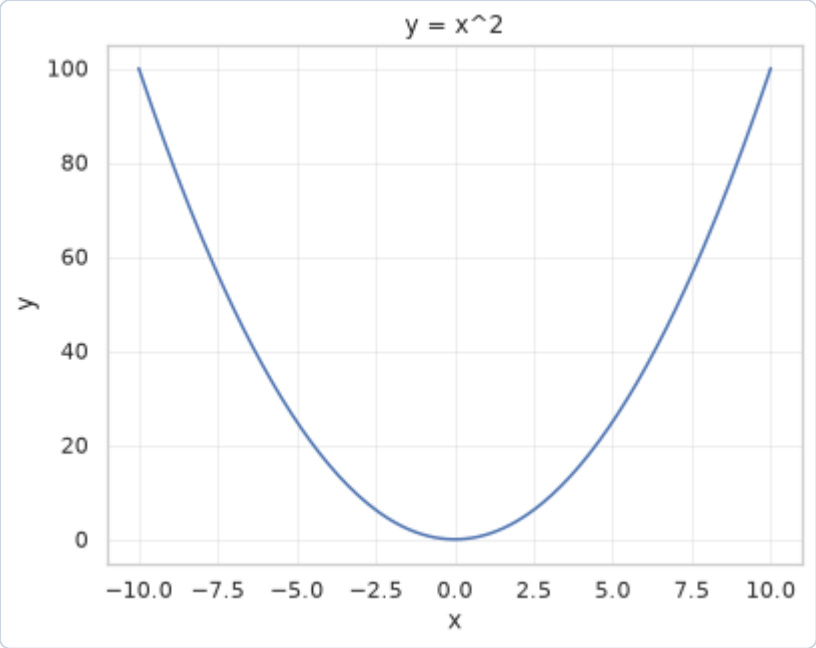
8. Solusi Latihan (Cuplikan)

```
import matplotlib.pyplot as plt
import numpy as np
x = np.linspace(-10, 10, 200)
plt.plot(x, x**2)
plt.title("y = x^2"); plt.xlabel("x"); plt.ylabel("y")
plt.grid(True, alpha=0.3); plt.show()

import requests, pandas as pd
r = requests.get("https://jsonplaceholder.typicode.com/comments", timeout=15)
df = pd.DataFrame(r.json())
print(df.groupby("postId").size().head())
```

OUTPUT:

```
postId
1    5
2    5
3    5
4    5
5    5
dtype: int64
```



Catatan: host jsonplaceholder.typicode.com tidak dapat diakses dari lingkungan pengujian (kebijakan jaringan), sehingga kode dijalankan dengan respons tiruan yang bentuknya sama dengan API asli. ***Nilai/teks pada output ini contoh, bukan data nyata.***

9. Checklist Sebelum Modul 6

- Beberapa jenis plot Matplotlib & Seaborn sudah dicoba
- requests → JSON → DataFrame berhasil
- arXiv atau parsing HTML contoh pernah dijalankan
- Minimal 2 mini project + figure tersimpan
- Siap Modul 6 — Capstone

> **Modul 5 selesai? Lanjut ke Modul 6 — Capstone.**
>
> Python Intensive — Modul Belajar Mandiri

6

Modul 6 — Capstone Project

Proyek akhir mandiri · Integrasi Modul 1–5 · Dokumentasi portfolio ·
Estimasi: ± 6–10 jam (tergantung scope) · Prasyarat: Modul 1–5

1. Apa Itu Capstone di Modul Ini
2. Tujuan Pembelajaran
3. Memilih Topik
4. Deliverable Wajib
5. Alur Kerja yang Disarankan
5b. Agenda Sehari & Alokasi Waktu
6. Struktur Folder Proyek
7. Template Dokumentasi Capstone
8. Skeleton Kode Capstone (Pola Umum)
9. Empat Jalur Capstone (Pilih Satu)
10. Checklist Kualitas (Self-Review)
11. Troubleshooting Capstone
11b. Demonstrasi Error — Output Nyata
12. Setelah Capstone

> **Catatan pengujian:** blok kode pada modul ini dijalankan (Python 3.11, pandas 3.0.6, matplotlib 3.11.2, seaborn 0.13.2). Modul 6 versi ini berisi kerangka (skeleton), sehingga ditambahkan **contoh lengkap** yang mengisi skeleton untuk Jalur A dan "Demonstrasi error" untuk kasus teknis pada Troubleshooting. Contoh lengkap keempat jalur ada di `contoh-capstone/`. Folder struktur dan template README pada modul asli berlabel `python`; di sini dilabeli `text` / `markdown` agar terbaca benar.

1. Apa Itu Capstone di Modul Ini

Capstone adalah proyek penutup: Anda memilih satu masalah, lalu menyelesaikan alur end-to-end dengan keterampilan yang sudah dipelajari — dari struktur data, function, dan OOP hingga Pandas, visualisasi, dan (bila relevan) API.

Bukan ujian pilihan ganda. Yang penting adalah kejelasan masalah, kualitas data dan kode, ketepatan analisis, visualisasi yang mendukung argumen, serta dokumentasi yang bisa dipahami orang lain.

> CATATAN
> **Prinsip**
>
> Lebih baik scope kecil yang selesai rapi daripada ide besar yang setengah jadi.
>
> Setiap figure dan tabel harus punya alasan (menjawab pertanyaan analisis).
>
> Simpan semua artefak di satu folder proyek agar siap dimasukkan portfolio.

2. Tujuan Pembelajaran

- Merumuskan pertanyaan analisis yang dapat dijawab dengan data.
- Menjalankan pipeline: data → cleaning → analisis → visualisasi → kesimpulan.
- Menulis kode terstruktur (function/module secukupnya) dan dapat dijalankan ulang.
- Menyusun dokumentasi singkat yang layak portfolio.

3. Memilih Topik

3.1 Kriteria topik yang sehat

Topik yang baik punya data yang bisa diakses, pertanyaan yang jelas, dan hasil yang bisa divisualisasikan. Hindari topik yang membutuhkan data rahasia, scraping agresif, atau model ML berat jika Anda belum mempelajarinya di modul ini.

3.2 Ide yang disarankan

Tema	Sumber data contoh	Pertanyaan contoh
Nilai / akademik	CSV sendiri / generate	Faktor apa terkait IPK tinggi?
Dataset Seaborn	titanic, tips, penguins	Siapa lebih mungkin survive?
API publik	Open-Meteo, JSONPlaceholder	Bagaimana pola suhu 7 hari?
Literatur arXiv	arXiv API	Topik apa sering di keyword X?
Operasional mini	CSV inventori / perpustakaan	Item apa sering kosong?

TIPS

Tips memilih

Kaitkan dengan jurusan Anda bila memungkinkan (lebih bermakna di portfolio).

Pastikan Anda bisa menjelaskan data kepada orang awam dalam 2 menit.

Jika data sulit didapat dalam 1 jam, ganti sumber — jangan habiskan waktu hanya mencari CSV.

4. Deliverable Wajib

1. Kode yang bisa dijalankan (script .py atau notebook .ipynb).
2. Data mentah dan/atau data bersih (CSV) — atau skrip yang mengunduh data.
3. Minimal 3 visualisasi bermakna (PNG) dengan judul dan label.
4. Dokumentasi proyek (template di bagian 7).
5. Ringkasan insight: minimal 5 poin + batasan analisis.

5. Alur Kerja yang Disarankan

1. Rumuskan 2–4 pertanyaan analisis (tulis di dokumentasi dulu).
2. Kumpulkan / muat data; inspeksi (head, info, missing).
3. Cleaning dan transformasi; simpan data_bersih.csv.
4. Analisis (groupby, merge, statistik deskriptif).
5. Visualisasi yang menjawab pertanyaan — bukan dekorasi.
6. Tulis insight + batasan; rapikan struktur folder.
7. Uji ulang: jalankan dari awal bila memungkinkan.

5b. Agenda Sehari & Alokasi Waktu

Waktu	Sesi	Durasi	Fokus
00:00–00:20	Briefing + Pemilihan Topik	20 mnt	Orientasi
00:20–01:00	Perencanaan (outline + struktur)	40 mnt	Design
01:00–04:00	Pengerjaan Capstone	180 mnt	Implementasi
04:00–04:20	Istirahat	20 mnt	—
04:20–05:20	Finalisasi + Dokumentasi	60 mnt	Polish
05:20–06:00	Showcase / Review	40 mnt	Presentasi

5b.1 Saran alokasi waktu saat Coding (180 menit)

Tahap	Waktu	Aktivitas
Load + Inspeksi	20 mnt	Baca data, cek shape/dtypes/missing/duplikat
Cleaning	25 mnt	Tangani missing, duplikat, tipe data
Analisis	40 mnt	Groupby, pivot, korelasi, filter, ranking
Visualisasi	45 mnt	Minimal 4–5 grafik, judul & label jelas
Insight + Export	25 mnt	Tulis insight, simpan CSV/Excel
Buffer / Perbaikan	25 mnt	Cadangan jika ada error

> **Tips Time Management**
>
> Jangan terlalu lama di tahap cleaning. Cleaning yang cukup baik lebih baik daripada sempurna tapi kehabisan waktu.
>
> Kerjakan visualisasi paling penting dulu.
>
> Simpan file sesering mungkin.

5b.2 Ekspektasi kualitas kode

> **Standar Kode yang Diharapkan**
>
> Nama variabel dan fungsi bermakna.
>
> Minimal ada beberapa function (atau class) — jangan semua kode di top-level.
>
> Ada docstring singkat pada function/class utama.
>
> Mengikuti PEP 8 secara wajar.
>
> Ada error handling pada bagian yang berisiko (baca file, panggil API).
>
> Kode dapat dijalankan dari atas ke bawah tanpa error.

5b.3 Rekomendasi dataset & sumber data

Dataset / Sumber	Cocok untuk	Keterangan
Student Performance	Jalur A	Kaggle / UCI — nilai, gender, pendidikan ortu
Titanic	Jalur A	Kaggle — klasik untuk EDA
Iris / Penguins	Jalur A	Seaborn bawaan — kecil dan bersih
Gapminder	Jalur A	Data pembangunan dunia
JSONPlaceholder	Jalur B	API palsu — users, posts
Open-Meteo	Jalur B	API cuaca gratis tanpa key
arXiv API	Jalur B	Metadata paper ilmiah — cocok S2/S3
Data nilai buatan sendiri	Jalur D	Buat CSV sendiri atau generate

6. Struktur Folder Proyek

```
capstone_nama_anda/
├── README.md
├── requirements.txt
├── data/
│   ├── raw/
│   └── processed/
├── src/                    # atau notebooks/
│   ├── 01_load_clean.py
│   ├── 02_analyze.py
│   └── 03_visualize.py
├── figures/
└── output/
    ├── ringkasan.csv
    └── insight.txt
```

Tidak wajib memecah 3 file Python; satu notebook rapi juga diterima. Yang penting alurnya jelas.

7. Template Dokumentasi Capstone

Salin struktur berikut ke README.md dan isi dengan kata Anda sendiri.

```
# Judul Proyek

## 1. Latar belakang & motivasi
## 2. Pertanyaan analisis
## 3. Data (sumber, ukuran, cara dapat, etika)
## 4. Metode (cleaning, analisis, tools)
## 5. Hasil utama (temuan + rujukan figure)
## 6. Visualisasi (daftar figure + arti singkat)
## 7. Batasan
## 8. Kesimpulan & kerja lanjutan
## 9. Cara menjalankan
## 10. Struktur folder (opsional)
```

8. Skeleton Kode Capstone (Pola Umum)

Sesuaikan nama function dan kolom dengan topik Anda.

```python
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

ROOT = Path(__file__).resolve().parent.parent
DATA_RAW = ROOT / "data" / "raw"
DATA_OUT = ROOT / "data" / "processed"
FIG = ROOT / "figures"
OUT = ROOT / "output"
for d in [DATA_OUT, FIG, OUT]:
    d.mkdir(parents=True, exist_ok=True)

def load_data():
    # pd.read_csv / sns.load_dataset / requests
    raise NotImplementedError

def clean(df):
    df = df.copy()
    # missing, tipe, filter, feature
    return df

def analyze(df):
    hasil = {}
    # groupby / agregasi → simpan di hasil
    return hasil

def visualize(df, hasil):
    # minimal 3 plot; savefig ke FIG
    pass

def tulis_insight(hasil, path):
    path.write_text("1. ...\n2. ...\n\nBatasan: ...", encoding="utf-8")

def main():
    df0 = load_data()
    df = clean(df0)
    df.to_csv(DATA_OUT / "data_bersih.csv", index=False)
    hasil = analyze(df)
    visualize(df, hasil)
    tulis_insight(hasil, OUT / "insight.txt")
    print("Selesai.")

if __name__ == "__main__":
    main()
```

OUTPUT (SKELETON DIJALANKAN APA ADANYA SEBAGAI `SRC/MAIN.PY`):

```
Traceback (most recent call last):
  File "capstone_contoh/src/main.py", line 45, in <module>
    main()
  File "capstone_contoh/src/main.py", line 36, in main
    df0 = load_data()
  File "capstone_contoh/src/main.py", line 16, in load_data
    raise NotImplementedError
NotImplementedError
```

Catatan: ini perilaku yang benar — skeleton sengaja memanggil `raise NotImplementedError` pada `load_data()` sampai Anda mengisinya. Perhatikan juga bahwa `Path(__file__)` hanya valid di skrip `.py`; di notebook pakai `Path.cwd()` (lihat Demonstrasi error).

Contoh lengkap — skeleton diisi untuk Jalur A (Titanic)

Berikut skeleton yang sama dengan semua function terisi (data: `sns.load_dataset("titanic")`, diunduh dari seaborn-data). Dijalankan sebagai `src/main.py` pada folder proyek kosong.

```python
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

ROOT = Path(__file__).resolve().parent.parent
DATA_RAW = ROOT / "data" / "raw"
DATA_OUT = ROOT / "data" / "processed"
FIG = ROOT / "figures"
OUT = ROOT / "output"
for d in [DATA_OUT, FIG, OUT]:
    d.mkdir(parents=True, exist_ok=True)

def load_data():
    raw = DATA_RAW / "titanic.csv"
    if raw.exists():
        return pd.read_csv(raw)
    DATA_RAW.mkdir(parents=True, exist_ok=True)
    df = sns.load_dataset("titanic")      # unduh dari seaborn-data
    df.to_csv(raw, index=False)           # simpan mentah sejak request pertama
    return df

def clean(df):
    df = df.copy()
    df["age"] = df["age"].fillna(df.groupby(["pclass", "sex"])["age"].transform("median"))
    df["embarked"] = df["embarked"].fillna(df["embarked"].mode()[0])
    df = df.drop(columns=["deck", "alive", "class", "embark_town"])
    df["family_size"] = df["sibsp"] + df["parch"] + 1
    df["age_group"] = pd.cut(df["age"], [0, 12, 18, 35, 60, 100],
                             labels=["anak", "remaja", "dewasa muda", "dewasa", "lansia"])
    return df

def analyze(df):
    hasil = {}
    hasil["sex"] = df.groupby("sex")["survived"].mean()
    hasil["kelas"] = df.groupby("pclass")["survived"].mean()
    hasil["usia"] = df.groupby("age_group", observed=True)["survived"].mean()
    hasil["keluarga"] = df.groupby("family_size")["survived"].agg(["count", "mean"])
    hasil["pivot"] = df.pivot_table(values="survived", index="sex", columns="pclass",
aggfunc="mean")
    hasil["median_usia"] = df.groupby("survived")["age"].median()
    return hasil

def visualize(df, hasil):
    sns.set_theme(style="whitegrid")

    plt.figure(figsize=(6, 4))
    sns.barplot(data=df, x="pclass", y="survived", hue="sex", errorbar=None)
    plt.title("Peluang selamat per kelas dan jenis kelamin")
    plt.xlabel("Kelas tiket"); plt.ylabel("Proporsi selamat")
    plt.tight_layout(); plt.savefig(FIG / "01_kelas_sex.png", dpi=120); plt.close()

    plt.figure(figsize=(6, 4))
    sns.histplot(data=df, x="age", hue="survived", bins=25, multiple="stack")
    plt.title("Distribusi usia berdasarkan keselamatan")
    plt.xlabel("Usia"); plt.ylabel("Jumlah penumpang")
    plt.tight_layout(); plt.savefig(FIG / "02_usia.png", dpi=120); plt.close()

    plt.figure(figsize=(6, 4))
    hasil["usia"].plot(kind="bar", color="steelblue")
    plt.title("Peluang selamat per kelompok usia")
```

```
    plt.xlabel("Kelompok usia"); plt.ylabel("Proporsi selamat"); plt.xticks(rotation=20)
    plt.tight_layout(); plt.savefig(FIG / "03_kelompok_usia.png", dpi=120); plt.close()

def tulis_insight(hasil, path):
    s, k, u = hasil["sex"], hasil["kelas"], hasil["usia"]
    baris = [
        f"1. Perempuan selamat {s['female']:.0%} vs laki-laki {s['male']:.0%} (lihat figure 01).",
        f"2. Kelas 1 selamat {k[1]:.0%}, kelas 3 hanya {k[3]:.0%} (figure 01).",
        f"3. Anak-anak selamat {u['anak']:.0%}, dewasa muda {u['dewasa muda']:.0%} (figure 03).",
        f"4. Median usia: selamat {hasil['median_usia'][1]:.0f} tahun vs tidak selamat
{hasil['median_usia'][0]:.0f} tahun (figure 02).",
        f"5. Penumpang sendirian selamat {hasil['keluarga'].loc[1, 'mean']:.0%} dibanding keluarga
2 orang {hasil['keluarga'].loc[2, 'mean']:.0%}.",
        "",
        "Batasan: data observasional satu kejadian; usia diimputasi (median per kelas & jenis
kelamin); "
        "tidak ada ID penumpang sehingga tidak bisa dikaitkan antar anggota keluarga.",
    ]
    path.write_text("\n".join(baris), encoding="utf-8")

def main():
    df0 = load_data()
    df = clean(df0)
    df.to_csv(DATA_OUT / "data_bersih.csv", index=False)
    hasil = analyze(df)
    visualize(df, hasil)
    tulis_insight(hasil, OUT / "insight.txt")
    hasil["pivot"].round(3).to_csv(OUT / "ringkasan.csv")
    print("Selesai.")

if __name__ == "__main__":
    main()
```

OUTPUT:

```
Selesai.
```

Berkas yang terbentuk setelah dijalankan (sesuai struktur folder bagian 6):

```
data/processed/data_bersih.csv
data/raw/titanic.csv
figures/01_kelas_sex.png
figures/02_usia.png
figures/03_kelompok_usia.png
output/insight.txt
output/ringkasan.csv
src/main.py
```

`OUTPUT/INSIGHT.TXT`:

```
1. Perempuan selamat 74% vs laki-laki 19% (lihat figure 01).
2. Kelas 1 selamat 63%, kelas 3 hanya 24% (figure 01).
3. Anak-anak selamat 58%, dewasa muda 36% (figure 03).
4. Median usia: selamat 27 tahun vs tidak selamat 25 tahun (figure 02).
5. Penumpang sendirian selamat 30% dibanding keluarga 2 orang 55%.

Batasan: data observasional satu kejadian; usia diimputasi (median per kelas & jenis kelamin);
tidak ada ID penumpang sehingga tidak bisa dikaitkan antar anggota keluarga.
```

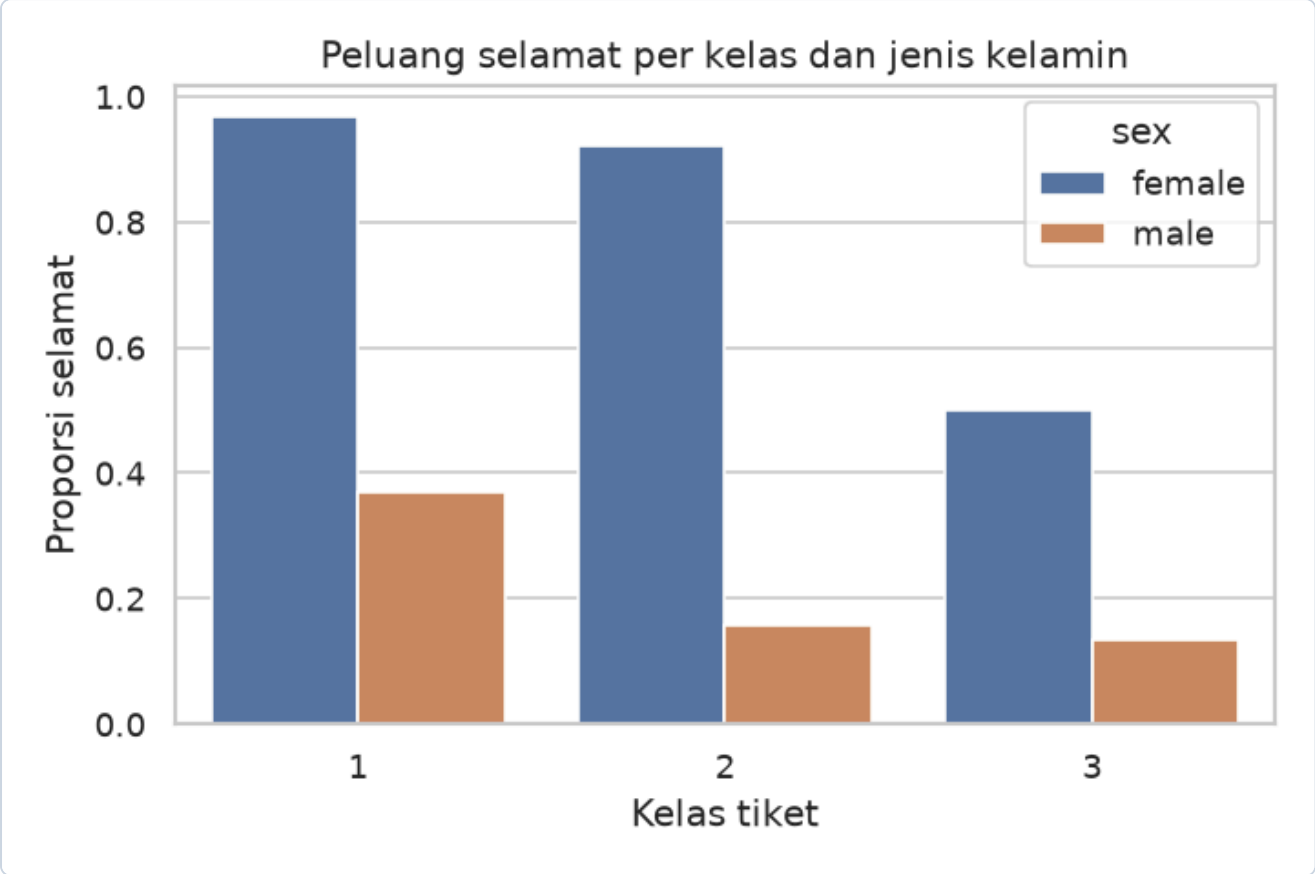
`OUTPUT/RINGKASAN.CSV` **(PIVOT PELUANG SELAMAT: JENIS KELAMIN × KELAS):**

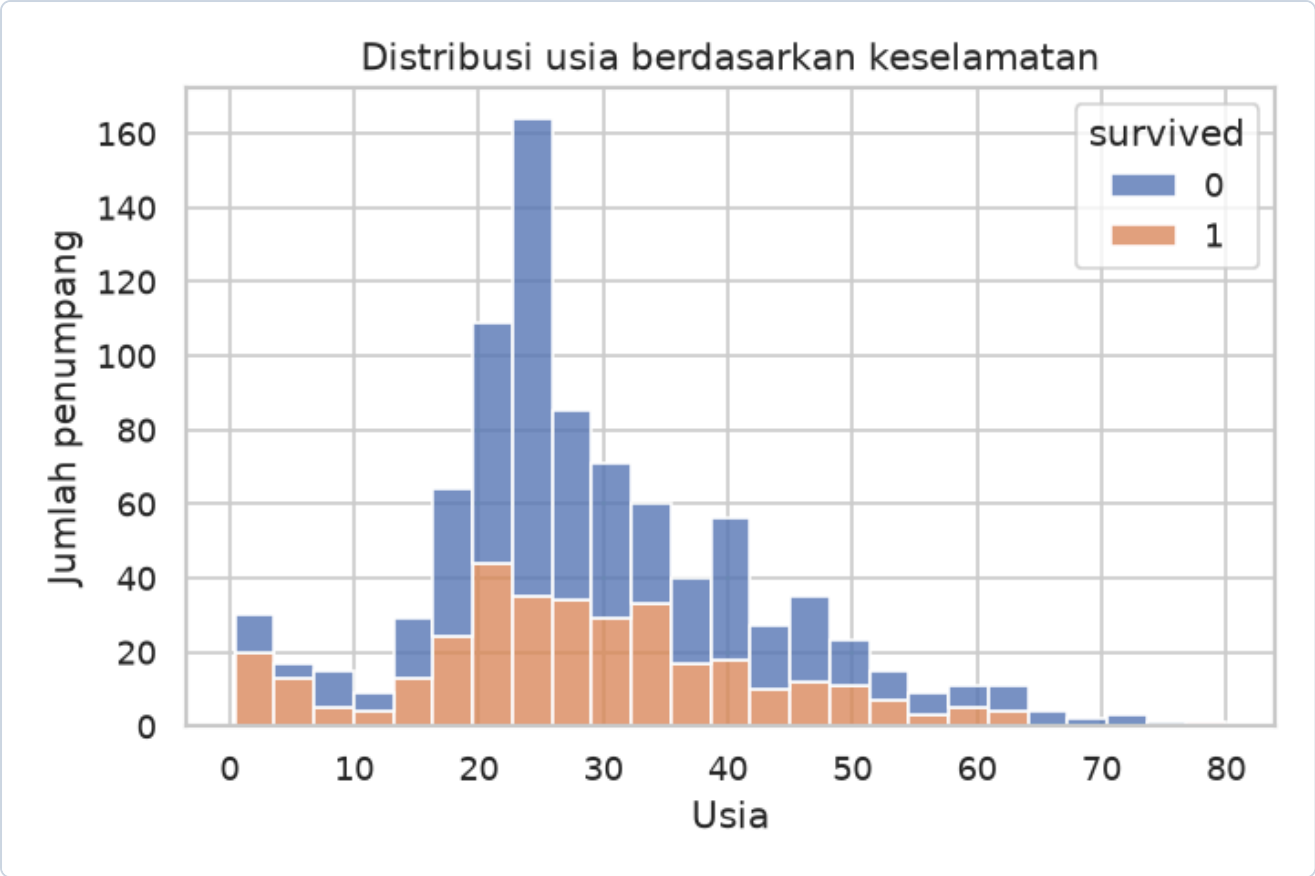
```
sex,1,2,3
female,0.968,0.921,0.5
male,0.369,0.157,0.135
```

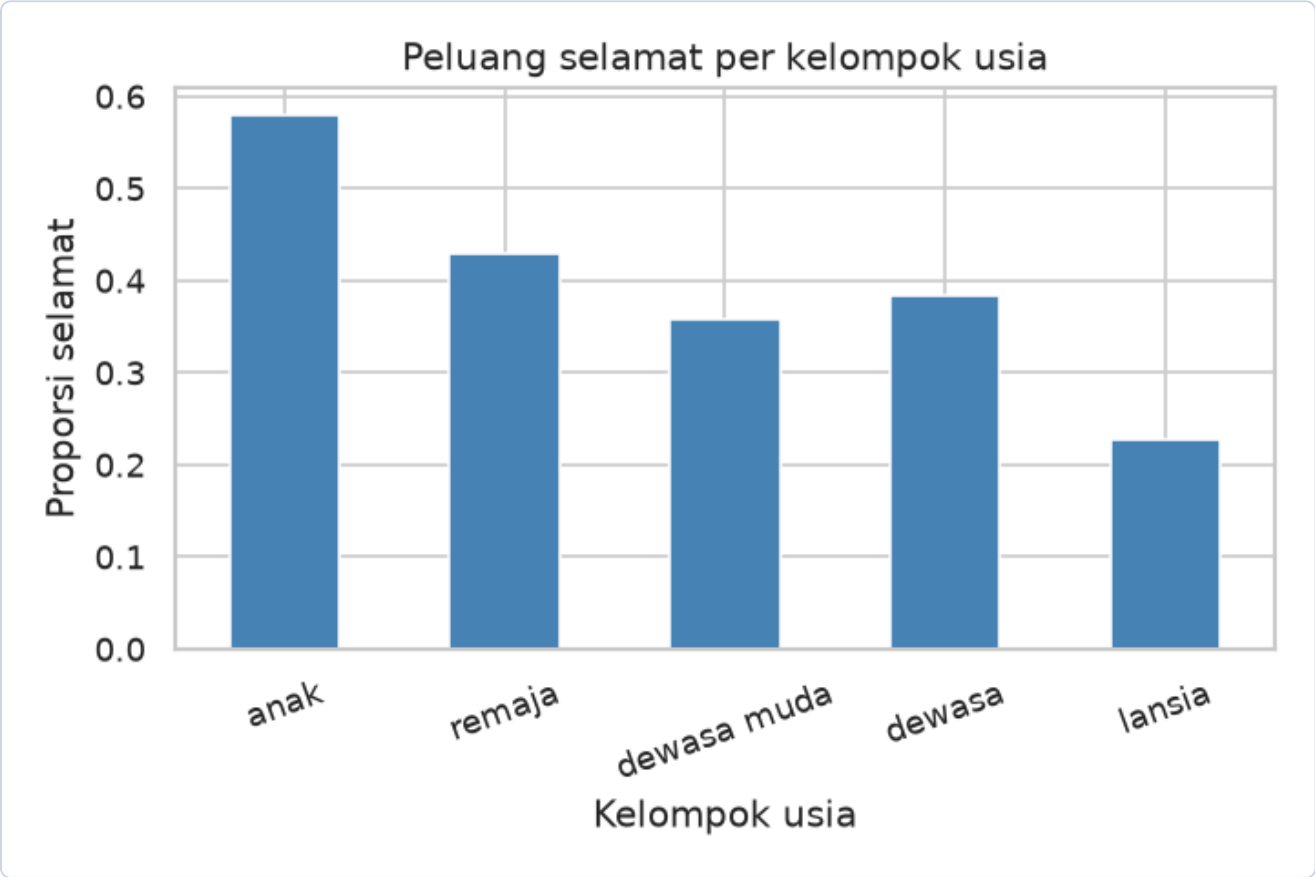
`data/processed/data_bersih.csv` (4 baris pertama):

```
survived,pclass,sex,age,sibsp,parch,fare,embarked,who,adult_male,alone,family_size,age_group
0,3,male,22.0,1,0,7.25,S,man,True,False,2,dewasa muda
1,1,female,38.0,1,0,71.2833,C,woman,False,False,2,dewasa
1,3,female,26.0,0,0,7.925,S,woman,False,True,1,dewasa muda
```

Tiga grafik yang dihasilkan di folder `figures/` :




Distribusi usia berdasarkan keselamatan
survived
0
1
Jumlah penumpang
0
20
40
60
80
100
120
140
160
0
10
20
30
40
50
60
70
80
Usia


Peluang selamat per kelompok usia
Proporsi selamat
0.0
0.1
0.2
0.3
0.4
0.5
0.6
anak
remaja
dewasa muda
dewasa
lansia
Kelompok usia

Uji ulang dari awal (checklist "kode berjalan dari awal"): menjalankan `src/main.py` untuk kedua kalinya memakai data mentah yang sudah tersimpan dan menghasilkan `insight.txt` yang identik dengan sebelumnya:

```
Selesai.
```

Catatan: ini hanya contoh bentuk pengerjaan. Contoh capstone lengkap untuk keempat jalur (dengan dokumentasi dan output) ada di `contoh-capstone/` *.*

9. Empat Jalur Capstone (Pilih Satu)

Jalur A — EDA Dataset Publik (paling stabil)

Pilih titanic / penguins / tips / CSV publik. Fokus pada pertanyaan, cleaning, groupby, dan 4–6 visualisasi yang saling terkait. Cocok jika koneksi API tidak andal.

Scope minimum

1. 3 pertanyaan analisis tertulis.
2. Cleaning terdokumentasi + data_bersih.csv.
3. Minimal 3 groupby/agregasi berbeda.
4. Minimal 4 figure + insight 5 poin + batasan.

Jalur B — API → Analisis → Visualisasi

Ambil data dari API publik yang legal. Simpan mentah, proses, visualisasikan tren atau perbandingan. Sertakan error handling dan catatan waktu pengambilan.

Scope minimum

1. Function fetch dengan timeout dan penanganan gagal.
2. CSV mentah + CSV olahan.
3. Minimal 3 figure; insight mengaitkan angka dengan konteks.

Jalur C — arXiv / literatur + ringkasan kuantitatif

Pipeline pencarian paper terkait bidang studi: paginasi, metadata, analisis waktu dan kata kunci, visualisasi, interpretasi untuk audiens akademik.

Scope minimum

1. Minimal sekitar 20 paper (paginasi + sleep antar request).
2. Analisis per tahun + frekuensi kata judul (atau penulis top).
3. 2–3 figure + CSV + pembahasan batasan (bias query, metadata saja).

Jalur D — Aplikasi Analisis Nilai Berbasis OOP (level tinggi)

Bangun mini sistem analisis data akademik yang memanfaatkan class/OOP, Pandas, dan visualisasi.

Scope minimum

1. Minimal satu class untuk mengelola data.
2. Fitur: tambah data, statistik, filter, ranking.
3. Minimal 4 visualisasi.
4. Export laporan ke CSV/Excel.
5. Kode modular dan mudah dibaca.

Stretch goals (bonus)

1. Method perbandingan dua prodi/kelas.
2. Mode interaktif sederhana (input terminal).
3. Simpan dan load state aplikasi.

Skeleton kode

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

class AnalisisNilai:
    def __init__(self, data_path=None):
        self.df = pd.read_csv(data_path) if data_path else pd.DataFrame()

    def tambah_mahasiswa(self, nama, prodi, nilai):
        pass

    def statistik(self):
        pass

    def ranking(self, n=10):
        pass

    def visualisasi(self):
        pass

    def export(self, path):
        self.df.to_csv(path, index=False)

app = AnalisisNilai("nilai.csv")
print(app.statistik())
app.visualisasi()
app.export("laporan_akhir.csv")
```

OUTPUT:

```
None
```

Catatan: dijalankan dengan `nilai.csv` contoh (20 baris). Semua method masih `pass`, sehingga `statistik()` mengembalikan `None`, `visualisasi()` belum menggambar apa pun, dan `export()` menulis ulang CSV masukan.

Contoh lengkap (class `AnalisisNilai`, 7 unit test, mode interaktif, simpan/muat state): `contoh-capstone/jalur_d_analisis_nilai_oop/`.

10. Checklist Kualitas (Self-Review)

- Pertanyaan analisis tertulis — bukan hanya setelah plot jadi
- Kode berjalan dari awal tanpa error di mesin Anda
- Setiap figure punya judul, label, dan disebut di insight
- Missing values dan tipe data ditangani secara sadar
- Tidak ada klaim berlebihan di luar jangkauan data
- README bisa diikuti orang lain dalam ~10 menit
- requirements.txt atau daftar library lengkap

11. Troubleshooting Capstone

```
# Scope membengkak → potong jadi 2–3 pertanyaan; selesaikan dulu
# Plot tidak nyambung ke cerita → hapus plot; tulis ulang insight dari pertanyaan
# API gagal mendadak → simpan JSON/CSV mentah sejak request pertama berhasil
# Notebook berantakan → restart kernel; Run All; pecah section jelas
# Data terlalu kecil → jangan paksakan statistik berat; fokus deskriptif jujur
```

> **OUTPUT: *TIDAK DIJALANKAN* — HANYA KOMENTAR (DAFTAR MASALAH DAN CARA MENGATASINYA); KASUS TEKNIS YANG BISA DIREPRODUKSI DIJALANKAN DI "DEMONSTRASI ERROR" DI BAWAH.**

11b. Demonstrasi Error — Output Nyata

Bagian Troubleshooting Capstone di atas berupa daftar masalah. Berikut kasus teknis yang bisa direproduksi, dijalankan nyata (pandas 3.x). Masalah non-teknis (scope membengkak, plot tidak nyambung ke cerita, dst.) tidak dapat didemonstrasikan dengan kode.

`Path(__file__)` di notebook / REPL — `__file__` tidak ada

```
from pathlib import Path
ROOT = Path(__file__).resolve().parent.parent
```

OUTPUT:

```
  -> NameError: name '__file__' is not defined
```

Solusi — fallback ke folder kerja bila `__file__` tidak ada

```
from pathlib import Path
try:
    ROOT = Path(__file__).resolve().parent.parent
except NameError:          # notebook / REPL
    ROOT = Path.cwd()
print(ROOT.name != "")
```

OUTPUT:

```
True
```

Folder `figures/` belum dibuat saat `savefig`

```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3])
plt.savefig("figures/grafik.png")
```

OUTPUT:

```
  -> FileNotFoundError: [Errno 2] No such file or directory: 'figures/grafik.png'
```

Solusi — buat folder dulu (seperti pada skeleton)

```
import matplotlib.pyplot as plt
from pathlib import Path
FIG = Path("figures"); FIG.mkdir(parents=True, exist_ok=True)
plt.plot([1, 2, 3])
plt.savefig(FIG / "grafik.png")
print((FIG / "grafik.png").exists())
```

OUTPUT:

```
True
```

API gagal mendadak — tanpa data mentah tersimpan

```
import requests
r = requests.get("http://127.0.0.1:9/data", timeout=2)
```

OUTPUT:

```
  -> requests.exceptions.ConnectionError: HTTPConnectionPool(host='127.0.0.1', port=9): Max
retries exceeded with url: /data (Caused by NewConnectionError("HTTPConnection(host='127.0.0.1',
port=9): Failed to establish a new connection: [Errno 111] Connection refused"))
```

Solusi — simpan mentah sejak request pertama, fallback ke cache saat gagal

```
import json, requests
from pathlib import Path

CACHE = Path("data_mentah.json")

def ambil(url):
    try:
        r = requests.get(url, timeout=2)
        r.raise_for_status()
        CACHE.write_text(json.dumps(r.json()), encoding="utf-8")   # simpan mentah
        return r.json(), "api"
    except requests.exceptions.RequestException as e:
        print("API gagal:", type(e).__name__)
        if CACHE.exists():
            return json.loads(CACHE.read_text(encoding="utf-8")), "cache"
        return None, "gagal"

CACHE.write_text(json.dumps({"suhu": [30.1, 31.4]}), encoding="utf-8")   # anggap request pertama
pernah berhasil
print(ambil("http://127.0.0.1:9/data"))
```

OUTPUT:

```
API gagal: ConnectionError
({'suhu': [30.1, 31.4]}, 'cache')
```

`groupby` pada kolom teks salah — rata-rata kolom non-angka

```python
import pandas as pd
df = pd.DataFrame({"prodi": ["IF", "SI", "IF"], "kota": ["A", "B", "C"], "nilai": [80, 70, 90]})
print(df.groupby("prodi").mean())
```

OUTPUT:

```
  -> TypeError: dtype 'str' does not support operation 'mean'
```

Solusi — pilih kolom angka secara eksplisit

```python
import pandas as pd
df = pd.DataFrame({"prodi": ["IF", "SI", "IF"], "kota": ["A", "B", "C"], "nilai": [80, 70, 90]})
print(df.groupby("prodi")["nilai"].mean())
```

OUTPUT:

```
prodi
IF    85.0
SI    70.0
Name: nilai, dtype: float64
```

12. Setelah Capstone

Anda telah menempuh jalur dari fundamental Python hingga proyek data mandiri. Langkah alami berikutnya (di luar modul ini): Git/GitHub, pengujian sederhana, SQL, atau machine learning — sesuai tujuan studi dan karier.

Simpan folder capstone dan modul latihan di satu tempat. Proyek yang terdokumentasi sering lebih berharga di portfolio daripada daftar topik yang hanya “pernah dipelajari”.

> **Selamat — rangkaian Modul 1–6 selesai.**
>
> Python Intensive — Modul Belajar Mandiri